ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

CyberSecurity

3 - "Basic Cryptographic Concepts"

[Chapter 2]

Gabriele D'Angelo <g.dangelo@unibo.it>

https://www.unibo.it/sitoweb/g.dangelo/en

Outline

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Scope** of Cryptography
- **Basic** Operations
- **Cipher** and **Key**
- Example: **Caesar Cipher**
- **Symmetric** Key Encryption
- Modes of Operation
- **Stream** Encryption
- **Secure Hash** Functions
- **Public-Key** Encryption
- Digital Signatures
- Public-Key **Certificates**
- **Random Numbers** Generation
- **Post Quantum** Encryption (NOT IN THE BOOK: study using slides)

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Cryptography (intro)

Scope of Cryptography

Different Cybersecurity Properties

Different encryption tool will provide different properties (not all, only a subset)

The usage of **cryptographic techniques** is necessary for:

Most common property when we use Cryptography

- **Confidentiality**: "*only the right people **can read** the information*"
- **Authentication**: "*the ability to verify the **sender's identity***" (example, email can be forged: so no Auth property)
- **Integrity**: "*protecting information from **being modified** by unauthorized parties*" (example, email content can be modified: so no Integrity property)
- **Anonymity**: "ensuring the **users' anonymity** while communicating"

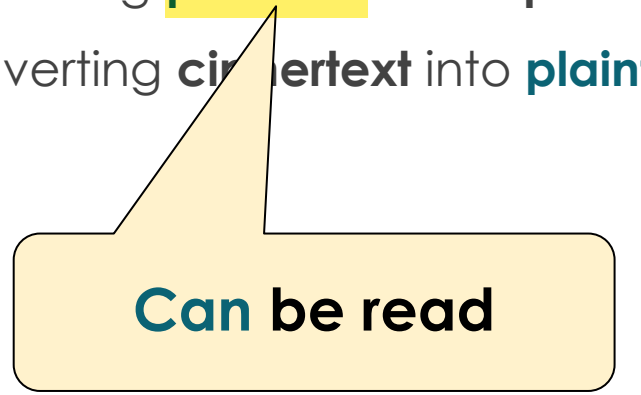
Basic Operations

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Encryption**: the process of converting **plaintext** into **ciphertext**
- **Decryption**: the process of converting **ciphertext** into **plaintext**

Basic Operations

- **Encryption**: the process of converting **plaintext** into **ciphertext**
- **Decryption**: the process of converting **ciphertext** into **plaintext**



Basic Operations

- **Encryption**: the process of converting **plaintext** into **ciphertext**
- **Decryption**: the process of converting **ciphertext** into **plaintext**

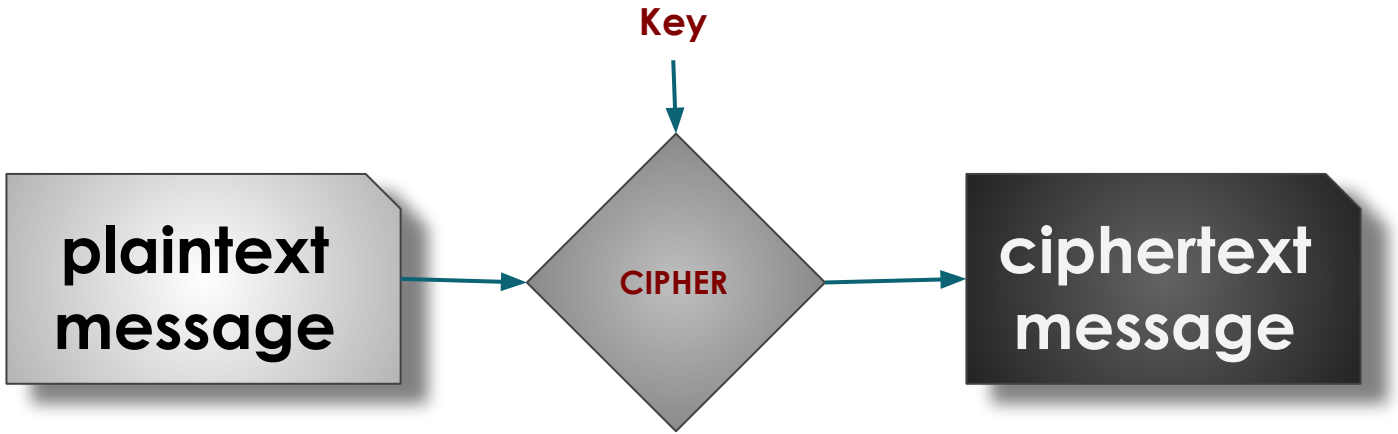
Can not be read
(not readable by attacker: for example, in a MITM attack, if the packets are encrypted the attacker cannot read it)

Basic Operations

- **Encryption**: the process of converting **plaintext** into **ciphertext**
- **Decryption**: the process of converting **ciphertext** into **plaintext**

Cipher and Key

- **Cipher**: an **algorithm** used to **encrypt** and **decrypt** data (must be secure and efficient)
- **Key**: a **piece of information** used by the cipher to encrypt the plaintext (and that is **necessary to decrypt the ciphertext**)



Example: Caesar Cipher

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- A very old example of cipher (i.e. Gaius Julius Caesar)
- It is a **substitution cipher** in which **each character in the message is substituted by another character**
- **For example**: "*substitute the letters in the second row for the letters in the top row to encrypt a message*"

Only letters are encrypted (not numbers and special character like '!' and '?')

```
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
    +3
D E F G H I J K L M N O P Q R S T U V W X Y Z A B C
```

Example: Caesar Cipher

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

The letters in the second row are shifted of a given number of positions

er (i.e. Gaius Julius Caesar)

which **each character in the message is**

acter

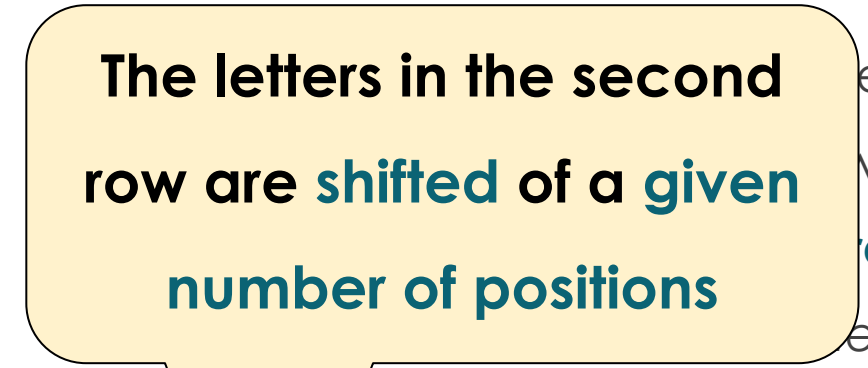
e letters in the second row for the letters in

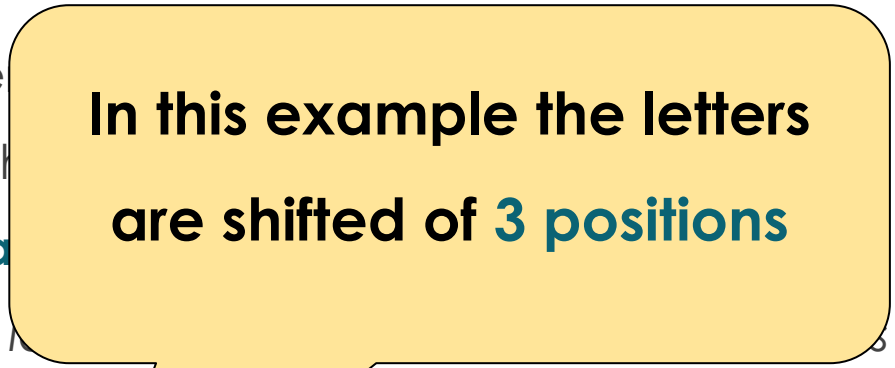
op row to encrypt a message"

A C D E F G H I J K L M N O P Q R S T U V W X Y Z

D E F G H I J K L M N O P Q R S T U V W X Y Z A B C

Example: Caesar Cipher





op row to encrypt a message

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

Example: Caesar Cipher

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

The letters in the second row are shifted of a given number of positions

In this example the letters are shifted of 3 positions

("3" is the secret key!)

...p row to encrypt a message

A C D E F G H I J K L M N O P Q R S T U V W X Y Z

D E F G H I J K L M N O P Q R S T U V W X Y Z A B C

Example: Caesar Cipher

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- A very old example of cipher (i.e. Gaius Julius Caesar)
- It is a substitution cipher in which **each character in the message is substituted with another character**
- **For example**, *"use the letters in the second row for the letters in the first row of the message"*

How many keys are possible in the Caesar Cipher?

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

Example: Caesar Cipher

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- A very old example of cipher (i.e. Julius Julius Caesar)
- It is a substitution cipher in which each character in the message is substituted by another character
- For example, with a shift of 3, all letters in the message

How many keys are possible in the Caesar Cipher?

Is it secure?

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

D E F G H I J K L M N O P Q R S T U V W X Y Z A B C

Example: Caesar Cipher

The "Big Enough" threshold in modern cryptography is generally considered to be 2^128 or higher to resist brute-force attacks from modern hardware. The Caesar Cipher's keyspace doesn't just fall short; it's practically non-existent by comparison.

- A very old example of cipher (i.e. Julius Julius Caesar)
- It is a substitution cipher in which each character in the message is substituted by another character
- For example, with a shift of 3, D would be replaced by A, E would become B, and so on for all letters in the message

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

D E F G H I J K L M N O P Q R S T U V W X Y Z A B C

The size of the key space must be sufficiently large to make a brute-force attack impractical. The concept of "sufficiently large" is not absolute, but evolves based on the computational power historically available to attackers.

How many keys are possible in the Caesar Cipher?

n-1 if n is the number of letters in the Alphabet

Is it secure?

No because it has a short number of keys

Let's play with it!

There is a third type of attack that is the Combination of Brute Force and Cryptanalytic Attack. If a cipher has a massive universe of keys, Brute Force is impossible. However, if Cryptanalysis can reveal even a tiny bit of information about the key, the math changes instantly (Reducing the Search Space).

Example: Dictionary & Rule-Based Attacks
When attacking passwords, hackers don't just try random characters (aaaa, aaab, aaac). That is pure Brute Force and it's too slow. Instead, they use a hybrid approach:
- Cryptanalysis (Pattern Study): They know humans use words, capitalize the first letter, and put "123" at the end.
- Brute Force (The Execution): They take a list of 100,000 common words and force only the variations that fit the human patterns.
- Why this is powerful: You are no longer searching the entire "Universe," you are only searching the "Likely Universe."

Caesar Cipher: Attacks

(The two types of attack are used to attack all the types of ciphers, not only the Caesar one)

How is possible to **attack** the Caesar Cipher? (*i.e.* ***to get the cleartext without having the secret key***)

It is normal that if i steal the key i can get the clear text: if we get the key, is not a failure of the Cipher, but a failure of the security of the device.

In Modern Ciphers, is difficult to do Brute Force Attacks (universe of keys and time required are too large)

Brute Force attack: **we can try all the possible keys**

- ***is it feasible?*** *It depends on the* ***number of keys*** *(i.e.* ***key size****) and the* ***time required to check each key*** *(... computers are fast!)*

The system designer selects an algorithm with a sufficiently large key space so that, given current computing power and its expected future development, the time required to test every key renders the attack impractical for the entire period during which the information must remain confidential

if the Brute Force attack is difficult to do or costly, we do this

Cryptanalytic attack: (in this case) we can use **the frequency of characters** (or other **weaknesses** of think kind)

that are in the Design of the Cipher

- *in human languages* ***each letter*** *has a* ***different frequency of use***
- *the Caesar Cipher translates letters and therefore* ***it translates also their frequency***

This is an example of how you can exploit vulnerabilities in the Design of the Cipher

For example, in English the letter E is the most used. So, if we see that a letter of the cipher text has the same frequency, that letter can be the E (so, we can find the key from the pattern of the language)

The Caesar Cipher provides zero confusion. It hides the letters, but it leaks the pattern of the language perfectly. This means an attacker doesn't even need to try all the keys

50% is not the probability of guessing correctly on the first try, but the probability that the key lies in the first half of your search. Since the key is chosen at random, there is no reason why it should necessarily be in the second half rather than the first. It's a coin toss: it's either in the first half or the second. So, you need to have half of the universe large enough

Caesar Cipher: Attacks

On average half of all possible keys must be tried to achieve success

How is possible to **attack** the Caesar Ciphe
without having the secret key)

- **Brute Force** attack: **we can try all the possible keys**
  - ***is it feasible?*** *It depends on the* ***number of keys*** *(i.e.* ***key size****) and the* ***time required to check each key*** *(... computers are fast!)*
- **Cryptanalytic** attack: (in this case) we can use **the frequency of characters** (or other **weaknesses** of think kind)
  - *in human languages* ***each letter*** *has a* ***different frequency of use***
  - *the Caesar Cipher translates letters and therefore* ***it translates also their frequency***

Caesar Cipher: Attacks

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

How is possible to **attack** the Caesar Cipher? (*i.e.* ***to get the cleartext without having the secret key***)

- **Brute Force** attack: **we can try all the possible keys**
  - ***is it feasible?*** *It depends on the* ***number of keys*** *(i.e.* ***key size****) and the* ***time required to check each key*** *(... computers are fast!)*
- **Cryptanalytic** attack: (in this case) we can use **the frequency of characters** (or other **weaknesses** of think kind)
  - *in human languages* ***each letter*** *has a* ***different frequency of use***
  - *the Caesar Cipher translates letters and therefore* ***it translates also their frequency***

Caesar Cipher: Attacks

Imagine that an attacker intercepts a message encrypted using the Caesar cipher and doesn't know the key.
- Count the letters: The attacker takes the ciphertext and counts how many times each letter appears. They construct a frequency chart.
- Find the peak: They see, for example, that in the ciphertext, the most frequent letter is H.
- Make a hypothesis: The attacker knows that the original text is in English, and knows that in English, the most frequent letter is E.
- So they hypothesize: E in plaintext corresponds to H in the ciphertext.
- Calculate the key: Count the alphabetical distance between E and H (it's 3 steps). The hypothesized key is +3.
- Verify: Try to decrypt the entire text by shifting back 3 positions. If meaningful words begin to appear, the attack was successful and the cipher has been broken.

How is possible to **attack** the C… …*eartext* ***without having the secret key***)

- **Brute Force** attack: **we car…**
  - ***is it feasible?*** *It depends on t…* …***e time required to check each key*** *(… compute…)*
- **Cryptanalytic** attack: (in this case) we can use … …**equency of characters** (or other **weaknesses** of think kind)
  - *in human languages* ***each letter*** *has a* ***different frequency of u…e***
  - *the Caesar Cipher translates letters and therefore* ***it translates also their frequency***

This can be exploited by the attacker!

How?

In cryptography, this means that the algorithm does not provide confusion: the statistical structure of the original message (the plaintext) remains perfectly visible in the ciphertext. AS MENTIONED IN THE PREVIOUS SLIDES

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Symmetric Encryption

Symmetric Key Encryption

It isn't obsolete.

- The universal technique (default and standard solution) for providing **confidentiality** for **transmitted** or **stored data**
- Two **requirements** for **secure use**:
  - a **strong encryption algorithm** against the three type of attacks that we saw
  - **sender** and **receiver** must have **obtained copies of the secret key** **in a secure fashion** and **must keep the key secure**

Symmetric Key Enc

Use Ciphers that are standards

Please remember the Kerckhoffs's principle:

"A cryptosystem should be secure even if everything about the system, except the key, is public knowledge"

- The universal technique

  stored data

- Two **requirements** for **secure use**

  - a **strong encryption algorithm**
  - **sender** and **receiver** must have **obtained copies of the secret key in a secure fashion** and **must keep the key secure**

Symmetric Key Encryption

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- The universal technique for providing **confidentiality** for **transmitted** or **stored data**
- Two **requirements** for **secure use**:
  - a **strong encryption algorithm**
  - **sender** and **receiver** must have **obtained copies of the secret key in a secure fashion** and **must keep the key secure**

Symmetric Key Encryption

onfidentiality for **transmitted** or

This is quite ridiculous: "*to get a secure communication channel you need a secure channel*"

to exchange the key to be used to encrypt/decrypt the message (to create a secure channel, i need a secure channel: BIG LIMITATION, is not suited to be used in the internet because if the destination is far away it is difficult to find a secure way to exchange key (Internet is insecure by nature, so attacker can get the key if they are listening))

- **sen** nd **receiver** must have **obtained copies of the secret key**

  in a secure fashion and **must keep the key secure**

Symmetric Key Encryption

- The universal technique f

  stored data

- Two **requirements** for **sec**
  - need a **strong encryp**
  - **sender** and **receiver** must have **obt** **copies of the secret key**

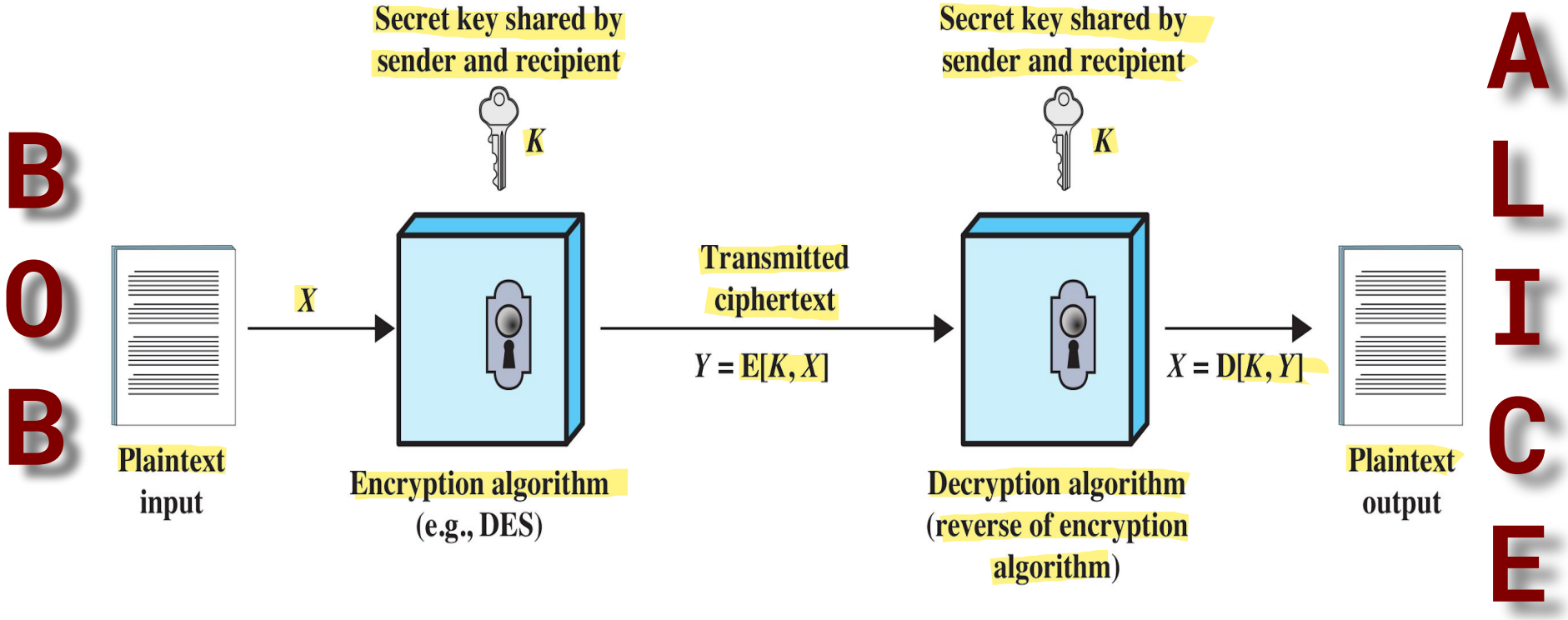
    in a secure fashion and **must keep the key secure**

We don't know if the key is leaked or not

The confidentiality of communications depends on the secrecy of the key

The security of symmetric key encryption depends entirely on the trustworthiness of both parties. It is also necessary to have complete trust in the recipient: if the recipient were compromised or acted maliciously, they could disclose the secret key to third parties, instantly compromising the confidentiality of the entire communication channel.

Simplified Model of Symmetric Encryption

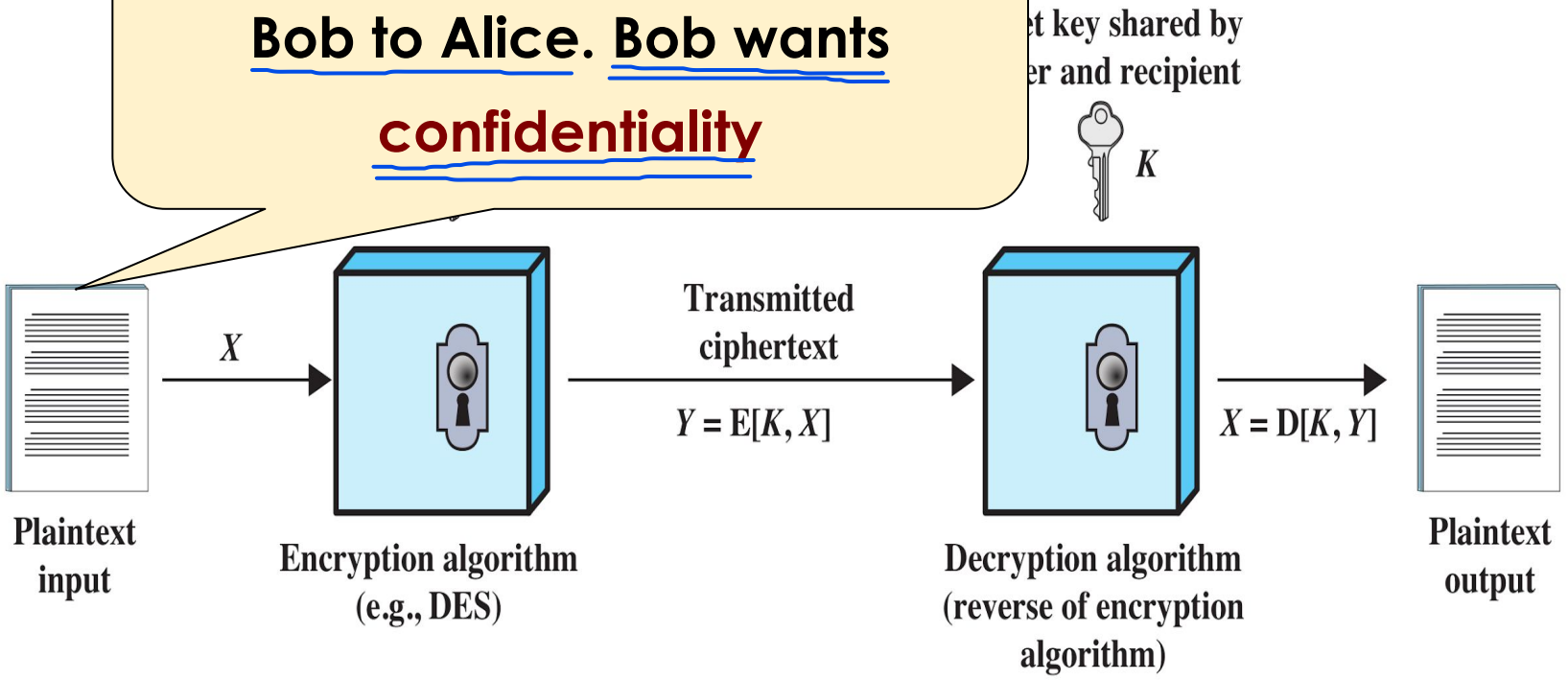


Simplified Model of Symmetric Encryption

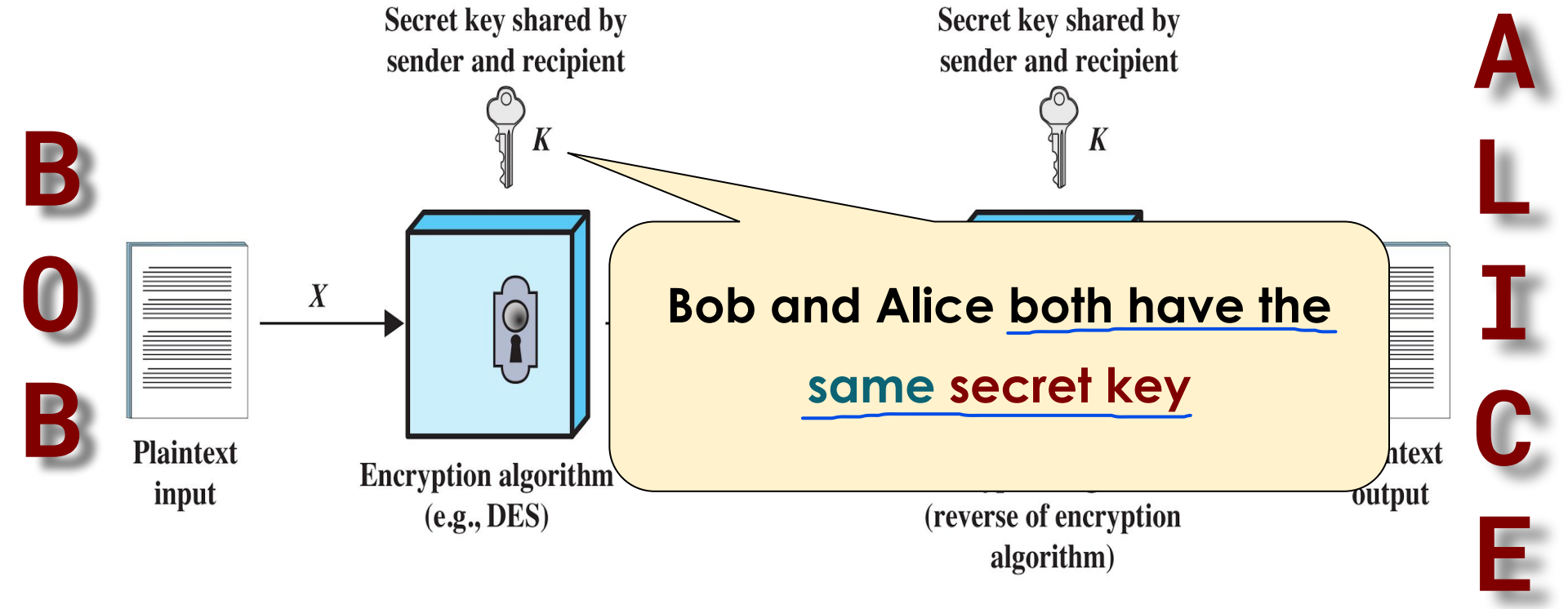
Secret message to be sent by Bob to Alice. Bob wants confidentiality

BOB

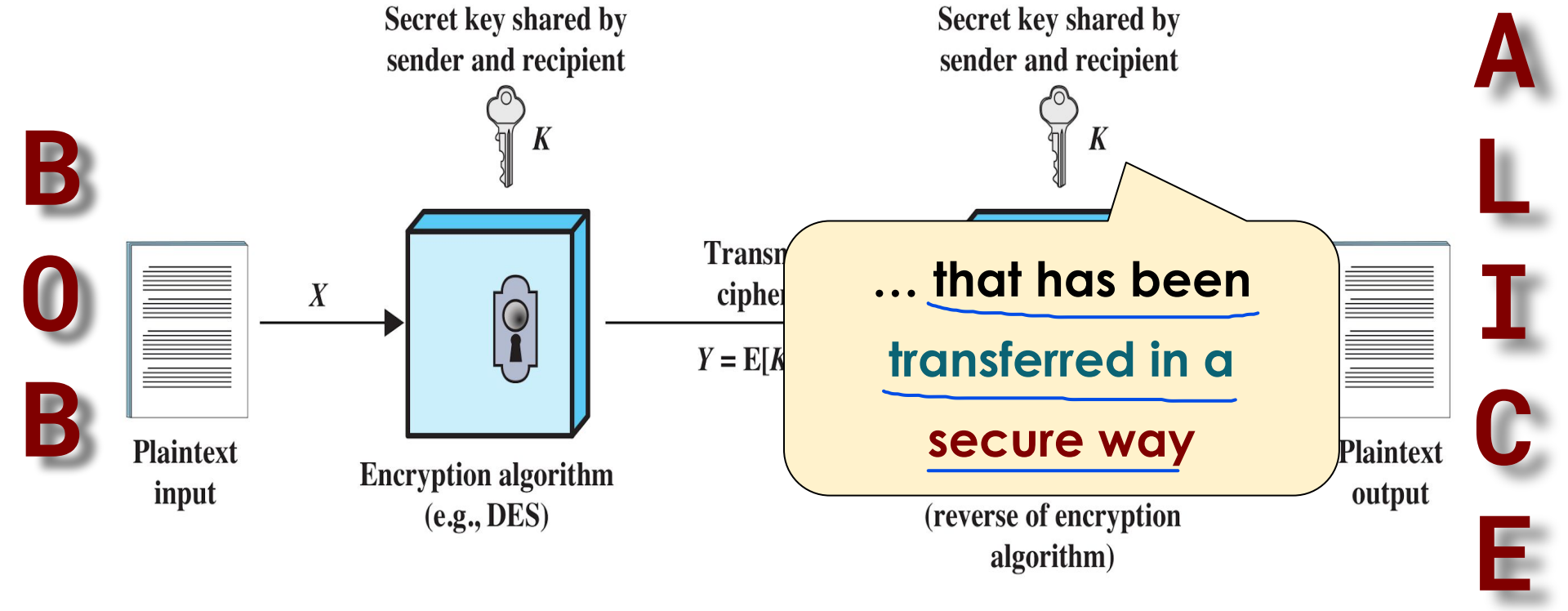
ALICE



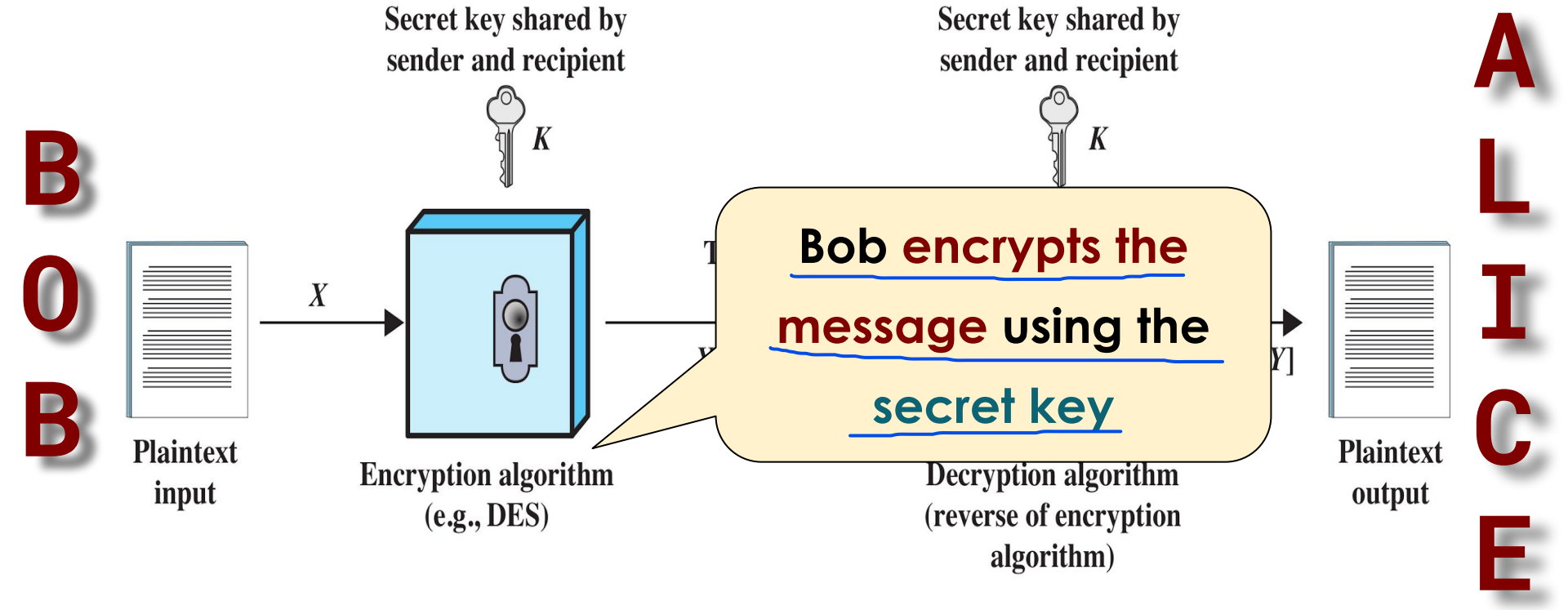
Simplified Model of Symmetric Encryption



Simplified Model of Symmetric Encryption

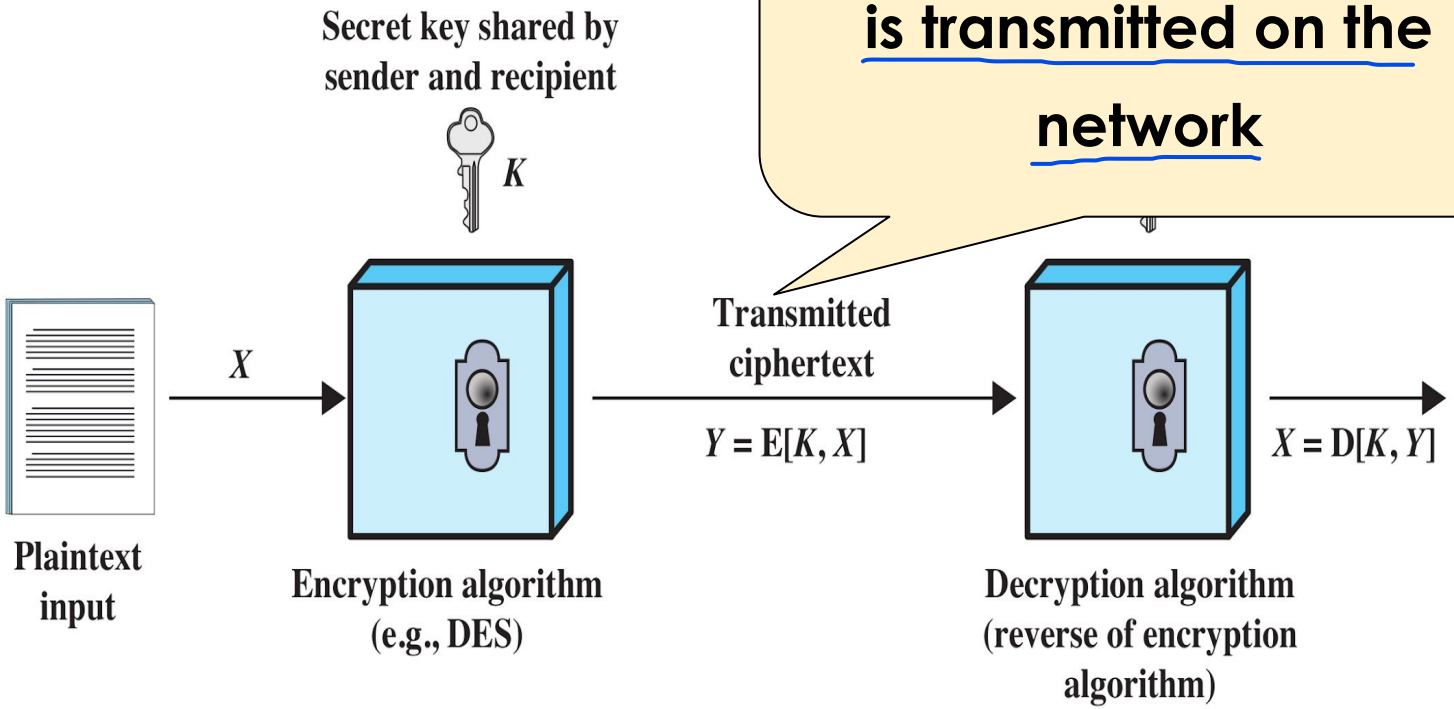


Simplified Model of Symmetric Encryption



Simplified Model of Symmetric Encryption

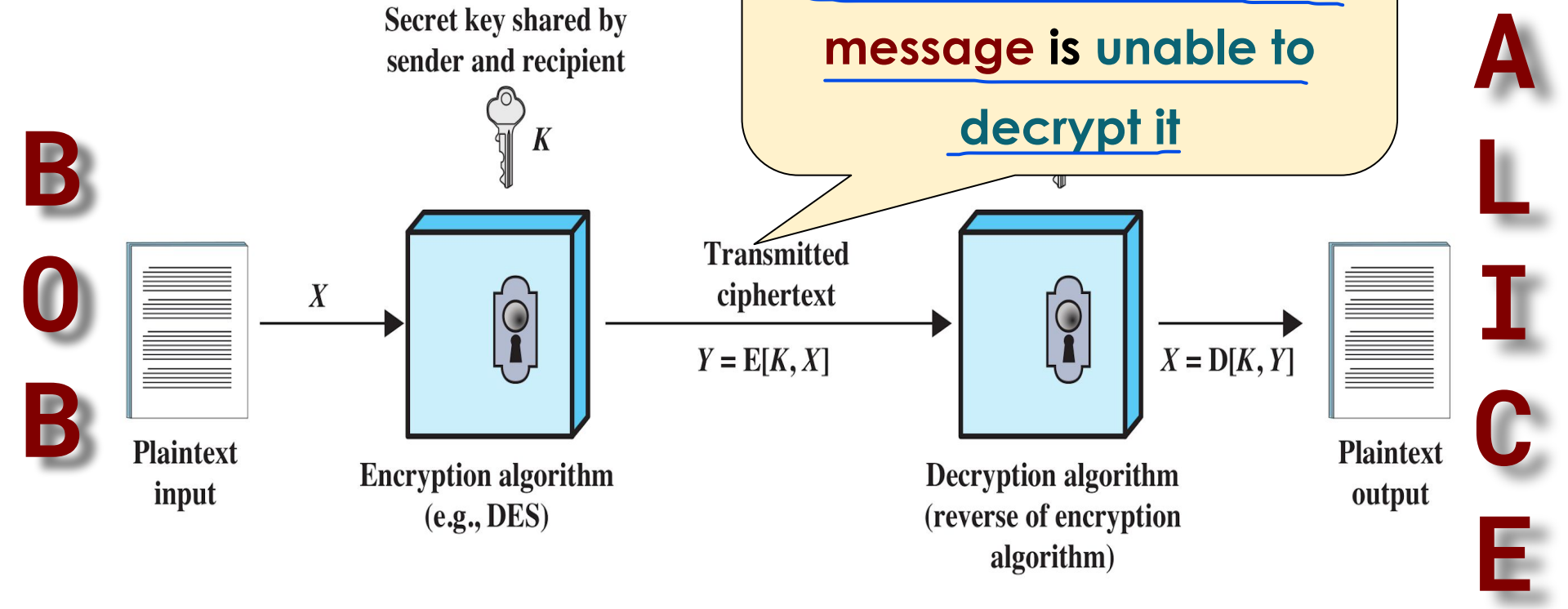
The encrypted message is transmitted on the network



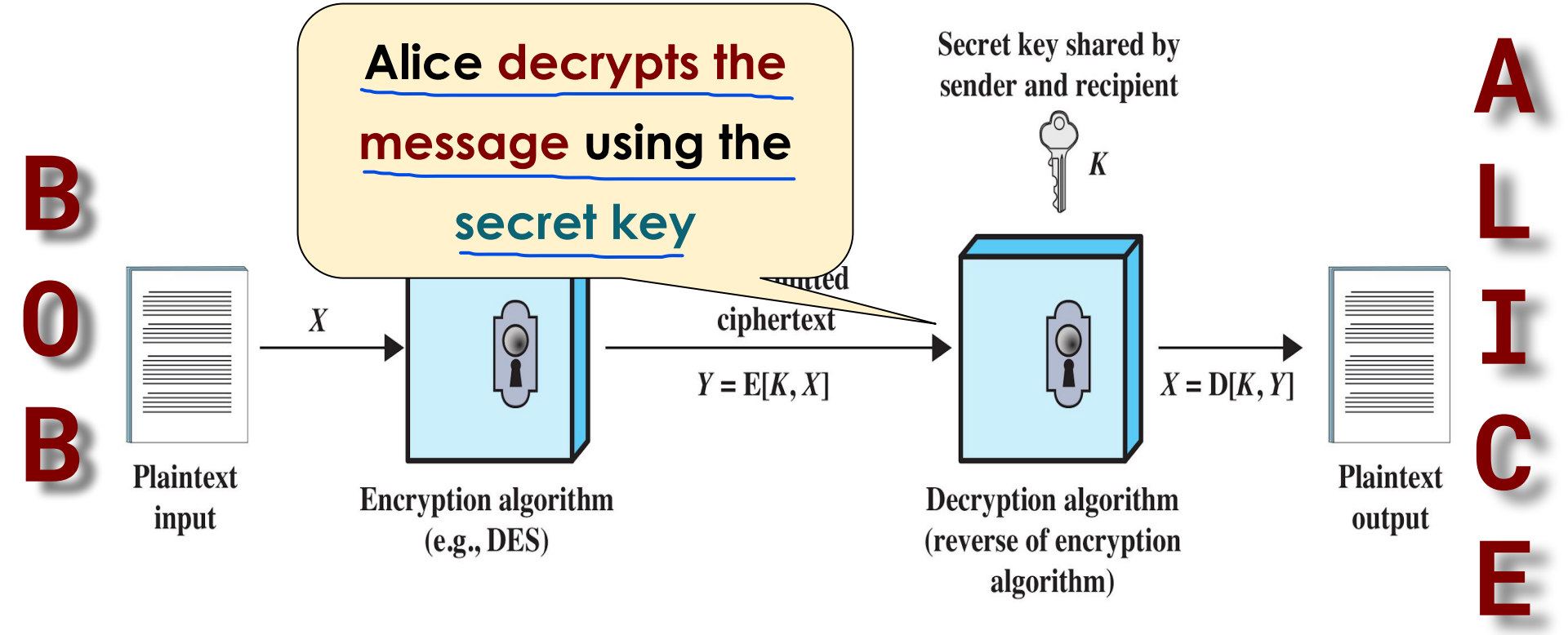
Simplified Model of Symm

MATER STUDIORUM
RSITÀ DI BOLOGNA

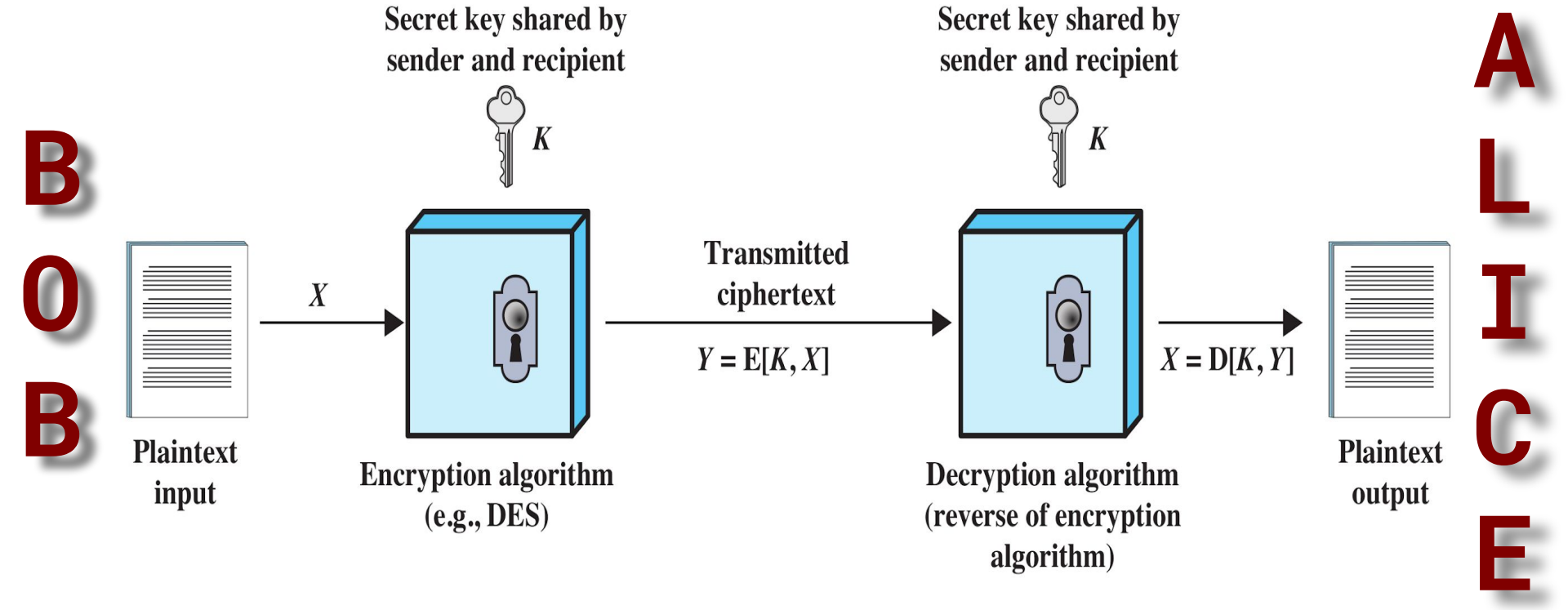
An attacker that intercepts the encrypted message is unable to decrypt it



Simplified Model of Symmetric Encryption



Simplified Model of Symmetric Encryption



If a key is random, an attacker has zero information before they start. Every key in the universe (N) is equally likely.This forces the attacker into the "Worst Case" scenario: a slow, linear Brute Force attack. If the key isn't random (e.g., it's a word from a dictionary, a date of birth, or a common pattern), the attacker can use a Cryptanalytic Attack to bypass the "Universe of Keys."

Symmetric Encryption: Attacks

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

1. The goal of Chiper Designers is to make this type of attack as costly as possible (if it is very costly, the attacker will not persue this approach)

Brute Force attack: **try all the possible keys**

The assumption that a key is random is fundamental because:
- It is the only way to guarantee that the attacker must perform a full Brute Force.
- It prevents Dictionary Attacks and Pattern Recognition from providing a shortcut.

(fundamental assumption)

- *again, it is about the **key size** (if the key is **randomic** and **not a dictionary word**) and how fast is our computer (or cluster of computers)*

when build a Cipher we take into consideration computer speed of today and future computers to assess the strongness of a Cipher

2. Attack using vulnerabilities caused by weak design of Ciphers

Cryptanalytic attack: if the cipher is **weak** then it could be possible to find ways to decrypt the ciphertexts (i.e. WW2 Enigma Machine)

- If the attack is **successful** then **all future and past messages** encrypted with that key are **compromised** (it is better to change frequently the key to avoid compromission (we don't know if the communication channel is compromised))

Before trying all the possible combinations of characters, I would try with all the words in dictionaries

- *again, it is about the **key size** (if the key is **randomic** and **not a dictionary word**) and how fast is our computer (or cluster of computers)*

- **Cryptanalytic** attack: if the cipher is **weak** then it could be possible to find ways to decrypt the ciphertexts (i.e. WW2 Enigma Machine)
- If the attack is **successful** then **all future and past messages** encrypted with that key are **compromised**

Before trying all the possible combinations of characters, I would try with all the words in dictionaries

- *again, it is about the **key size** (if the key is **randomic** and **not a dictionary***
- *(or cluster of computers)*

What is a dictionary?

- e cipher is **weak** then it could be possible to find ways to decrypt the ciphertexts (i.e. WW2 Enigma Machine)
- If the attack is **successful** then **all future and past messages** encrypted with that key are **compromised**

Before trying all the possible combinations of characters, I would try with all the words in dictionaries

È un file di testo che contiene un elenco sterminato di parole, una per riga. Include:
- Parole di uso comune in diverse lingue.
- Password reali che sono state rubate in passato durante i grandi attacchi informatici della storia (i cosiddetti data breach di siti come Yahoo, Adobe, LinkedIn, ecc.) e poi raccolte dagli hacker.
- Errori di battitura comuni (es. p4ssword al posto di password).

Come funziona l'attacco: L'attaccante dà in pasto questo file a un software di cracking (come John the Ripper). Il software prende la prima parola del dizionario, ne calcola la cifratura o l'hash, e controlla se corrisponde a quella della vittima. Se non corrisponde, passa alla seconda parola. Questo processo avviene a una velocità di milioni di tentativi al secondo.

- *again, it is about the **key size** (if the key is **randomic** and **not a dictionary***
  - *(or cluster of computers)*

What is a dictionary?

- e cipher is **weak** t… it could be possible to find ways to decrypt the ciphert… (i.e. WW2 Enigma Machine)
- If the at… **…uture and past messages** encrypted with tho…

non esiste un unico dizionario "perfetto"

What is the best dictionary?

Symmetric Encryption: Attacks

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Brute Force** attack: **try all the possible keys**
  - *again, it is about the **key size** (if the key is **randomic** and **not a dictionary word**) and how fast is our computer (or cluster of computers)*
- **Cryptanalytic** attack: if the cipher is **weak** then it could be possible to find ways to decrypt the ciphertexts (i.e. WW2 Enigma Machine)
- If the attack is **successful** then **all future and past messages** encrypted with that key are **compromised**

Right now, we are considering only the key size and the speed of each decryption by hardware (we do not consider the type of the cipher)

Symmetric Encryption: Attacks

Depends on the computer power (can be general or specialized hardware)

Key size (bits)	Number of alternative keys	Time required at 1 decryption/μs microsec	Time required at 10^6 decryptions/μs
32	$2^{32} = 4.3 \times 10^9$	35.8 minutes	2.15 milliseconds
56	$2^{56} = 7.2 \times 10^{16}$	1142 years	10.01 hours
128	$2^{128} = 3.4 \times 10^{38}$	5.4×10^{24} years	5.4×10^{18} years
168	$2^{168} = 3.7 \times 10^{50}$	5.9×10^{36} years	5.9×10^{30} years
26 characters (permutation)	$26! = 4 \times 10^{26}$	6.4×10^{12} years	6.4×10^{6} years

Average time required for exhaustive key search (i.e., Brute-Force)

If you find a weakness in the cipher, all the messages that use this cipher are compromised. In case of brute forcing, only the messages that use that key are compromised, if you find it (so the cipher is not broken and if they change the key, the attackers needs to do again the brute force attack)

Symmetric Encryption: algorithms

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Many different **symmetric encryption algorithms** have been proposed
- It is better to use algorithms that are an **international standard** (you must consider only standards and not the ones published by researched and not defined as standards)
- **DES** (Data Encryption Standard, 1977):
  - key size = 56 bits
  - It is **slow** and **very insecure** (the key size is too small) (it is slow using Software, so they introduced hardware to perform the same task fastly (HW acc)) — insecure because
- **3DES**: "*execute 3 times the standard DES*" — Key size was increased without changing the encryption/decryption algorithm (so it is possible to use the same hardware accelerators of the DES for the 3DES: in order to avoid exorbitant costs when switching to a new secure symmetric encryption algorithm, as DES was no longer secure)
  - it is **very slow** but quite secure (key size = 56*3) — More slow than DES (3x) but more secure
- **AES** (Advanced Encryption Standard, 2002): — Standard of Symmetric Cipher
  - key size = {128, 192 and 256} bits — You can select the key size (this makes brute-force attacks more difficult and encryption will be slower as the key size
  - very **fast** and **secure** (**now**, but in the future?) — It is designed to be extremely fast in terms of both hardware and software; will still be secure

Performing the same encryption operation three times with two or three different keys (message encrypted with the first, then second and then third key (sequential pipeline))

So, with 3DES, they resolved both the cost issue (by keeping the same hardware accelerators) and the security issue (by increasing the key size)

Symmetric Enc

Likely the most studied encryption algorithm: some cryptanalytic issues but no relevant weaknesses

- Many different
- It is bett[illegible] use alg[illegible]
- **DES** (Data Encryption Standard, 1977):
  - key size = 56 bits
  - It is **slow** and **very insecure** (the key size is too small)
- **3DES**: "*execute 3 times the standard DES*"
  - it is **very slow** but quite secure (key size = 56*3)
- **AES** (Advanced Encryption Standard, 2002):
  - key size = {128, 192 and 256} bits
  - very **fast** and **secure** (**now**… in the future?)

Symmetric Enc

Likely the most studied encryption algorithm: some cryptanalytic issues but no relevant weaknesses of the cipher

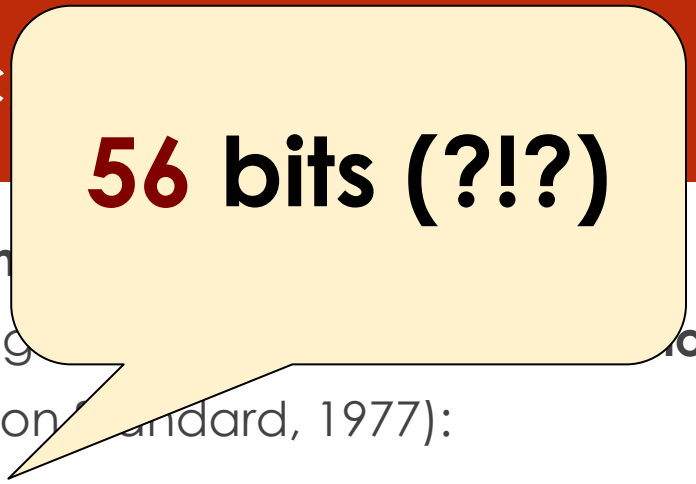
(few minor problems: not so important)

- Many different
- It is bett[illegible] use alg[illegible]
- **DES** (Data Encryption Standard, 1977):
  - key size = 56 bits
  - It is **slow** and **very insecure** (the key size is too small)
- **3DES**: "*execute 3 ti[illegible]e standard DES*"
  - it is [illegible]

In 1998, Electronic Frontier Foundation (EFF) announced that it had broken a DES in < 3 days with specialized hardware

Designed and Built for doing only this task

Symmetric Enc



- Many different **sym** have been proposed
- It is better to use alg **onal standard**
- **DES** (Data Encryption Standard, 1977):
  - key size = 56 bits
  - It is **slow** and **very insecure** (the key size is too small)
- **3DES**: "*execute 3 times the standard DES*"
  - it is **very slow** but quite secure (key size = 56*3)
- **AES** (Advanced Encryption Standard, 2002):
  - key size = {128, 192 and 256} bits
  - very **fast** and **secure** (**now**… in the future?)

Symmetric Enc

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Why not 64 bits?

56 bits (?!?)

- Many different **sym** have been proposed
- It is better to use alg **onal standard**
- **DES** (Data Encryption Standard, 1977):
  - key size = 56 bits
  - It is **slow** and **very insecu** (the key size is too small)
- **3DES**: "*execute 3 times the st*
  - it is **very slow** but quite sec
- **AES** (Advanced Encryption
  - key size = {128, 192 and 25
  - very **fast** and **secure** (**now**

It is not a power of 2 :-(

Symmetric Encryption: algorithms

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Many different **symmetri**
- It is better to use algorith
- **DES** (Data Encryption Sto
  - key size = 56 bits
  - It is **slow** and **very insec**
- **3DES**: "*execute 3 times t*
  - it is **very slow** but quite
- **AES** (Advanced Encrypti
  - key size = {128, 192 and
  - very **fast** and **secure** (**now**… in the future?)

Explaination for why 56 instead of 64 (or a 2-power number)

"***NSA*** *worked closely with* ***IBM*** *to strengthen the algorithm against all* ***except brute force attacks*** *and to strengthen substitution tables, called S-boxes. Conversely, NSA tried to convince IBM to* ***reduce the length of the key from 64 to 48 bits****. Ultimately they compromised on a* ***56****-bit key*"

Thomas R. Johnson

it is a Political Decision, not technical one

Symmetric Encryption: algorithms

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Many different **symmetric encryption algorithms** have been proposed
- It is better to use algorithms that are an **international standard**
- **DES** (Data Encryption Standard, 1977):
  - key size = 56 bits
  - It is **slow** and **very insecure** (the key size is too small)
- **3DES**: "*execute 3 times the standard DES*"
  - it is **very slow** but quite secure (key size = 56*3)
- **AES** (Advanced Encryption Standard, 2002):
  - key size = {128, 192 and 256} bits
  - very **fast** and **secure** (**now**… in the future?)

Symmetric Encryption: algorithms

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Many different **symm** posed
- It is better to use alg
- **DES** (Data Encryptio
  - key size = 56 bits
  - It is **slow** and **very insecure** (size is too small)
- **3DES**: "*execute 3 times the standard DES*"
  - it is **very slow** but quite secure (key size = 56*3)
- **AES** (Advanced Encryption Standard, 2002):
  - key size = {128, 192 and 256} bits
  - very **fast** and **secure** (**now**… in the future?)

The same hardware accelerators used for DES can still be used with 3DES

Symmetric Encryption: algorithms

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Many different **symmetri** proposed
- It is better to use algorith **rd**
- **DES** (Data Encryption St
  - key size = 56 bits
  - It is **slow** and **very insecure** (the ize is too small)
- **3DES**: "*execute 3 times the standard DES*"
  - it is **very slow** but quite secure (key size = 56*3)
- **AES** (Advanced Encryption Standard, 2002):
  - key size = {128, 192 and 256} bits
  - very **fast** and **secure** (**now**… in the future?)

The block size is too small

(see next slides)

It is essential to consider the trade-off between key size and computational overhead; as the key length increases, so does the number of mathematical rounds, leading to higher execution time and energy consumption. For instance, transitioning from AES-128 to AES-256 requires a processor to perform 40% more operations per data block (14 rounds instead of 10). For a financial institution processing billions of transactions per second, this shift could result in noticeable server latency and significantly higher electricity costs, without yielding a meaningful increase in practical security against contemporary threats.

Symmetric Encryption: algorithms

These algorithms are unable to encrypt the entire input data if it exceeds the specified limit; the data must therefore be divided into several blocks. Of course, there are algorithms that do not use a block-based approach and therefore take the input and encrypt it directly

	DES	Triple DES	AES
Plaintext block size (bits)	64	64	128
Ciphertext block size (bits)	64	64	128
Key size (bits)	56	2x or 3x the DES Key size 112 or 168	128, 192, or 256

In the DES and 3DES, the only change is on the key size not in the block size of the input

Performance will be higher if the block size is bigger (AES has a better performance than DES and 3DES: not only for block size)

The choice of AES key size is a strategic decision based on the specific threat model, performance requirements, and data longevity (The decision is almost always a balance between Performance (Speed/Power) and Security (Longevity/Threat Resistance)). For example, in Messaging about not so important data, you can use 128 (because the primary goal is speed), but if you need to encrypt your backups you can use a 256 key (because speed is secondary and requires more security on the data than the first one: need to resist future (Quantum) computers.)

Symmetric Encryption: block ciphers

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

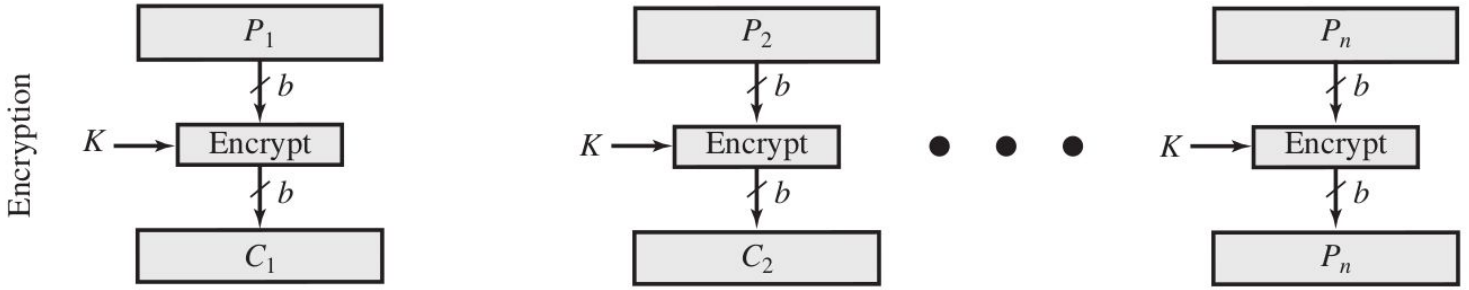
- **DES**, **3DES** and **AES** are **symmetric block ciphers**
- Typically data units (e.g., messages or files) are **larger than a single 64-bit** or **128-bit block**
- Possible solution… **Electronic CodeBook** (**ECB**) mode:
  - (the simplest approach to multiple-block encryption)
  1. **divide** the input data in a **sequence of blocks**
  2. apply **padding** to the last block (if necessary) (if the block hasn't the length required)
  3. each block is **encrypted** using **the same key**
  - (**regularities** in the plaintext could be exploited for **cryptanalysis**)

1) If I am the man-in-the-middle (MITM), even if I do not understand the blocks, I can reorder them and the recipient will not be able to tell that the blocks have been reordered (each block is encrypted in such a way that it does not depend on the others: no dependency between blocks): The ECB does not protect the logical structure of the message, which allows an attacker to manipulate the order of the pieces without the recipient ultimately noticing anything, as the decrypted message appears 'valid', because the order of the message depends only on its entire semantics.

Modes of Operation: ECB

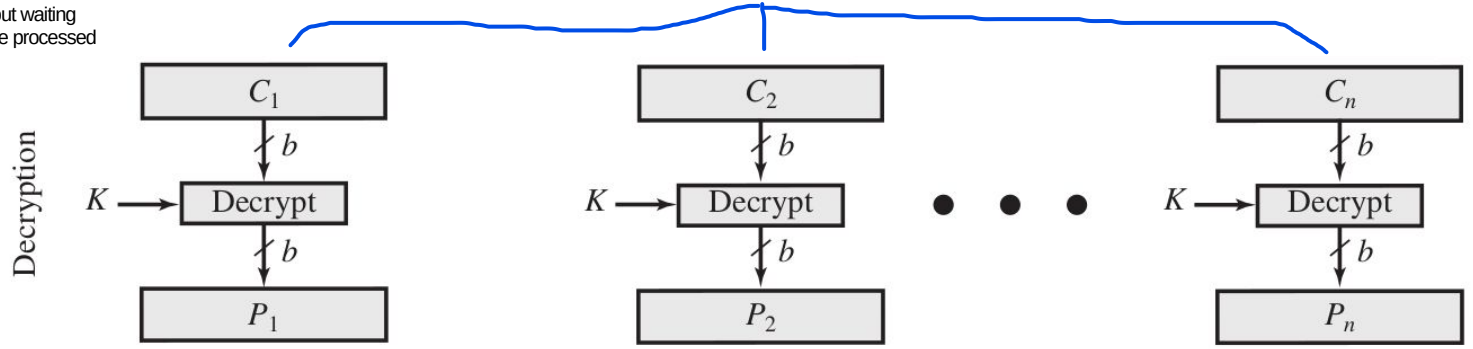
Using OpenSSL, there are different modes of operation to divide a message in blocks: this is the simplest one

For each partition, you do the same operation with same key and cipher



2) Another issue is the parallel nature of this approach: attackers can exploit this parallelism to try to decrypt the entire message, which consists of independent blocks, without waiting for the previous block to be processed

For each partition, Reverse operation (decryption)



To mitigate block reordering attacks, modern modes of operation use chaining mechanisms to enforce a strict sequence (in which each block k is functionally dependent on the ciphertext of the previous block). Even though this ensures structural integrity, it introduces a sequential dependency that eliminates the possibility of parallel processing during encryption/decryption : a significant trade-off in terms of performance compared to the independent nature of the ECB mode (negative thing for each the attacker and user).

In a replay attack, an adversary exploits the lack of inter-block dependency by intercepting a specific ciphertext block and retransmitting it later. Even though the attacker cannot decrypt the block and remains unaware of its exact content, they may know it contains critical information. If the high-level protocol implementation lacks mitigations for block reordering and replay, the system is left highly vulnerable. Crucially, if the encryption scheme does not chain blocks together (failing to provide structural integrity) and the upper-layer protocol fails to enforce sequence numbering or timestamps (failing to provide protocol integrity), you are completely defenseless. In such scenarios, even the strongest encryption in the world, such as AES-256, cannot prevent a replay attack.

Modes of Operation: ECB

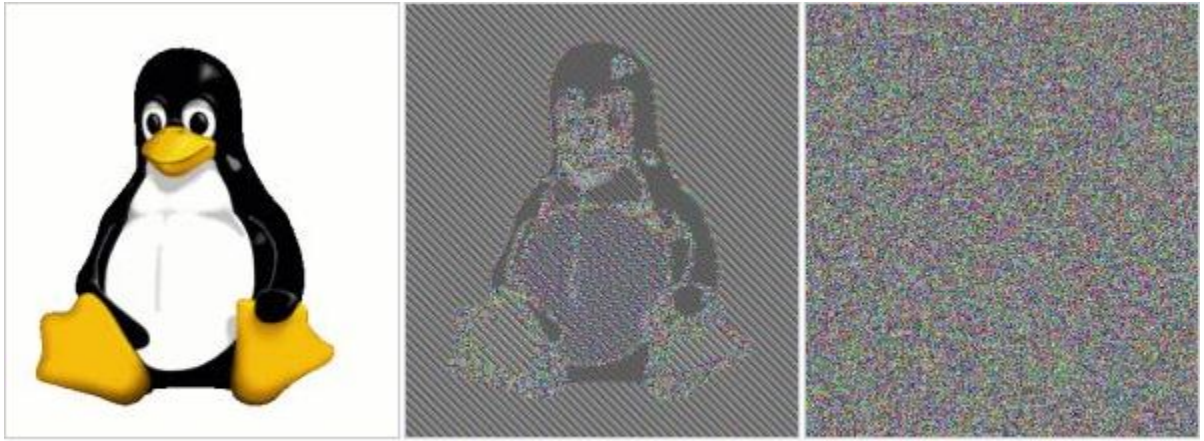
In cryptography, diffusion ensures that modifying a single bit of the plaintext changes multiple bits of the ciphertext in a seemingly random manner. The fatal flaw of ECB mode is that diffusion is strictly confined within individual blocks. For instance, in a message spanning 100 blocks, altering one bit in Block 1 only affects Ciphertext Block 1; Blocks 2 through 100 remain completely unchanged. Because ECB lacks inter-block diffusion, it preserves the underlying structure of the data. Consequently, if the plaintext contains repetitive patterns (such as headers, long strings of zeros, or specific image formats) those same patterns will visibly leak into the ciphertext. This structural vulnerability is famously illustrated by the 'ECB Penguin' (shown on the next slide), where the shape of the image remains clearly recognizable even after encryption.

- Consider the **advantages** vs. **disadvantages** of **ECB** in terms of:
  - **performance** (e.g., encryption and decryption throughput)
  - potential **attacks** (e.g., message **reordering** and **replay** attacks)
  - there is a huge problem of "**diffusion**" (see the next slide)
- Other "more complex" **modes of operation** are available (but they have **~~at~~ a cost**)

For example, when a dependency is introduced between blocks, the ability to parallelise the computation is lost

Modes of Operation: ECB

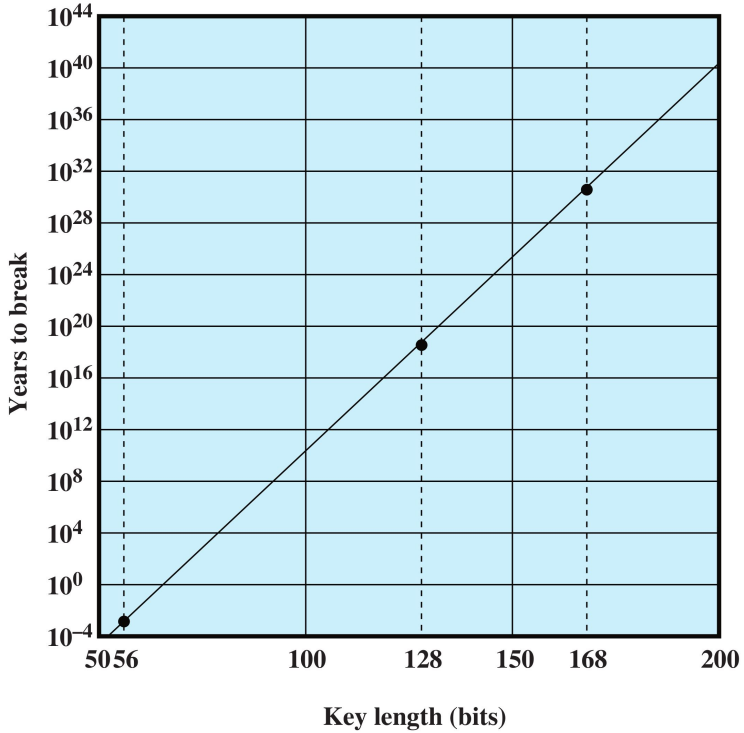
In ECB mode, the encryption process is entirely deterministic; if two plaintext blocks contain identical data, they will inevitably yield identical ciphertext blocks due to the lack of any randomization mechanism. It is crucial to note that the flaw does not lie in the core encryption algorithm—such as AES, which remains a robust pseudorandom function—but rather in the mode of operation itself. This architectural flaw leads to a 'leakage of patterns,' wherein the underlying structure of the original data remains visible in the encrypted output. Due to the absence of inter-block diffusion, regularities in the plaintext are directly mirrored as regularities in the ciphertext, thereby exposing a significant amount of information. This issue cannot be solved by choosing a stronger key or a better cipher; it is strictly a consequence of ECB mode. Migrating to an alternative mode of operation that ensures proper diffusion fully mitigates this problem, preventing any structural leakage of the plaintext.



Original image | Encrypted using ECB mode | Modes other than ECB result in pseudo-randomness

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Brute-Force



Time To Break a Code

(assuming 10^6 decryptions/μs)

Is this assumption realistic?

Check it out!

T2

Symmetric Encryption: block ciphers

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **DES**, **3DES** and **AES** are **symmetric block ciphers**

Assumption: data produced in fixed blocks.

How can we deal with data that is NOT produced in blocks?

Why block ciphers struggle in a "stream" environment:

1. The Latency Problem (The "Waiting" Factor): A block cipher is like a factory that only operates when a shipping container is 100% full. If your block size is 128 bits (AES) but your application produces data 8 bits at a time (like a keystroke or a voice sample), the algorithm must wait for the buffer to fill up before it can encrypt anything or to add a lot of padding in respect to the meaningful data. This introduces "Lag" in a VoIP call or a gaming session, waiting for blocks to fill creates noticeable delays that ruin the real-time experience. A lot of computation (adding padding or pseudo-random content) for nothing.

2. Data Expansion (The "Padding" Problem): Block ciphers must process a full block. If your message isn't a perfect multiple of the block size, you have to add Padding Example: If you want to send a single "Yes" (3 bytes) using a 16-byte block cipher, you have to add 13 bytes of "trash" data just to make the encryption work. Impact: You are wasting bandwidth. If you are sending millions of tiny updates (like GPS coordinates from a fleet of trucks), padding can increase your data costs by 50% or more. A stream cipher, by contrast, results in a ciphertext that is exactly the same size as the plaintext.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Stream Encryption

this tecnique is good for lower latency (don't need to wait to full the block to send it) and good usage of bandwidth (only useful data, not padding or metadata)

Stream Encryption

no blocks are required

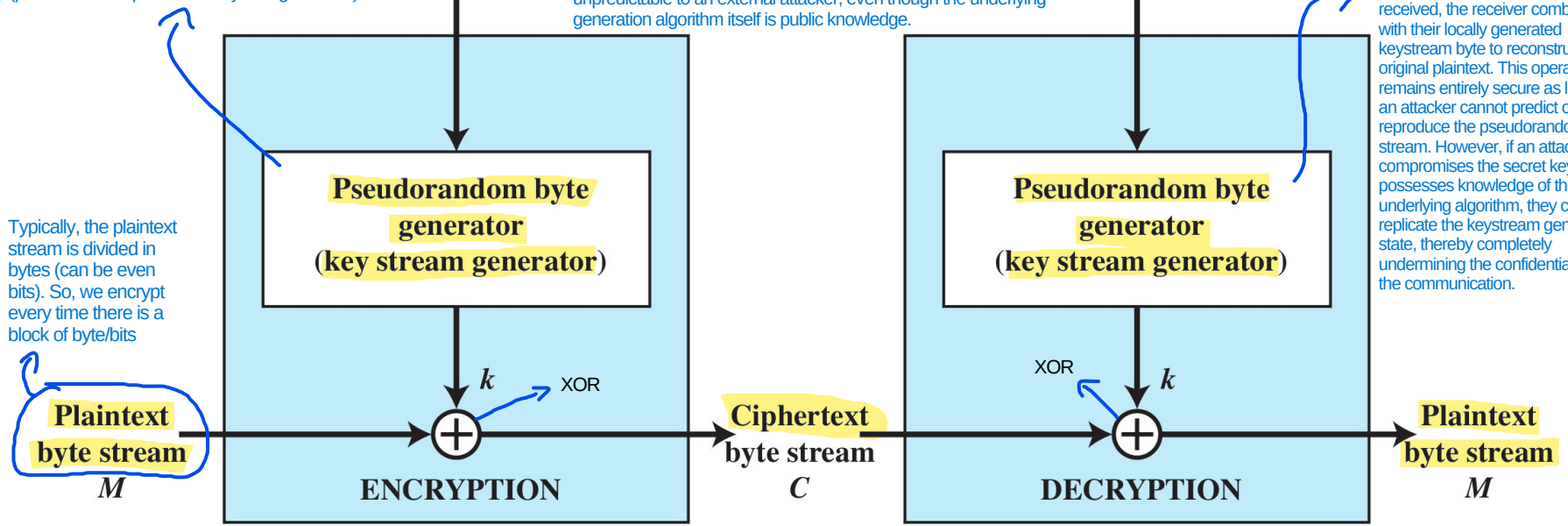
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

We need something that is unpredictable but, at the same time, deterministic: the same exactly sequence of pseudorandom byte must be generated by sender and receiver and the syncronization of the two generators is based on the key (parameter that permits the syncro generator)

The Stream Encryption remains secure provided that the key is not compromised and the pseudorandom generation process is executed correctly, meaning that the generated stream cannot be predicted without knowledge of the secret key. The primary prerequisites are a shared secret key and a robust pseudorandom number generator. This function must produce a stream of bytes that is entirely unpredictable to an external attacker, even though the underlying generation algorithm itself is public knowledge.

In a stream cipher, the keystream generator of the receiver must be perfectly synchronized with that of the sender. Each time a new byte is required, both parties generate the exact same pseudorandom byte. Once an encrypted byte is received, the receiver combines it with their locally generated keystream byte to reconstruct the original plaintext. This operation remains entirely secure as long as an attacker cannot predict or reproduce the pseudorandom byte stream. However, if an attacker compromises the secret key and possesses knowledge of the underlying algorithm, they can replicate the keystream generator's state, thereby completely undermining the confidentiality of the communication.

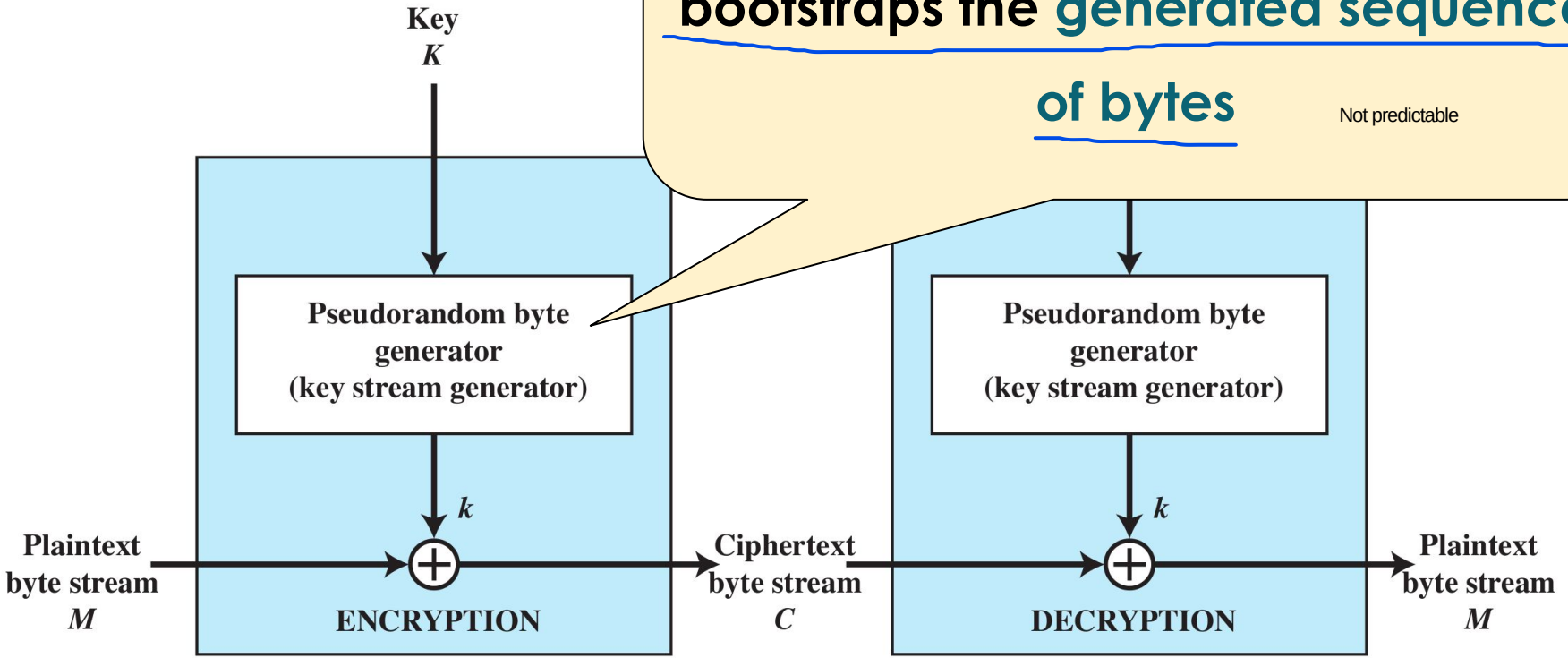
Typically, the plaintext stream is divided in bytes (can be even bits). So, we encrypt every time there is a block of byte/bits



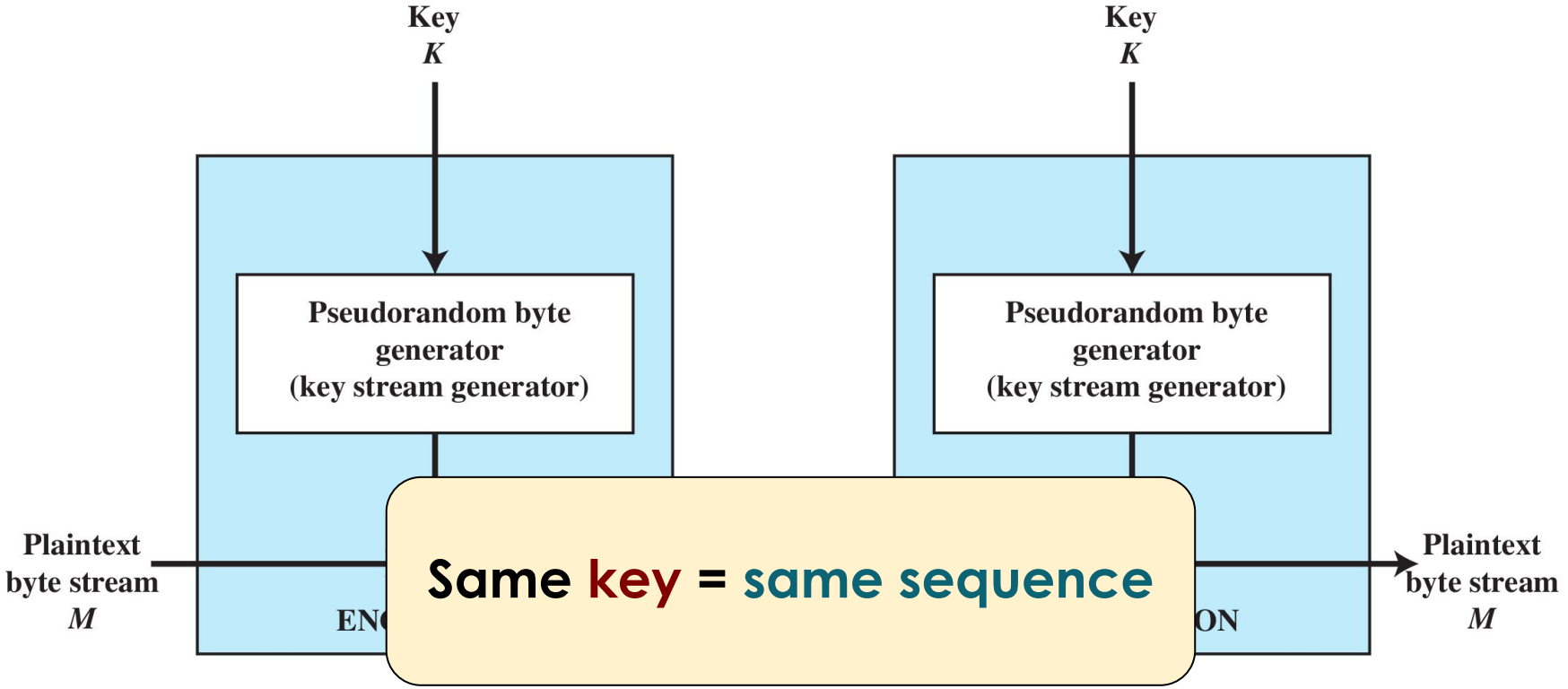
Stream Encryption

The key is the parameter that bootstraps the generated sequence of bytes

Not predictable



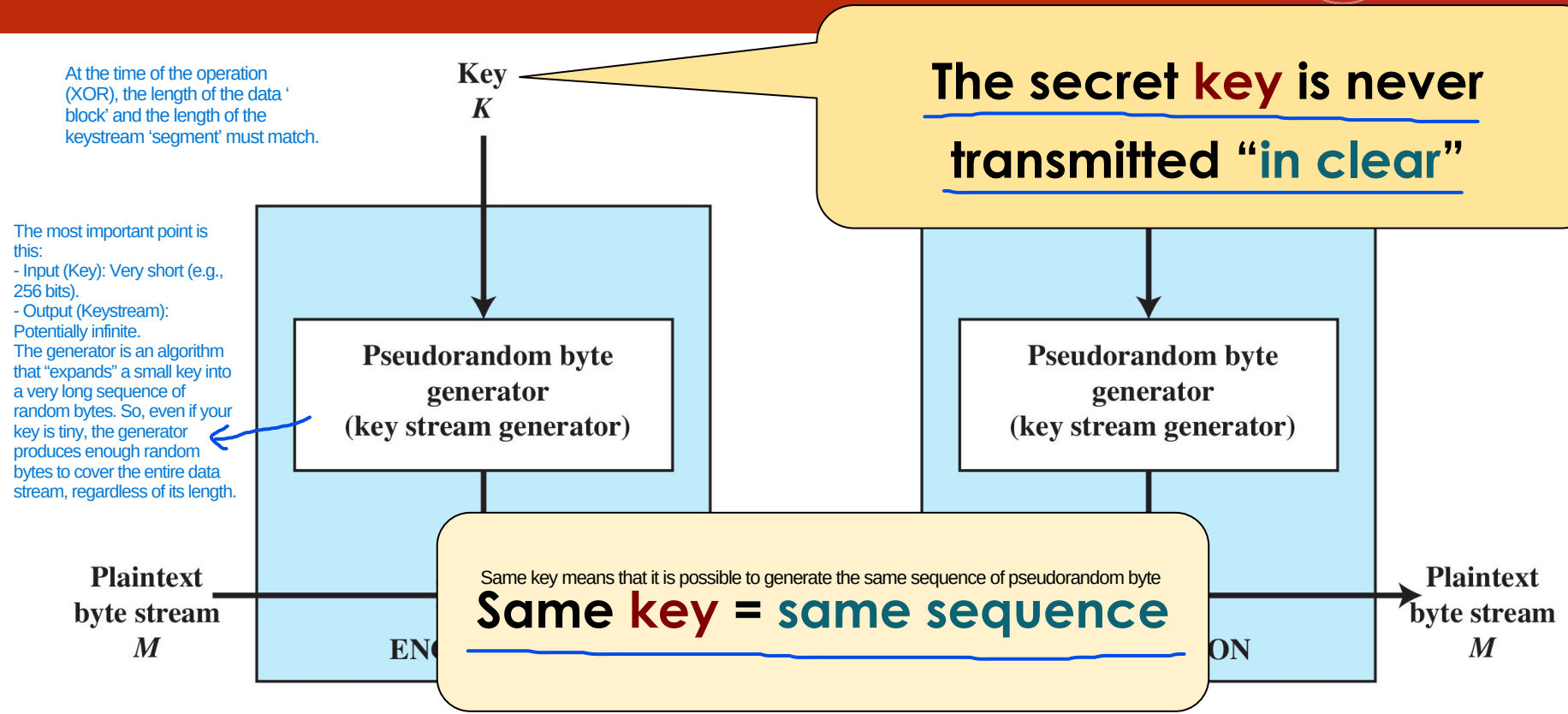
Stream Encryption



In a stream cipher, The generator does not work with 'fixed blocks' in the traditional sense. It is like a tap left running: it keeps outputting bytes for as long as needed. There is no need for 'padding' (adding empty bytes). If your data is 7 bits, the generator gives you 7 bits. If it is 1 gigabyte, it gives you 1 gigabyte.

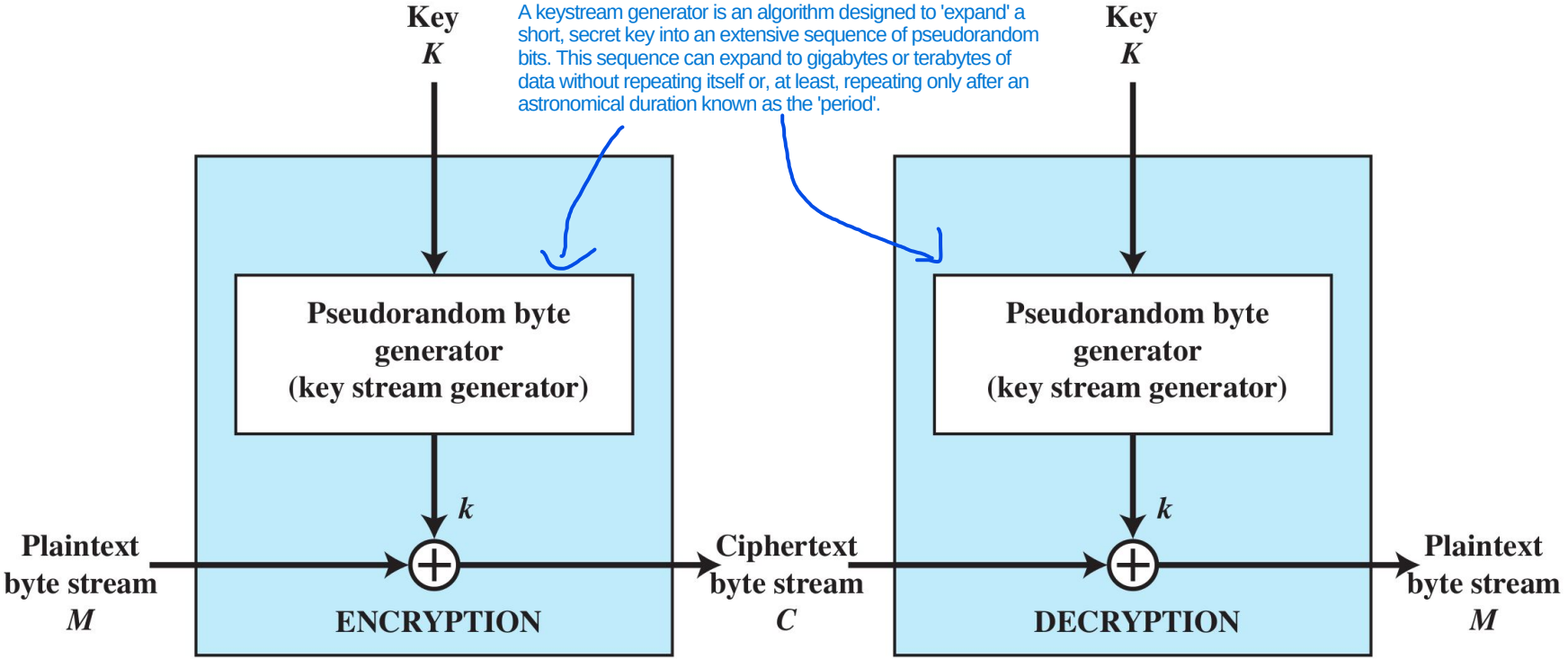
Stream Encryption

At the time of the operation (XOR), the length of the data ' block' and the length of the keystream 'segment' must match.



The most important point is this:
- Input (Key): Very short (e.g., 256 bits).
- Output (Keystream): Potentially infinite.
The generator is an algorithm that "expands" a small key into a very long sequence of random bytes. So, even if your key is tiny, the generator produces enough random bytes to cover the entire data stream, regardless of its length.

Stream Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Leakage in Stream Ciphers: since stream ciphers encrypt bit-by-bit or byte-by-byte without padding, the ciphertext is exactly the same length as the plaintext.
Length Leakage: If I send the word "Yes" (3 bytes), the encrypted message is 3 bytes. If I send "No" (2 bytes), it's 2 bytes.
What the attacker learns: By monitoring the size of encrypted packets in a chat app, an attacker can guess which words are being used.

Example: If a "Help me" message is always 7 bytes, an attacker seeing a 7-byte burst of traffic knows exactly what you just asked for, even if they can't "read" the encrypted bytes.

Block vs. Stream Ciphers

Comparison

Block vs. Stream Ciphers

The choice between using zero-padding or alternative character-padding schemes depends largely on the encryption algorithm's resistance to cryptanalytic attacks.

- **Block ciphers**:
  - process the **input** **one block of elements** **at a time**
  - produce an **output block** **for each input block**
  - can **reuse keys** (and usually they do that)
  - can require to add **padding** — if the block is not completely filled (fixed block size), you need to add trash data or zeros
    - bad for **efficiency**!
    - how is **padding generated**? → Using zeros is not a good idea because it is highly likely to cause diffusion problems. It is very likely that a random data generator ('trash') will be needed to resolve the diffusion problems: but we need to assess how fast it is
  - are **more common** (than stream ciphers) → In typical modern CPUs, block ciphers are implemented in hardware (not the stream ones)

Block vs. Stream Ciphers

- **Stream ciphers**:
  - process the **input** elements **continuously** (immediately, without having to wait: no introduction of latency)
  - produces **output** **one element** **at a time** every single time there is a new byte to be encrypted, immediately they are able to do it and send it
  - almost always **faster** and **use far less code** (In block ciphers, the complexity is in the algo: AES is very complex. In stream ciphers, the algo is the pseudorandom byte generator: the complexity is external, not in the direct encryption, and the complexity is simpler than the block ciphers)
  - encrypts plaintext **one byte** **at a time**
  - pseudorandom stream is one that is **unpredictable without knowledge of the input key**

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

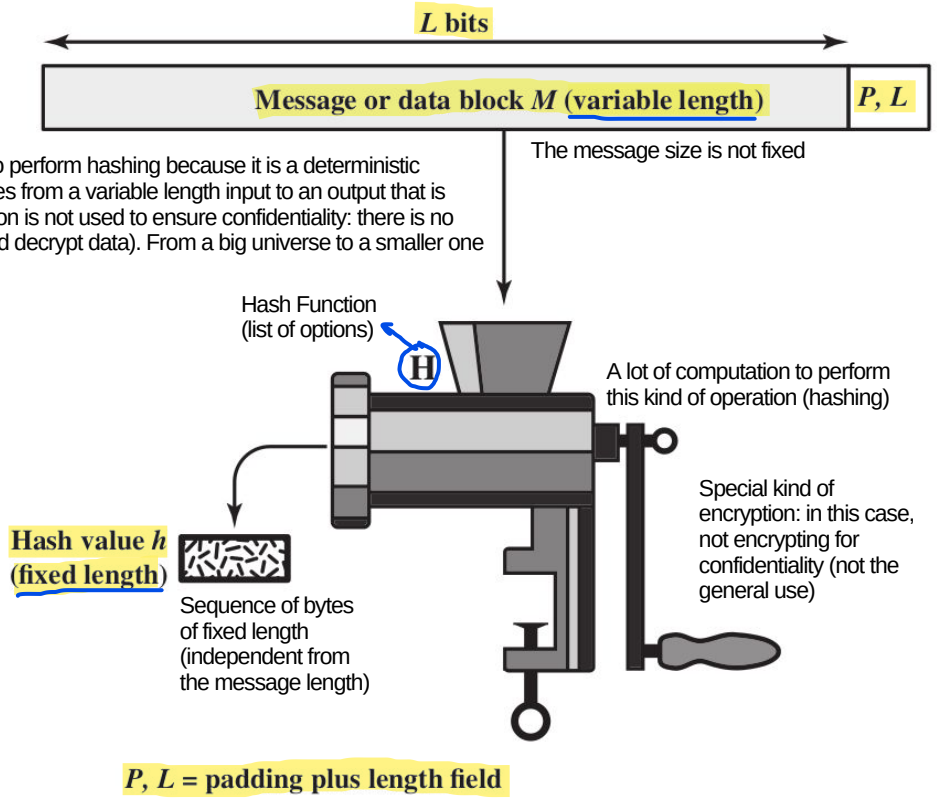
TLS (used for HTTPS) is based on Symmetric Encryption, Secure hash functions and Asymmetric Encryption: all of three is required

It's not just for TLS: it's also used to verify software integrity

Secure Hash Functions

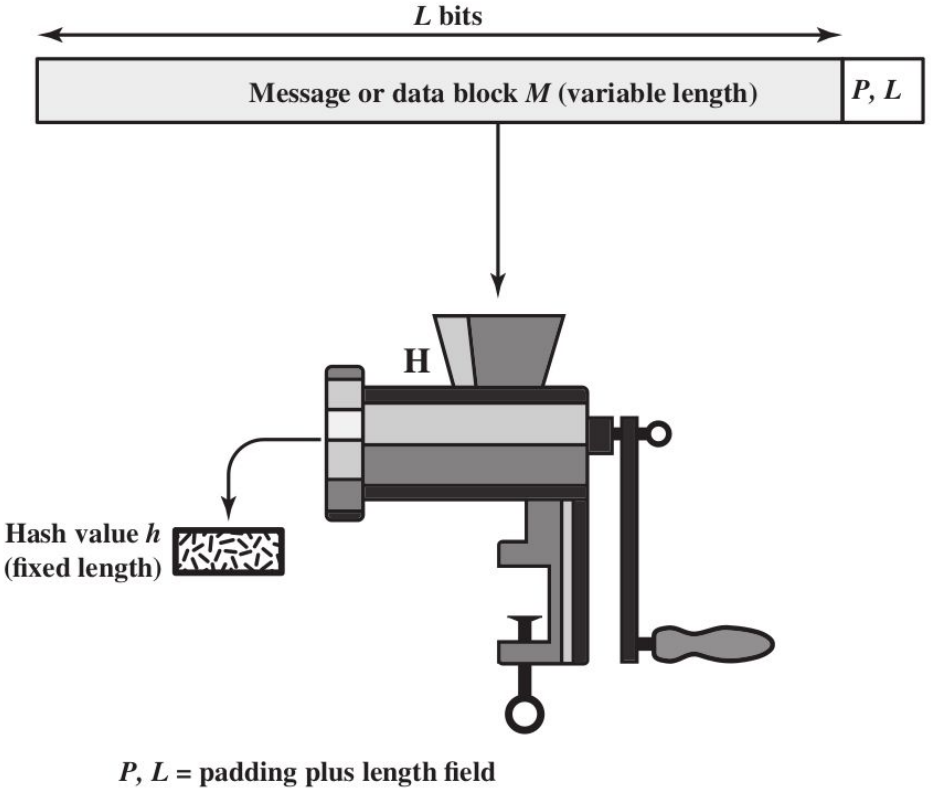
Secure Hash Functions

isn't for confidentiality



Secure Hash Functions

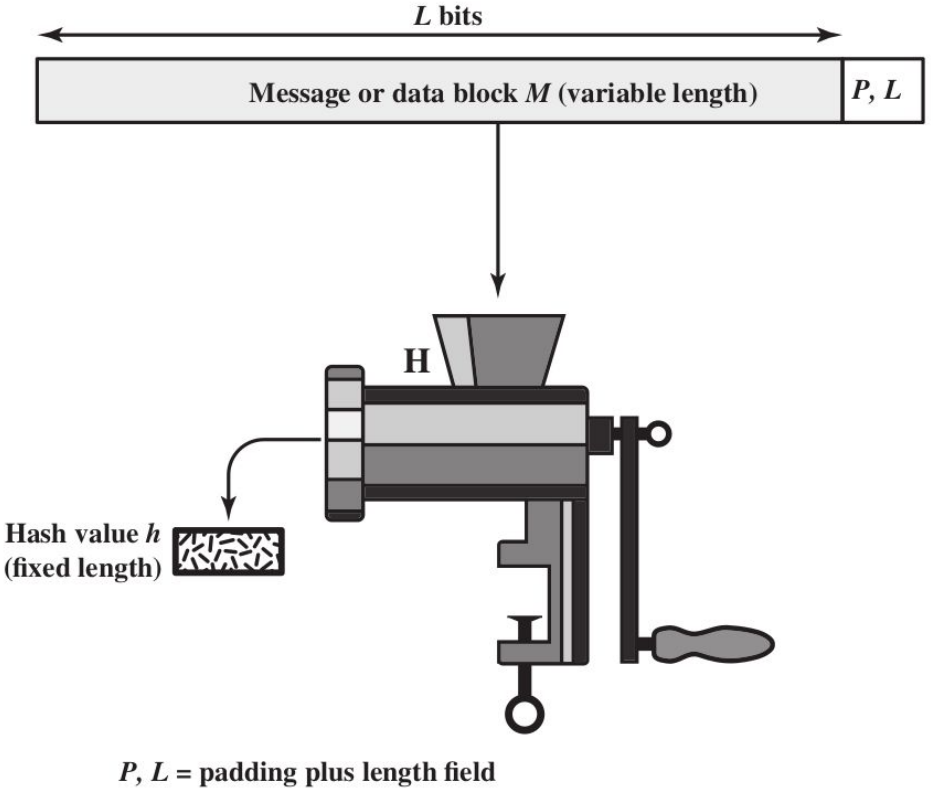
The input size is variable



Secure Hash Functions

The input size is variable

H is a mathematical function



Secure Hash Functions

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

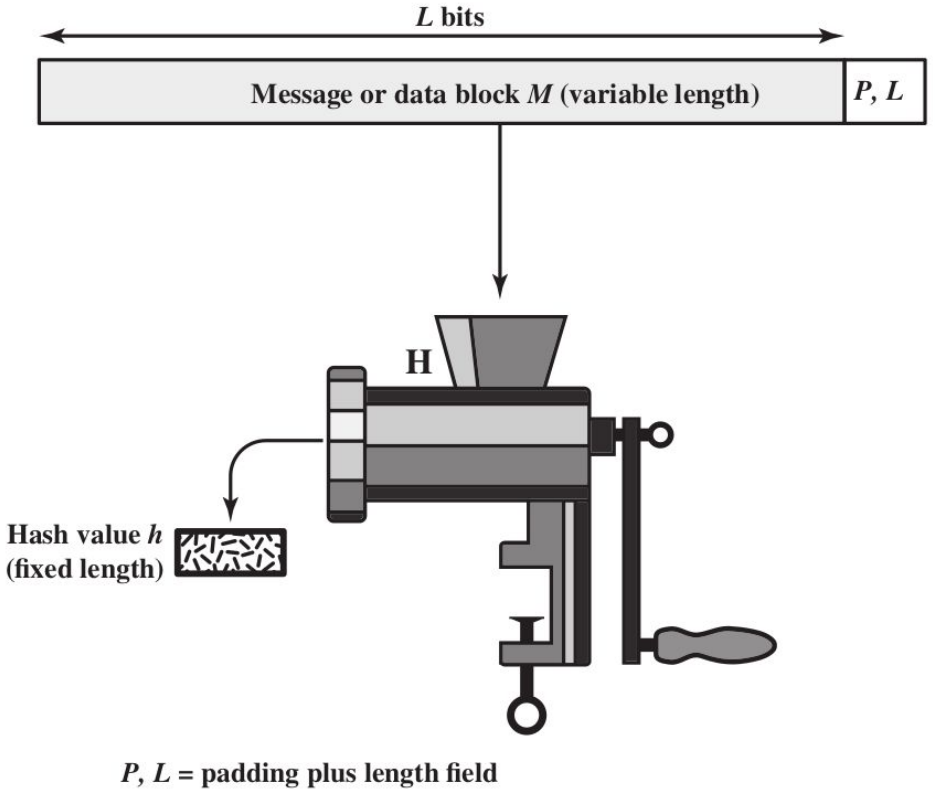
The input size is variable

The hash function must be chosen carefully to guarantee specific required properties

H is a mathematical function

The output size is fixed

cannot be changed (it is a caracteristic of hash function that we are using: design choice)



if it isn't like this, it is easy to get the plaintext through bruteforcing. For example, firmware update with hash (signature) of the update to check if it is compromised; but if the attacker can get the pre-signature (data block before hashing) easily can create a compromised version of the firmware (having the same signature of the original version of the firmware)

A small modification on the input will cause a complete different hash than the original message (if the hash function is secure)

Secure Hash Functions: properties

WHAT ARE THE PROPERTIES THAT ARE IMPORTANT FOR US?

1. ***H*** *can be applied to a block of data of any size*

2. ***H*** *produces a fixed-length output* — if i change the size of the output, i am changing the function itself (so, its design)

3. **H(x)** is **relatively easy to compute** for any given **x** — (qualitative property: reasonable amount of time to do hashing)

4. For ~~any given~~ (a specified hash) code h, it is computationally infeasible to find **x** such that

   H(x) = h (**one-way** or **preimage resistant**) → The inverse operation must be incredibly computationally intensive. So, there is no easy way for performing the inverse operation of hashing and the only way to perform it is try every single combination that is in the start universe (huge: exponential) and check if the output match the hash

We are fixing a specific message (the attacker isn't able to generate a specific collision with this specific message)

5. **For ~~any given~~ block x** (a specified input), it is computationally infeasible to find **y ≠ x** with ~~**H(y)**~~ (same) **= ~~H(x)~~** (hash code h) (**second preimage resistant** or **weak collision resistant**) → different messages but same hash code

Point 6 is the father generalisation of Point 5

This time, the attacker has complete freedom: there is no fixed file imposed by the system.
The attacker's goal is simply to find any pair of distinct data points (x, y) such that H(x) = H(y).

6. It is computationally infeasible to find **any pair (x, y)** such that **H(x) = H(y)**

   (**strong collision resistant**) → From a mathematical point of view, finding any two elements that collide is much easier than finding a specific element that collides with an existing file. Consequently, a hash function might satisfy Point 5 but fail Point 6. To say that a function is Strong Collision Resistant means that it has passed the most stringent security test of all: the attacker cannot find a collision even when given the freedom to choose both files.

This property guarantees that, given a starting file, it is impossible to find another file that generates the exact same fingerprint. It is called "weak" because the attacker's hands are tied: the first file x was not chosen by the attacker; it was imposed on them.

Secure Hash Functions: security

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

An algorithm is officially declared compromised when cryptoanalysis discovers a logical or structural weakness that allows it to be broken with far less effort than a brute-force attack. In technical terms, an algorithm is said to be compromised when an attack is mathematically more efficient than an exhaustive search.

- Approaches to **attack** a **secure hash function**
  1. **cryptanalysis**: exploits **logical weaknesses** in the algorithm
     The attacker studies the mathematics and logical structure behind the algorithm to look for a shortcut, a design flaw, or a statistical weakness. The Goal is to find a way to calculate a collision or reverse the function without having to go through brute-forcing.
  2. **brute-force**: the strength of the hash function depends solely on the **length of the hash code** produced by the algorithm
     If the algorithm is logically well-designed (i.e., it has no structural weaknesses), the attacker has no choice but to try an indefinite number of random input combinations until one generates the desired hash. The strength of the function depends exclusively on the length of the hash produced (e.g., 128 bits, 256 bits, 512 bits). If the output is n bits long, the number of possible solutions is 2^n. The longer the output, the more mathematically impractical a brute-force attack becomes, as it would require computational power and time exceeding the age of the universe.
- **SHA** (Secure Hash Algorithm) is the most widely used hash function
  - many different "families" (SHA0, SHA1, SHA2, SHA3) and hash value lengths

Secure Hash Functions: security

- Approaches to **attack** a **secure hash function**
  - **cryp** … gorithm
  - **brut** … nds solely on the … ithm
- **SHA** (Se … sh function
  - many different "families" (SHA0, SHA1, SHA2, SHA3) and hash value lengths

If a collision is found, the hash function is no longer secure (because the property 6, part of properties to define a secure hash function, is no longer valid)

Lifetime of some famous cryptographic hash functions

Some hash function are no longer safe

Important for certificates and for implementing VPNs

Secure Hash Functions: applications

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Secure hash functions are fundamental for these two tasks

- **Passwords**:
  - the operating systems (and most online services) **store only** the **hash of passwords** (not the plaintext of passwords)

Other two problems: only a part of the system can be checked (for data is difficult, only binary). If you have updates, you need to update the DB of hash code of the binary

- **Intrusion detection**: Create a fingerprint of each file and check if there is a compromise (check the hash code with the original program one)
  - somehow **compute** and **securely store** the **hash code of relevant files** in the system. This permits to verify their **integrity** of the software running on the computer

1st Problem: asking to a program in the file system if the file system or a specific program is compromised (the attacker, if is in the system, is easy for him to compromise even the program that do checks about hash code). Solution: let the check be done by a program that is external to the system to be checked
2nd Problem: you need a DB to store the hash codes of the original programs and it can be compromised if it is online. Solution: store a copy offline of the DB in a secure environment

Secure Hash Functions: applications

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Passwords**:
  - the operating systems (and most online services) **store only** the hash of passwords
- **Intrusion detection**:
  - somehow **compute** a
    files in the system. This

Plaintext passwords are never stored. Why? To be discussed in the authentication chapter

Secure Hash Functions: applications

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Passwords**:
  - the operating systems (and most online services) **store only** the **hash of passwords**
- **Intrusion detection**:
  - somehow **compute** and **securely store** the **hash code of relevant files** in the system. This permits to verify their **integrity**

Secure Hash Functions: applications

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Passwords**:
  - the operating syste[illegible] **only** the **hash of password**
- **Intrusion detection**
  - somehow **compute** and **securely store** the **hash code of relevant files** in the system. This permits to verify their **integrity**

If you want fingerprints for each file and update of the system, in terms of computation, is heavy

This operation is quite costly in terms of computation

Secure Hash Functions: applications

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Passwords**:
  - the operating syste… …y the **hash of passwords**
- **Intrusion detection**:
  - somehow **compute** and **securely store** the **hash code of relevant files** in the system. This permits to verify their **integrity**

Need to update the firgerprint when there is update of programs

It is not so easy! Why?

For different reasons…

Secure Hash Functions: applications

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Passwords**:
  - the operating systems (ar

    hash of passwords

- **Intrusion detection**:
  - somehow **compute** and **securely store** the **hash code of relevant files** in the system. This permits to verify their **integrity**

Why do we not check all the files in the system?

Secure Hash Functions: applications

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

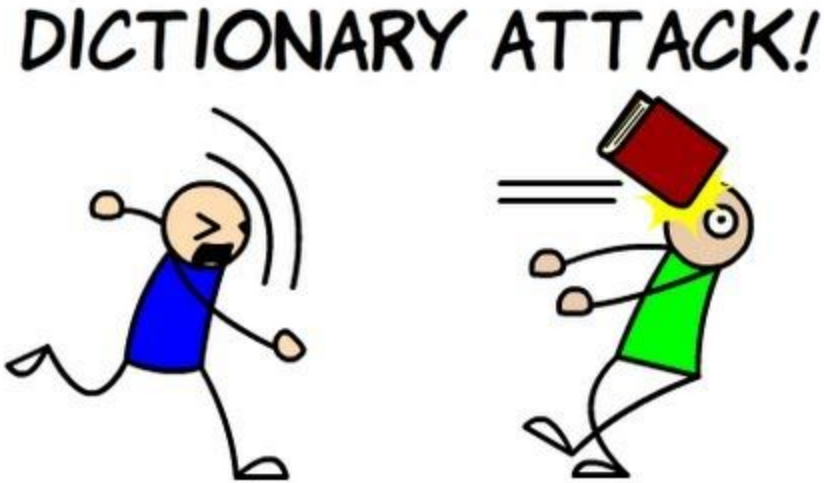
- **Passwords**:
  - the operating systems[illegible]e **hash of passwords**
- **Intrusion detection**:
  - somehow **compute** and **securely** [illegible]**e** the **hash code of relevant files** in the system. This permits to verify their **integrity**

The threat model must be considered very carefully and what part of system can be checked

It works… if the tool used for verifying the integrity is not compromised

We lack of tools for checking the integrity of systems

Dictionary Attack



Practical dictionary attack to the symmetric encryption

How does it works?

Check it out!

T3

Symmetric Key Encryption

Symmetric encryption alone cannot guarantee secure communication over the internet because it requires both participants to have the same key before they can begin communicating. However, there is no simple way to send this key to someone on the other side of the world without an attacker being able to intercept it along the way.

This is quite ridiculous: "*to get a secure communication channel you need a secure channel*"

How is possible to implement secure communication in online banking or e-commerce with such a limitation?

- **sen** nd **receiver** must have **obtaine**

in a secure fashion and **must keep the key secure**

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

To solve the problem about key distribution (talked in the previous slide), we use Public-Key encryption

Public-Key Encryption

Public-Key Encryption

Why does this solve the problem?
- No keys are transmitted: The key that is exchanged is not the final key. The key is generated simultaneously at both locations.
- Resistance to interception: A hacker who intercepts the messages sent between the two parties to generate the key would be faced with a mathematical problem that would take thousands of years to solve using today's computers.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Bits of history**

The Diffie-Hellman (DH) protocol was the first truly revolutionary solution to the problem of key distribution. It is a method that allows two people (let's call them Alice and Bob) to establish a shared secret key over a public channel (the internet), without ever having met before and without the key ever actually being "sent."

  - **1969** - United Kingdom - James Ellis (GCHQ): **theoretical basis**
  - **1976** - USA - Whitfield Diffie and Martin Hellman: **Diffie–Hellman key exchange** (solution to the key distribution problem)

asymmetric key algorithm

  - **1977** - USA - **R**onald Rivest, Adi **S**hamir and Leonard **A**delman: **RSA public-key cryptosystem**

Plently of current implementation are based on this

  - **1991** - USA - Phil Zimmermann, **Pretty Good Privacy (PGP)**

Public-Key Encryption

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Bits of history**
  - **1969** - United Kingdom - James Ellis (GCHQ): **theoretical basis**
  - **1976** - USA - Whitfield Diffie
    key exchange (solution to
  - **1977** - USA - **R**onald Rivest
    public-key cryptosystem
  - **1991** - USA - Phil Zimmermann, **Pretty Good Privacy (PGP)**

state secret
Classified (i.e., secret) and disclosed only in 1997

SA

Public-Key Encryption

The security of public-key infrastructure depends on the binding between a public key and its owner. While the public key is intended for open distribution, its provenance must be guaranteed. If a sender cannot verify that a key belongs to the intended recipient, they risk encrypting data for an adversary who is impersonating that identity. This highlights the necessity of Digital Certificates or a Web of Trust to prevent impersonation.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Public-key encryption is based on mathematical problems that are easy to solve in one direction (encryption) but extremely difficult to reverse (decryption without the key).

- The **public-key encryption** is based on some **complex mathematical problems**

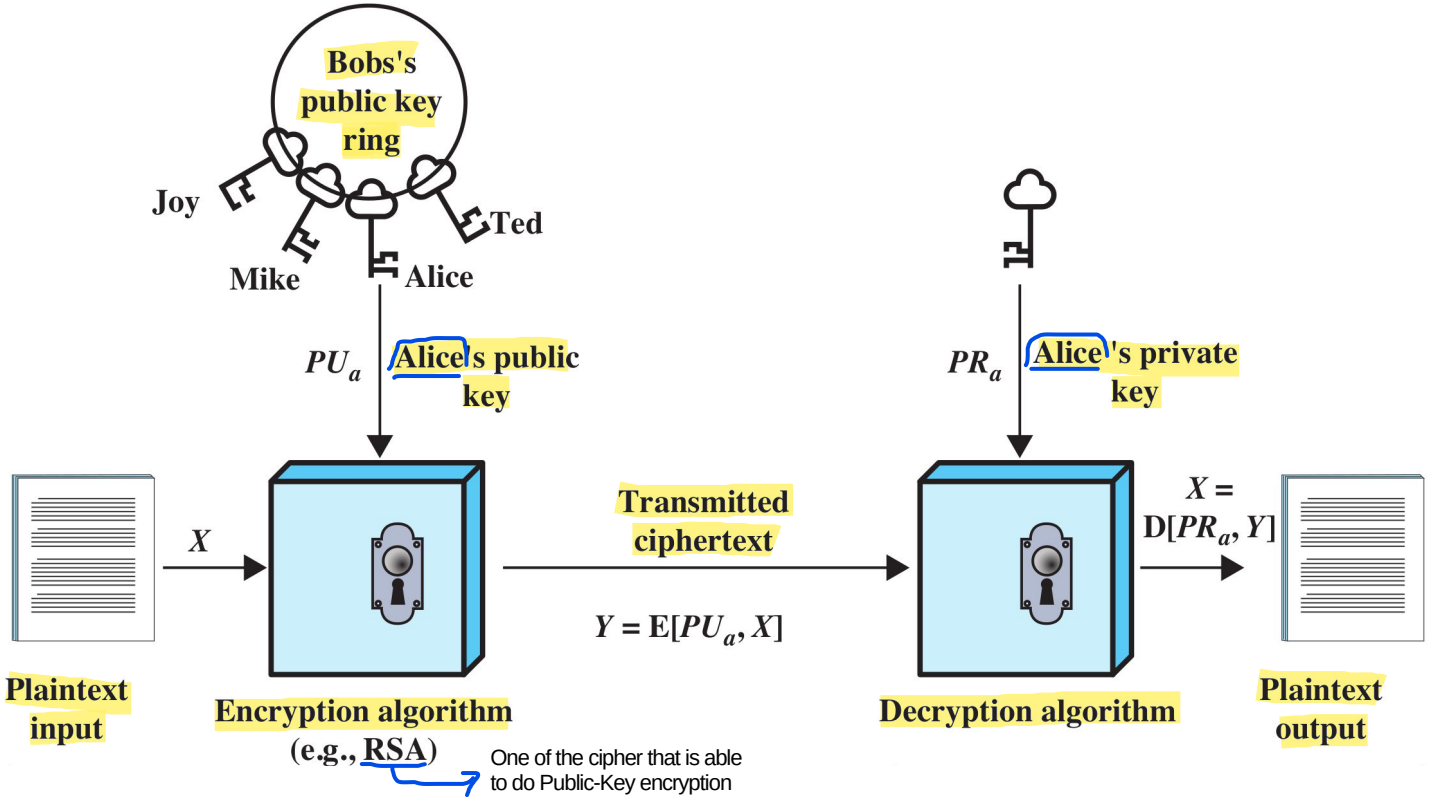
  - Integer Factorization: Given a large composite number, the challenge is to determine its unique prime factors. This problem serves as the mathematical foundation for the RSA algorithm.
  - Discrete Logarithms: The difficulty of solving for an unknown exponent x in a finite field (g^x mod p = h). This principle underpins the Diffie-Hellman key exchange and DSA.
  - Elliptic Curves: Based on the algebraic structure of elliptic curves over finite fields. It provides equivalent security to RSA but with significantly smaller key sizes and higher efficiency.

4 keys for a bi-directional comunication

- It is a form of **asymmetric encryption** since **every user has 2 keys**:
  - one **public key**: *that is made* ***as public as possible*** (must be known to everyone: a public key associated with the identity of the user who possesses the corresponding private key)
  - one **private key**: *that is* ***maintained carefully secret***
- Every **couple of keys** is **generated together**, the two keys are **linked by some mathematical properties**

Public-Key Encryption

B O B

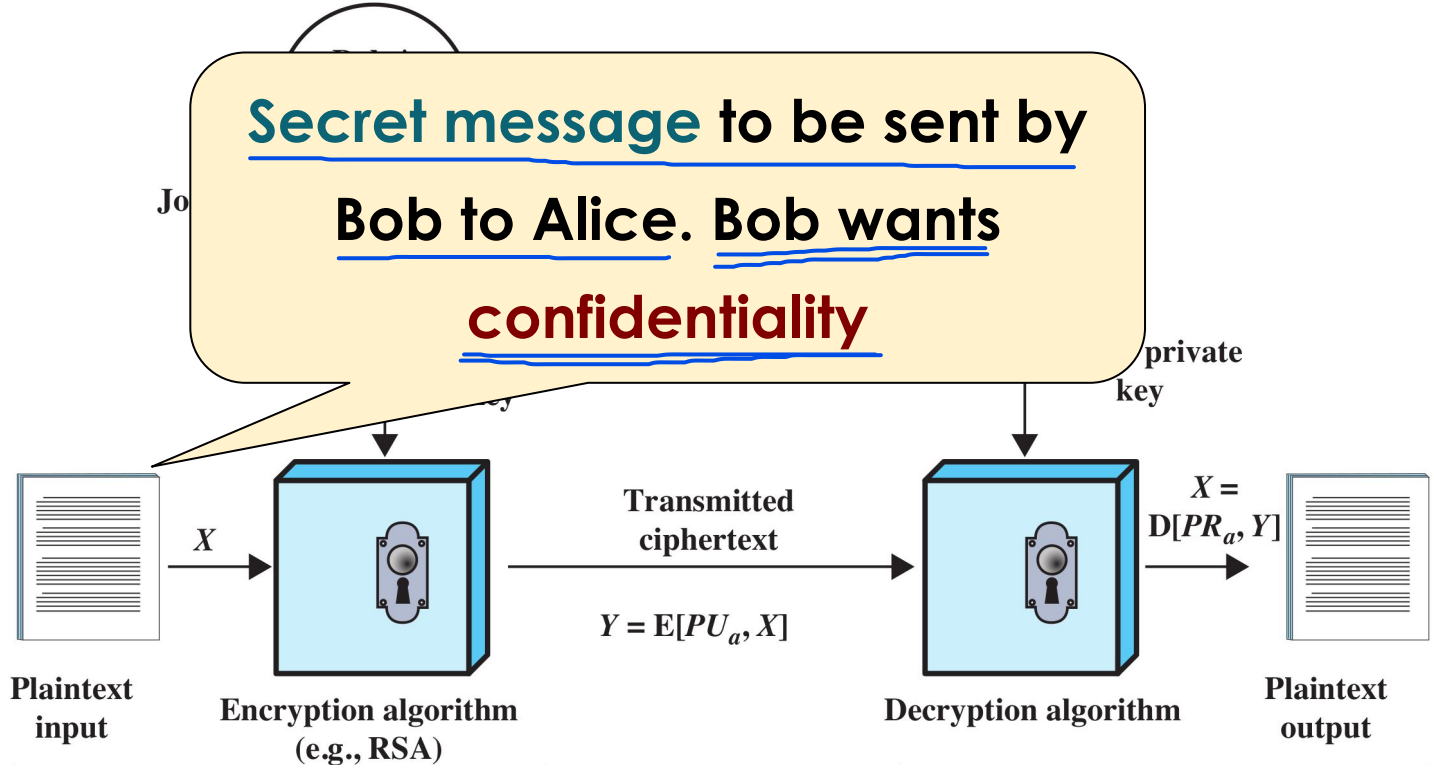


A L I C E

Public-Key Encryption

B O B

A L I C E

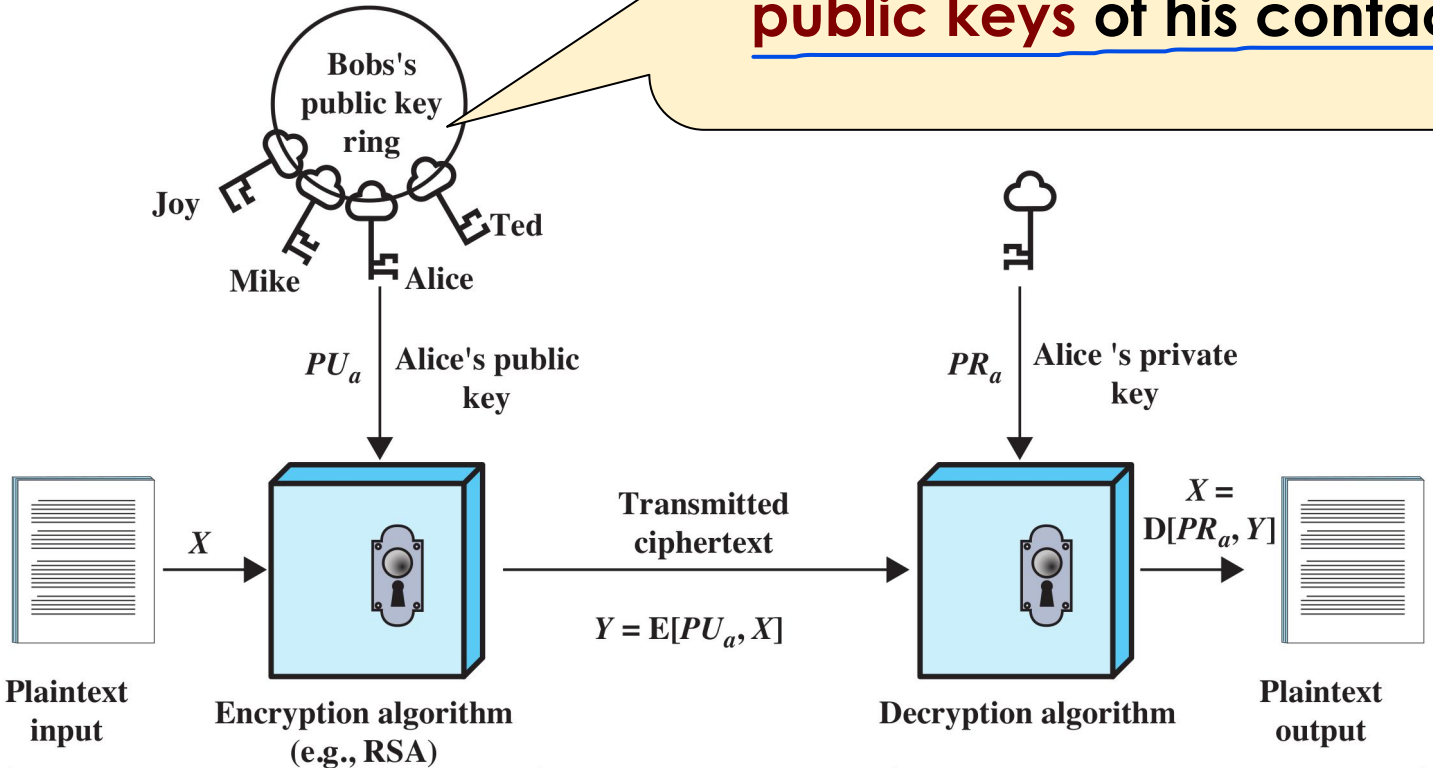


Public-Key Encryptio

Bob has a keyring with all the public keys of his contacts

B O B

A L I C E

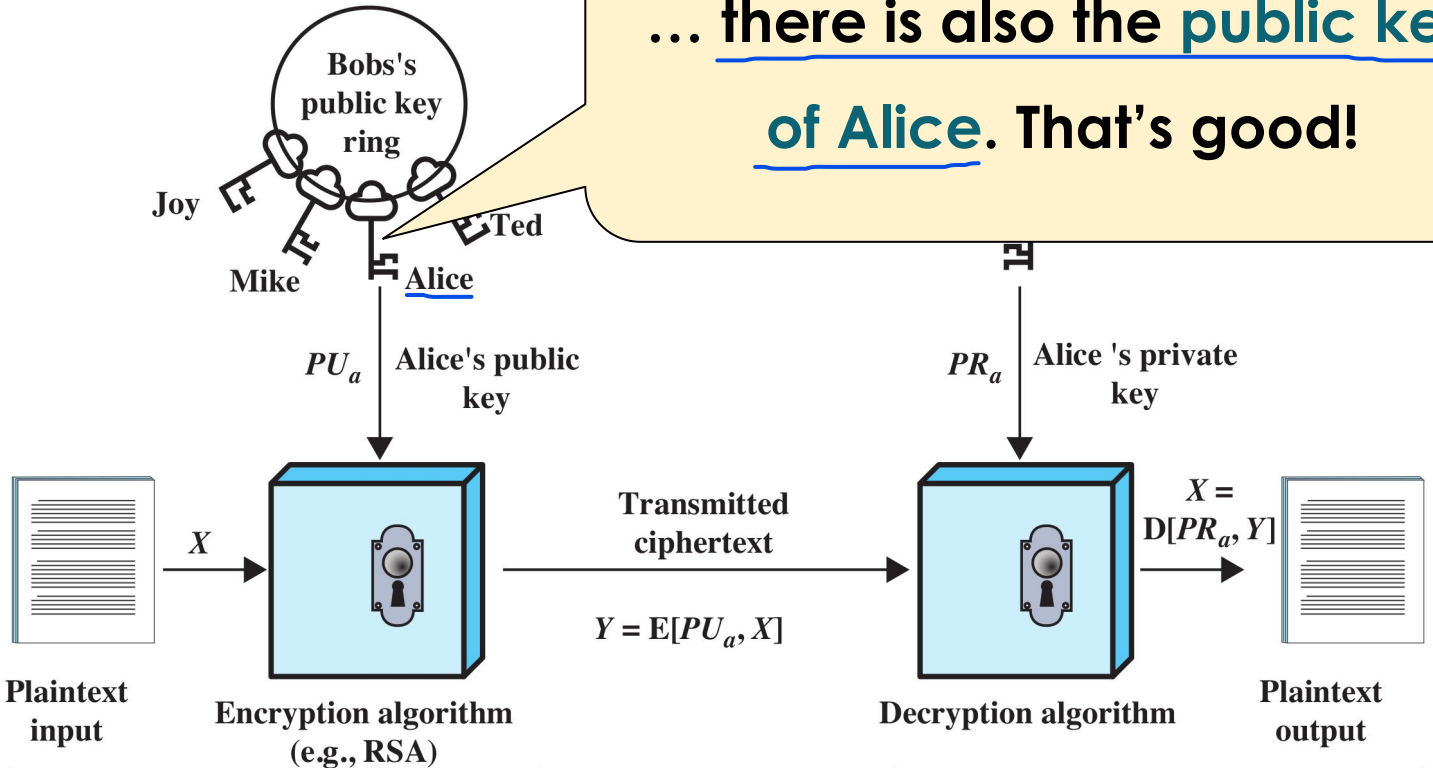


Public-Key Encryption

… there is also the public key of Alice. That's good!

B O B

A L I C E

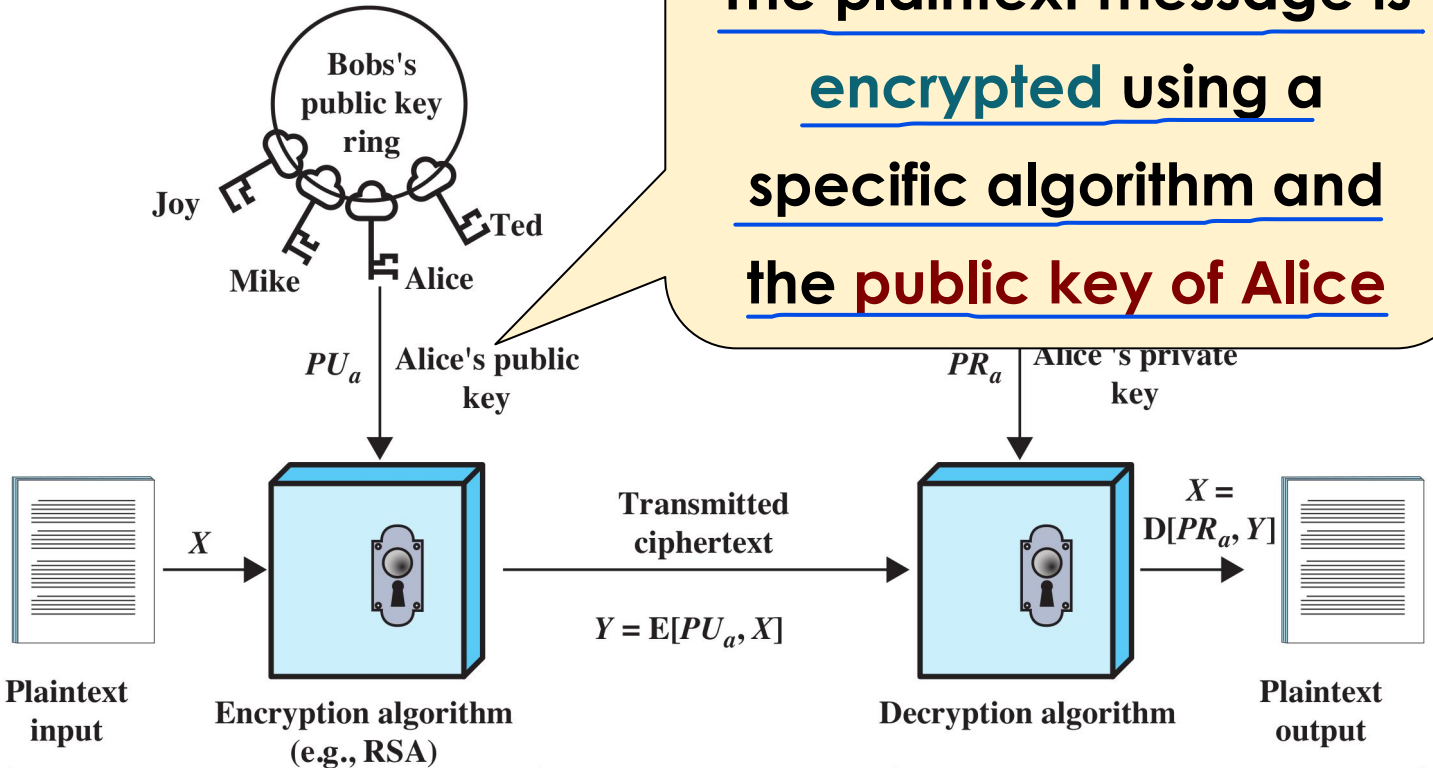


Public-Key Encryption

The plaintext message is encrypted using a specific algorithm and the public key of Alice

B O B

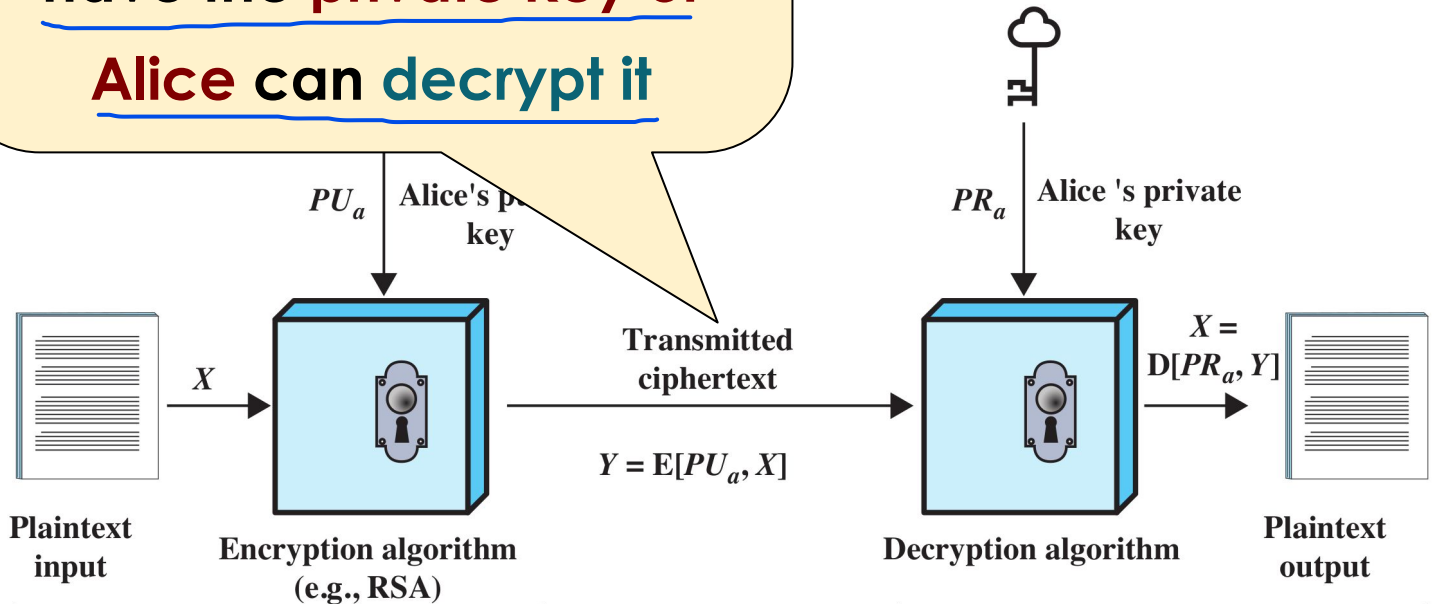
A L I C E



The ciphertext can be transmitted on the Internet since only those who have the private key of Alice can decrypt it

B O B

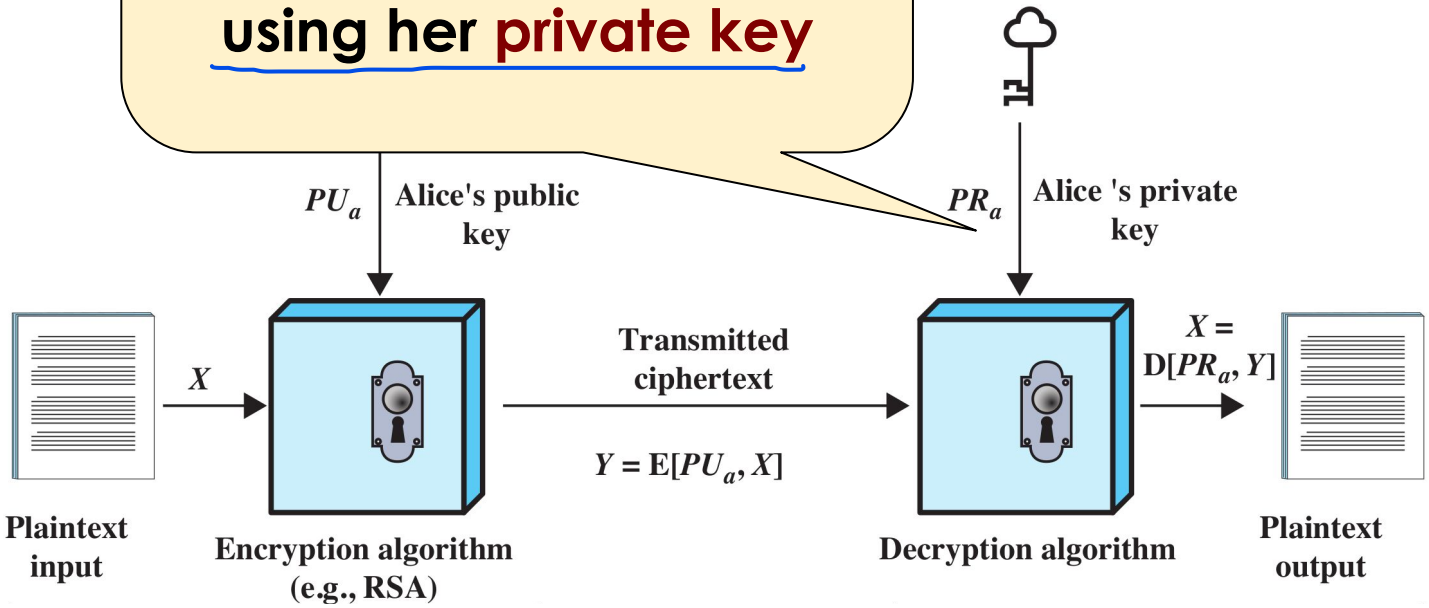
A L I C E



Alice decrypts the encrypted message using her private key

B O B

A L I C E

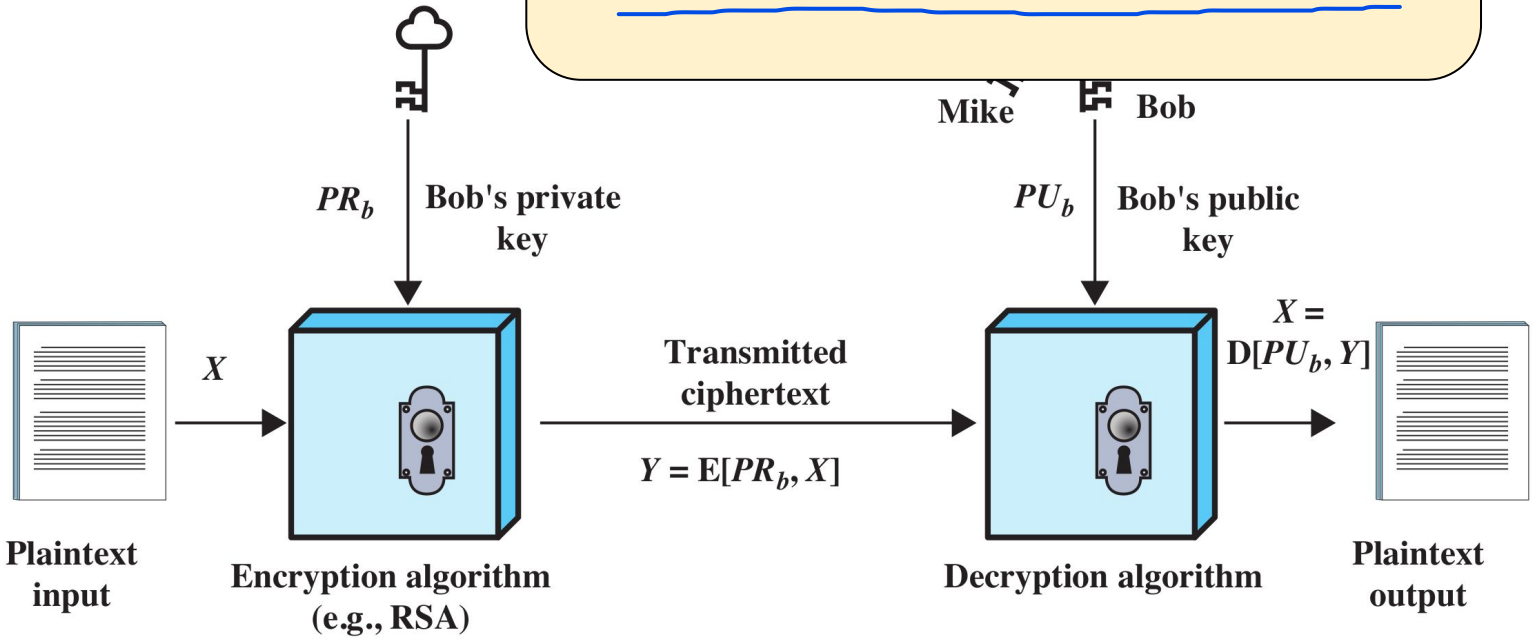


Public-Key Encryption

Public key encryption can be also used in a different way

B O B

A L I C E



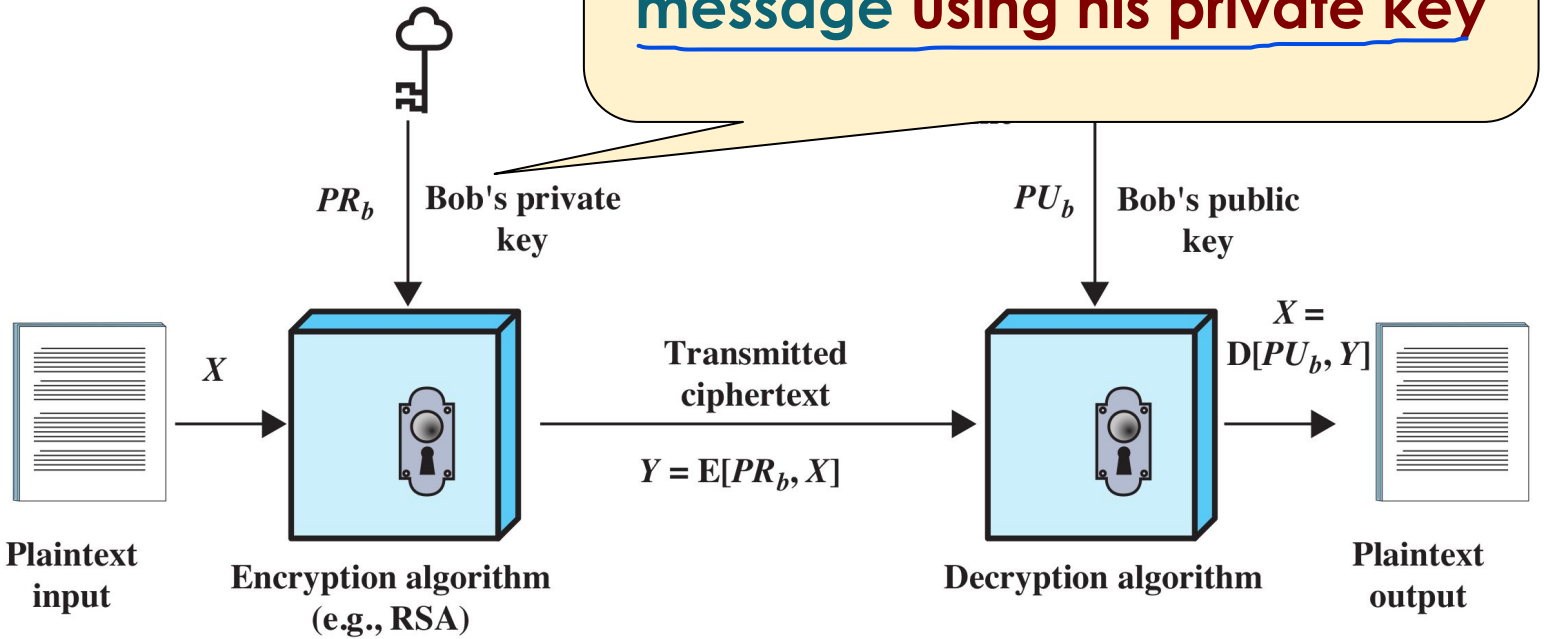
Public-Key Encryption

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

In this case, Bob encrypts the message using his private key

B O B

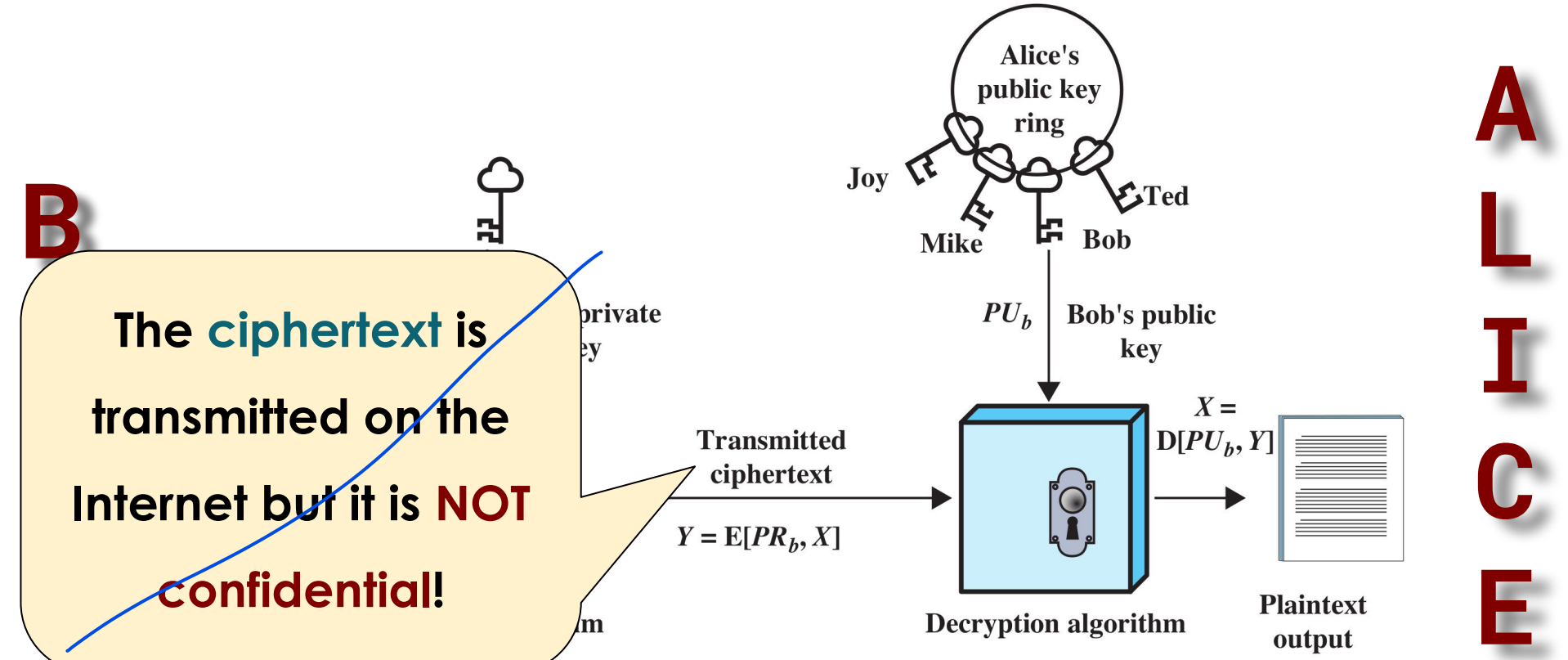
A L I C E



Public-Key Encryption

B

The ciphertext is transmitted on the Internet but it is NOT confidential!



A L I C E

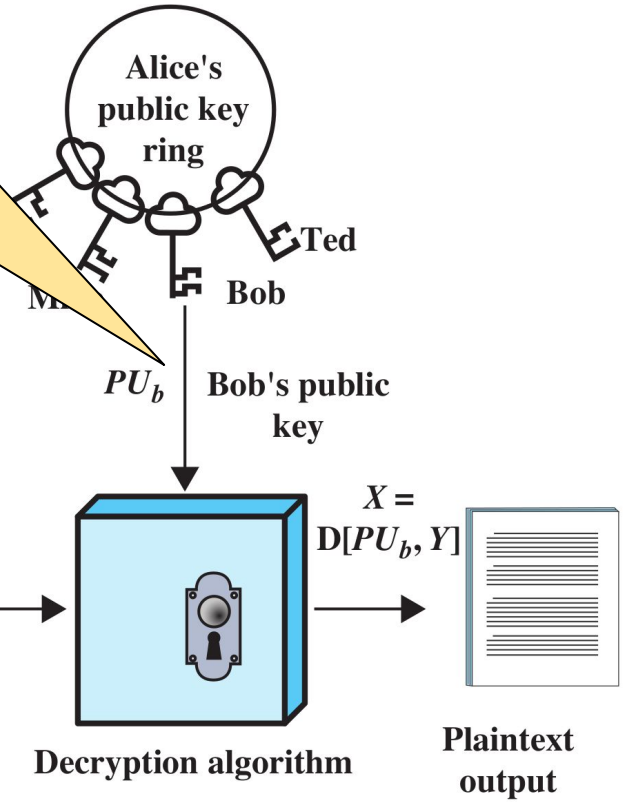
Public-

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Because everyone can get the public key of Bob since it is public!

B

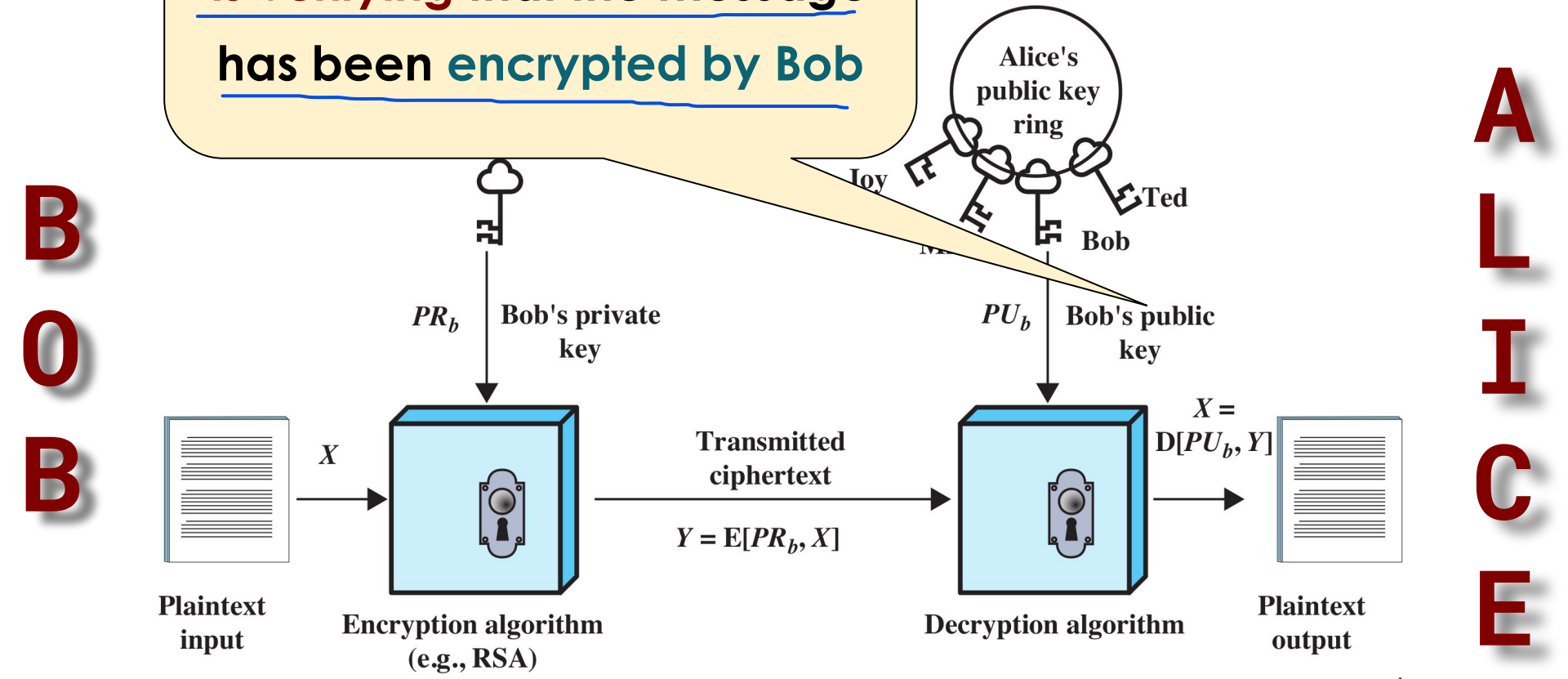
The ciphertext is transmitted on the Internet but it is NOT confidential!



A L I C E

Paternity of Message

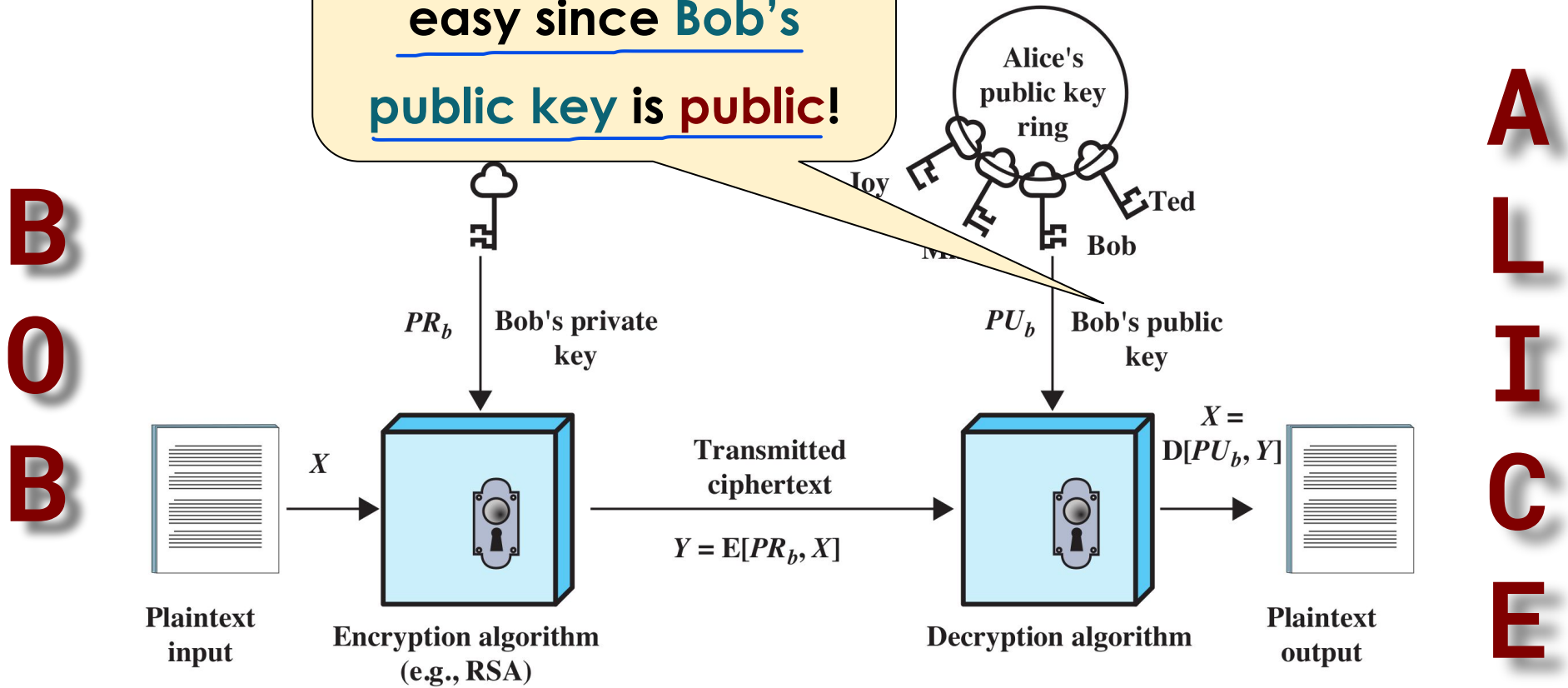
Using Bob's public key, Alice is verifying that the message has been encrypted by Bob



Public-K

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

This verification is easy since Bob's public key is public!

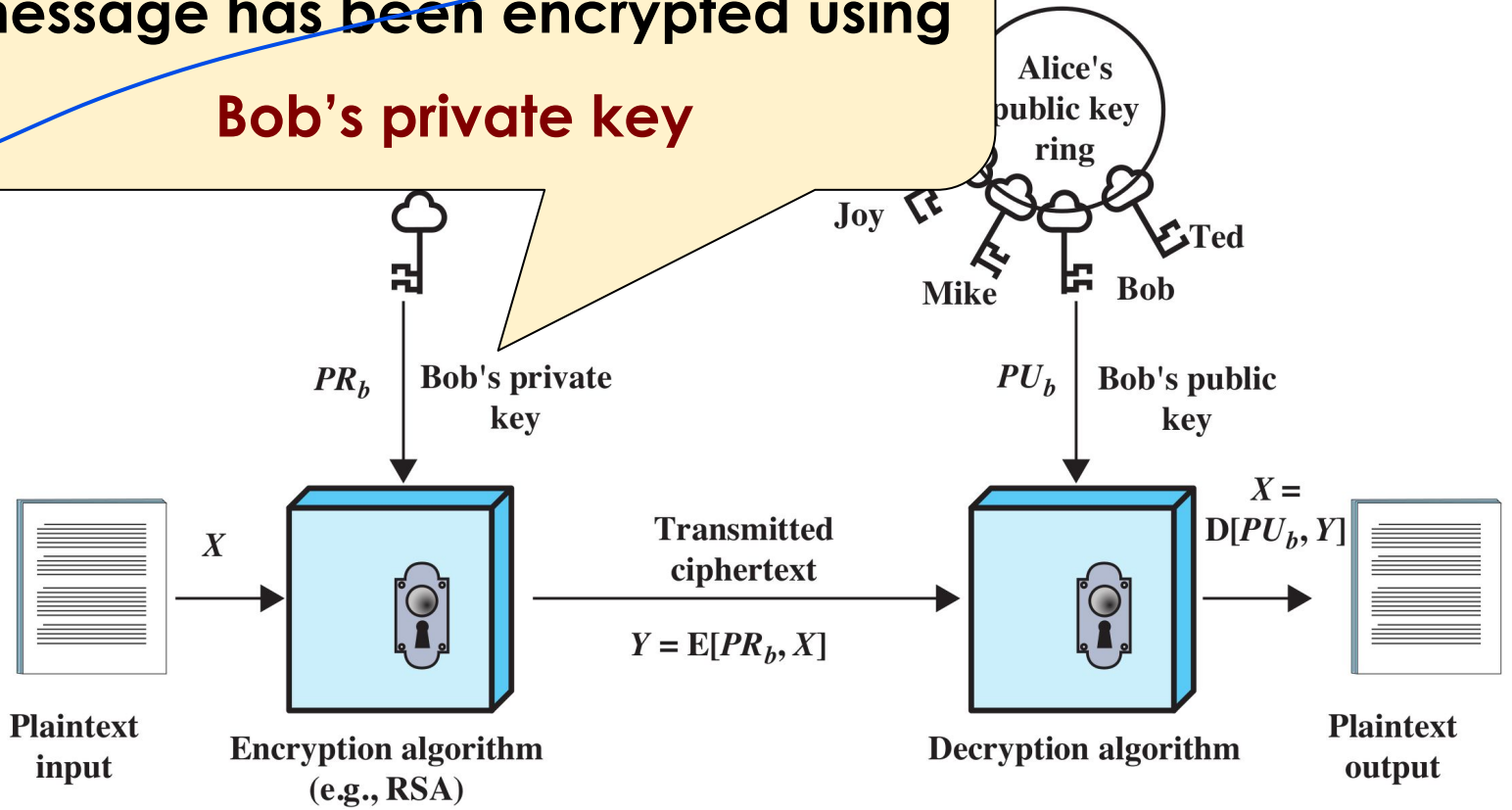


The verification is positive only if the message has been encrypted using

Bob's private key

B O B

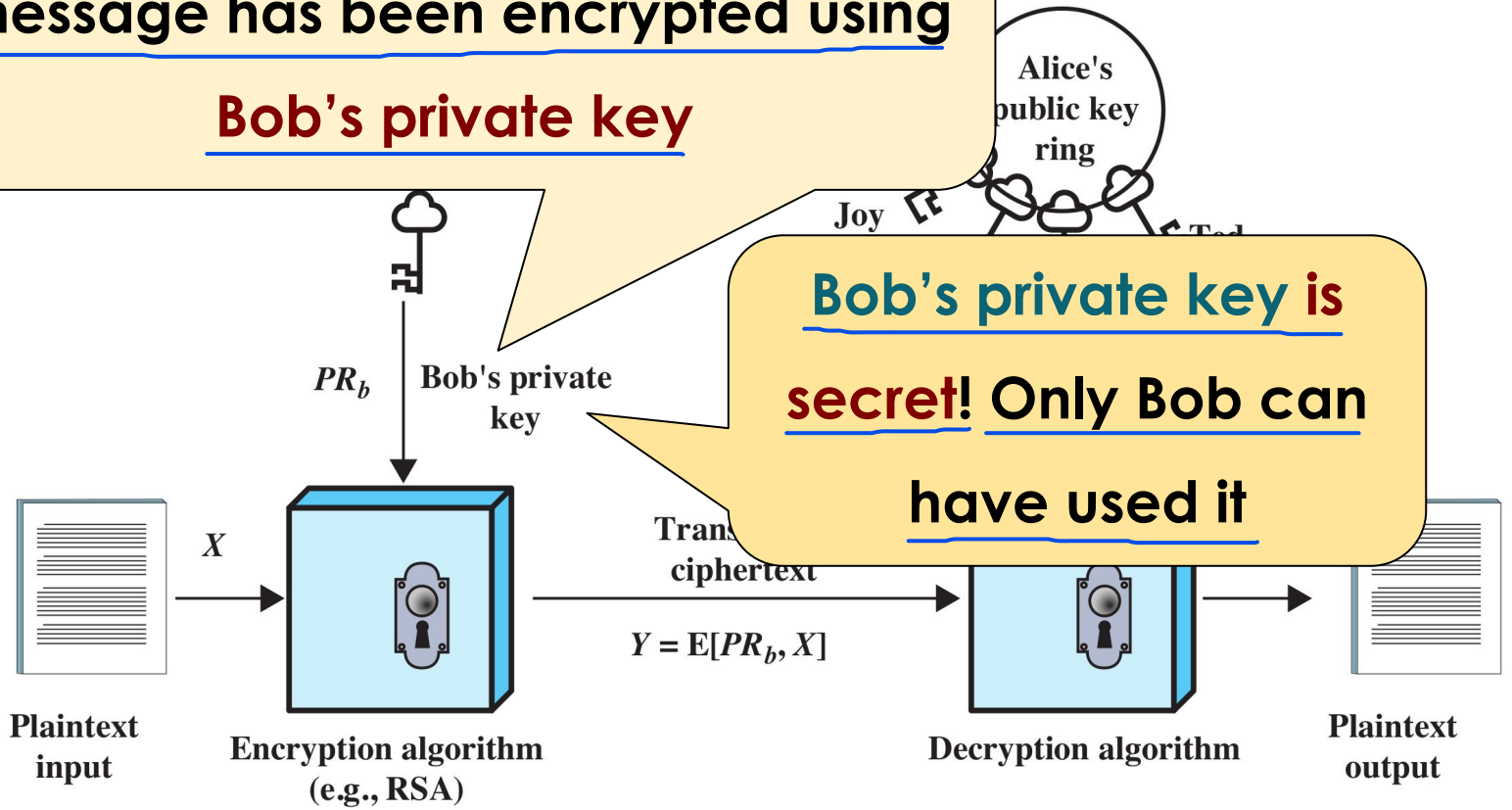
A L I C E



The verification is positive only if the message has been encrypted using Bob's private key

B O B

A L I C E



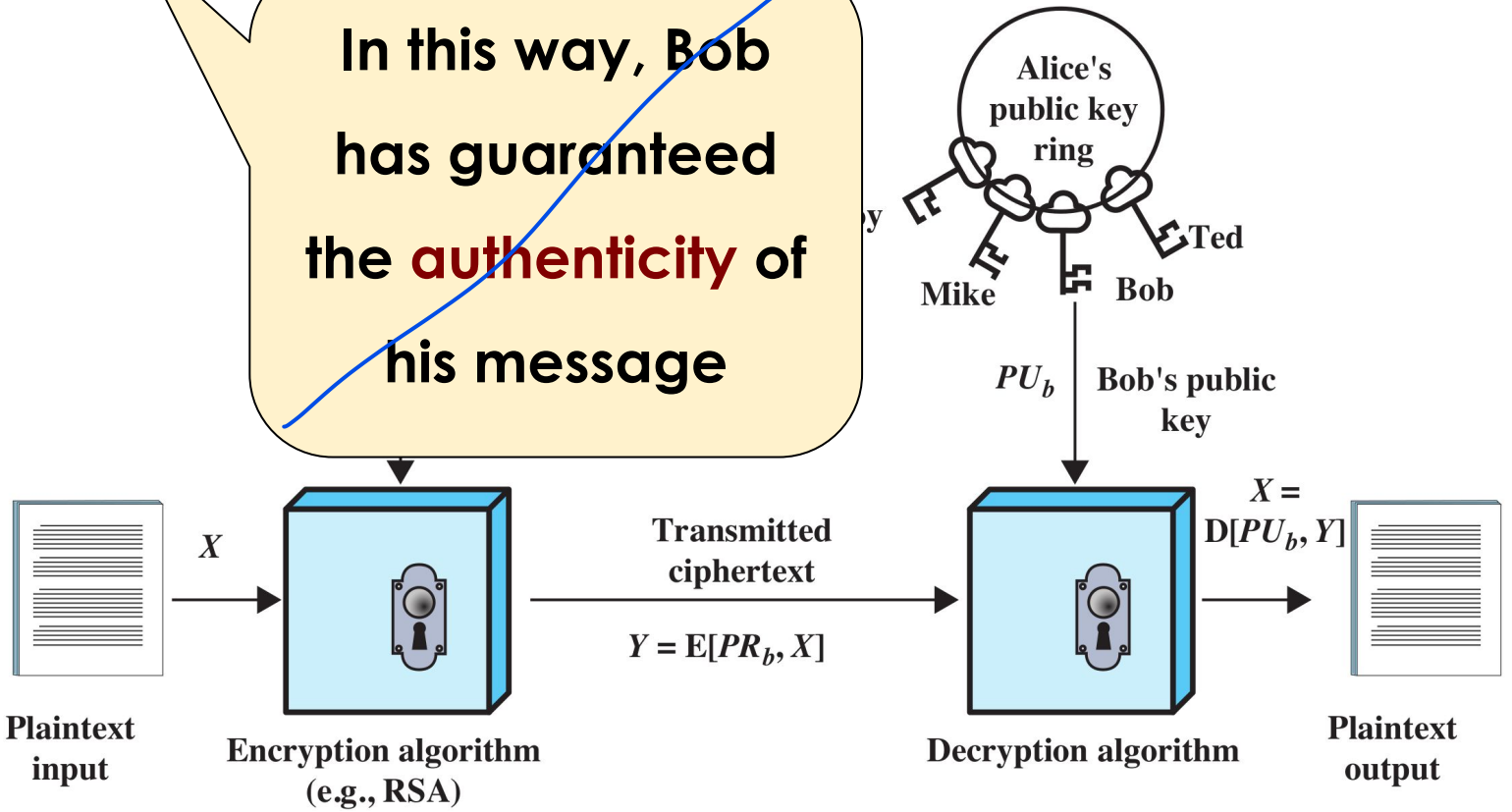
Bob's private key is secret! Only Bob can have used it

Public-Key Encryption

B O B

A L I C E

In this way, Bob has guaranteed the authenticity of his message



Public-Key Encryption

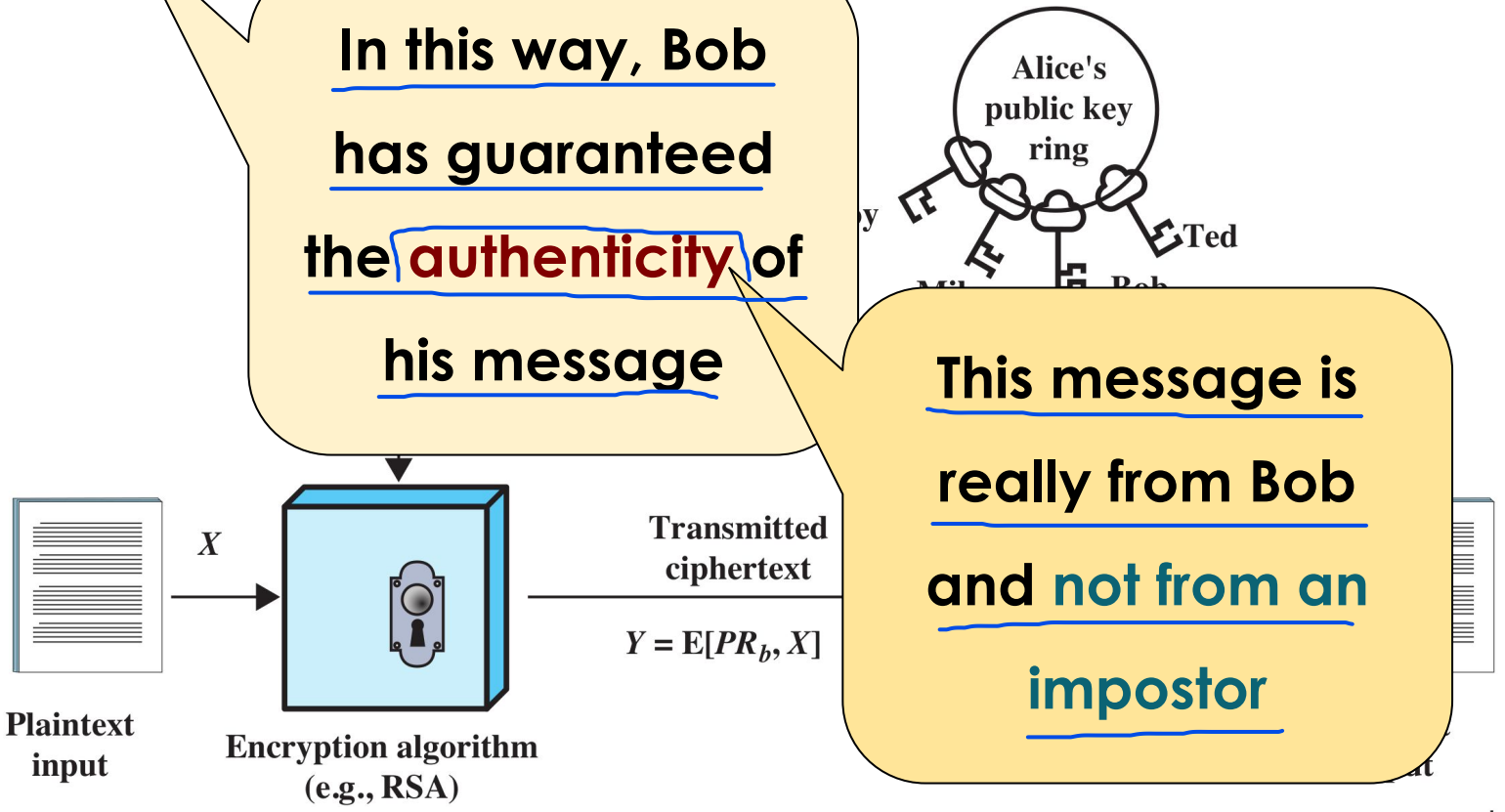
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

B
O
B

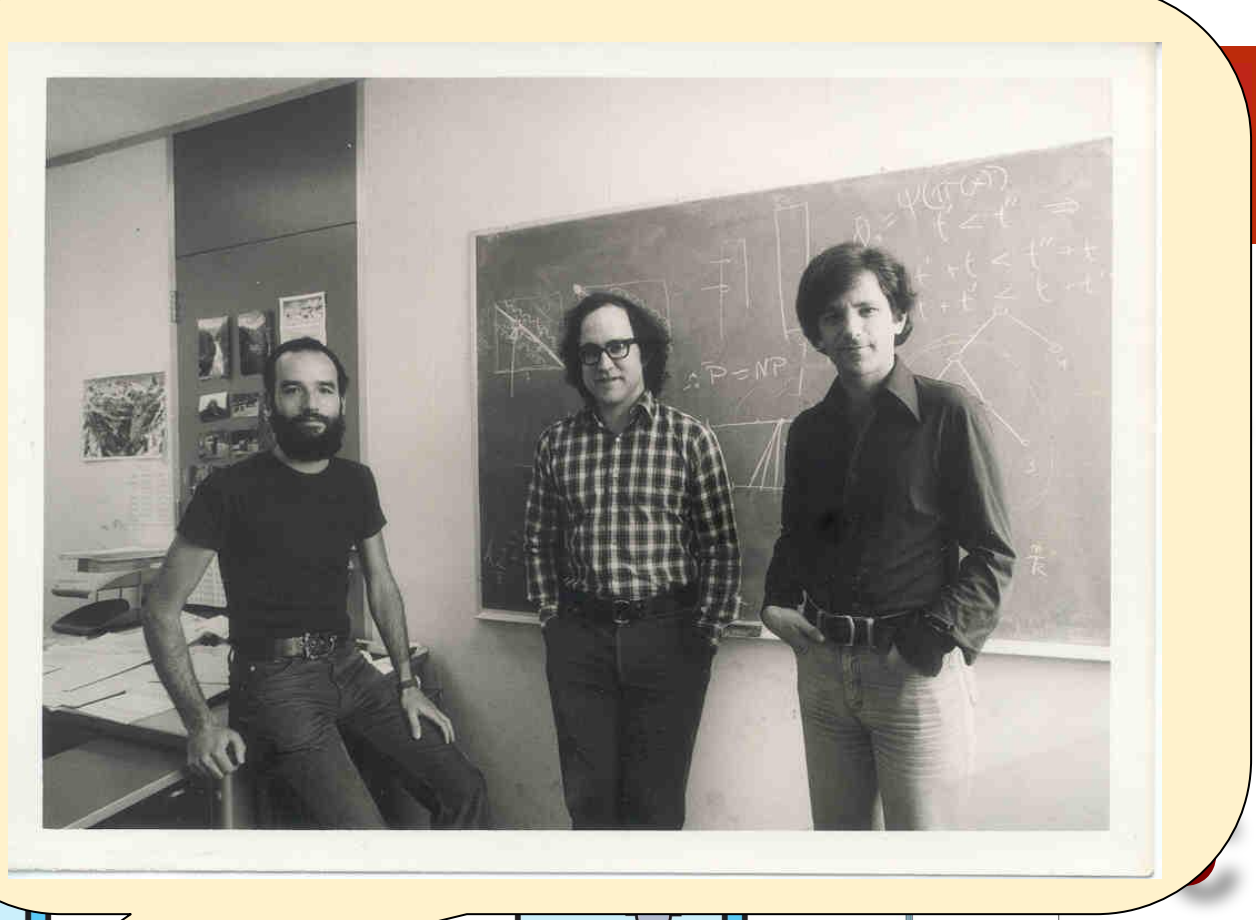
A
L
I
C
E

In this way, Bob has guaranteed the authenticity of his message

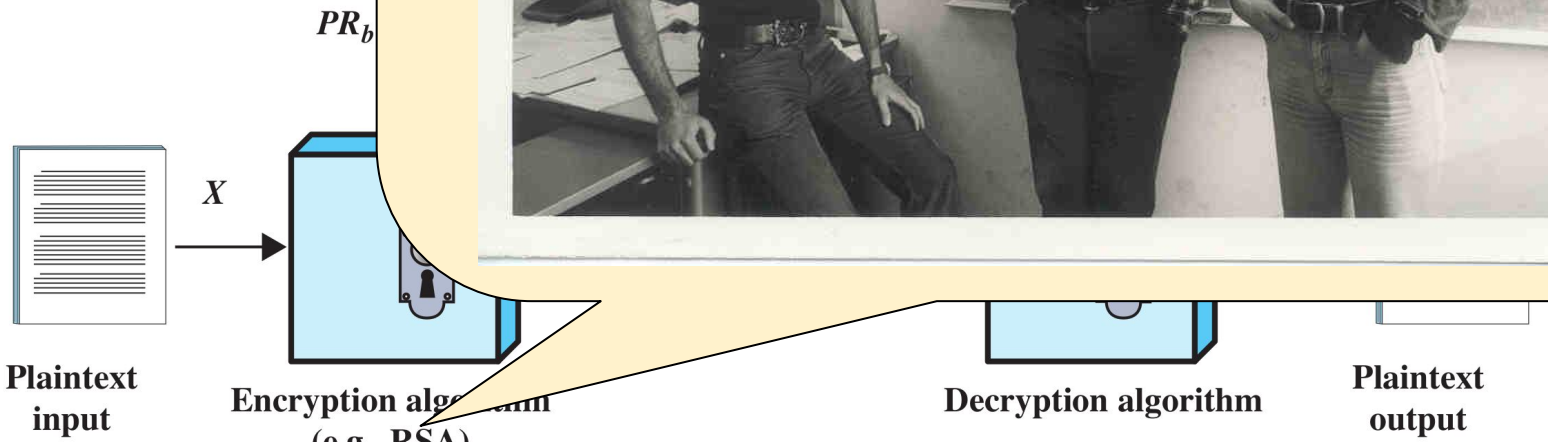
This message is really from Bob and not from an impostor



Public-Key E



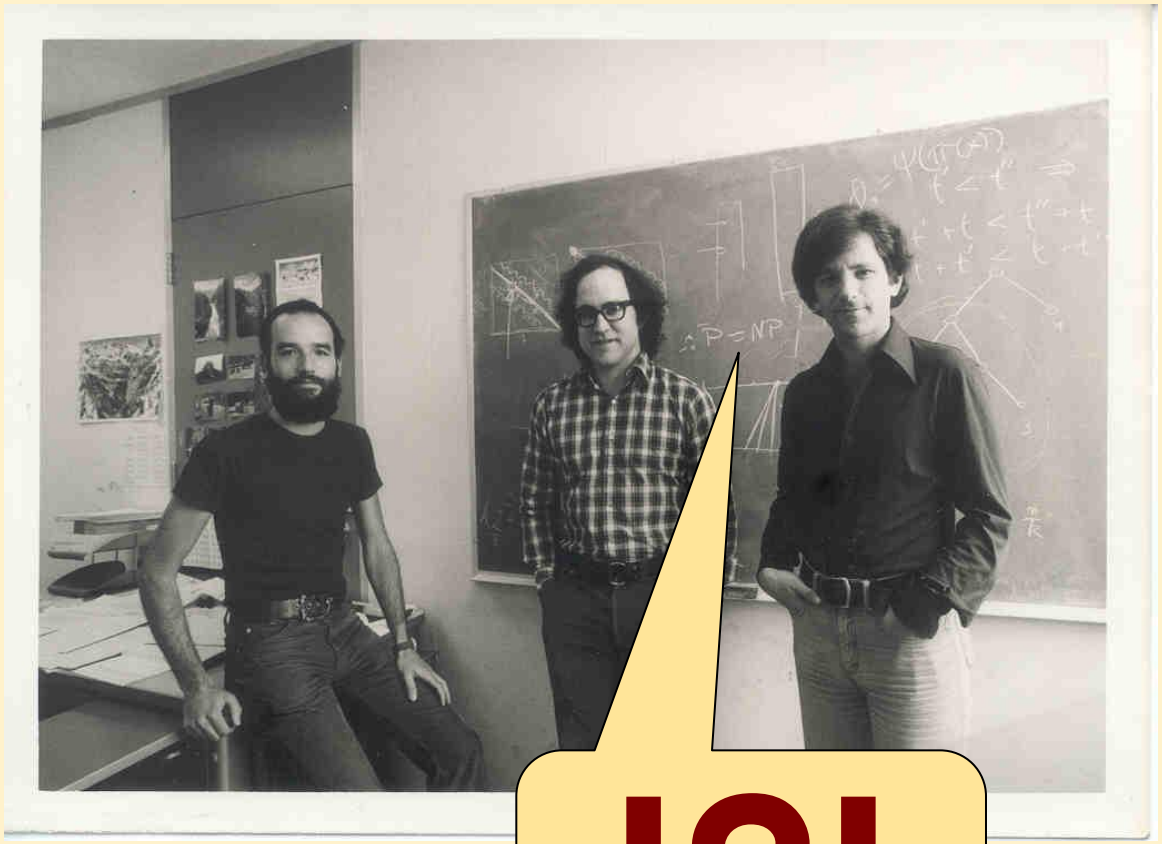
B
O
B



E

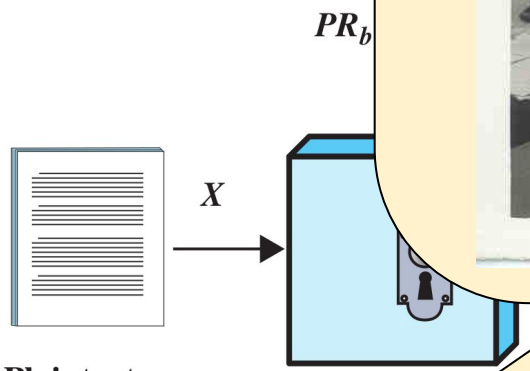
Public-Key E

B
O
B



!?!

E



Public-Key Encryption: requirements

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

6 computationally **easy** to **create key pairs**

1 useful if **either key** can be used for **each role**
(each key can do encryption/decryption)

5 computationally **easy** for sender knowing public key to **encrypt messages**

2 computationally **infeasible** for opponent to otherwise **recover original message**

4 computationally **easy** for receiver knowing private key to **decrypt ciphertext**

3 computationally **infeasible** for opponent to **determine private key** from **public key**

Public-Key Encryption: a big problem

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- The mechanism described before works very well but **there is still a problem**

  A public key is simply a very long string of numbers and letters (for example, a 2048-bit number). On its own, the key does not contain Bob's DNA, his photo, or his ID card. It is just a mathematical object. If you find a public key on the internet labeled "This is Bob's key," you are trusting a label that anyone could have written.

- How is it possible to **guarantee** (i.e. **verify**) that the **Bob's public key is really the public key of Bob**?

  To solve this "big problem," cryptography had to come up with a third-party certification system. We can't trust Bob just because he gives us his key; we need a "digital notary" that everyone trusts.
  This problem is solved through:
  - CAs (Certificate Authorities): Public and official bodies that verify Bob's identity (e.g., by checking his documents) and then digitally sign his public key.
  - Digital Certificates (X.509 Standard): A file that links Bob's public key to his personal information, signed by the CA. It is the digital equivalent of an ID card issued by the city.
  - PKI (Public Key Infrastructure): The entire global infrastructure that manages these certificates (and is what allows your browser to display the green padlock when you visit a banking site, guaranteeing that it is the bank's real public key and not that of a scammer).

- **For example**: *an impostor could **impersonate Bob** with the aim to **publish a fake "Bob's public key"***

Public-Key Encryption: a big problem

If an impostor (Eve) successfully publishes a fake public key under Bob's name, it sets the stage for a classic Man-in-the-Middle (MitM) attack.
The Execution of the Attack
- The Trap: Alice wants to send a confidential message to Bob. She searches for Bob's public key, but unknowingly downloads the fake key published by Eve.
- The Encryption: Alice encrypts her sensitive data using this fake public key, believing it belongs to Bob.
- The Interception: Alice sends the encrypted message over the network. Eve intercepts it.
- The Decryption: Because the message was encrypted with Eve's fake public key, Eve can decrypt it easily using her own private key.

- The mechanism described before **re is still a problem**
- How is it possible to **guarantee** (i.e **ublic key is really the public key of Bob**?
- **For example**: *an impostor could* ***impersonate Bob*** *with the aim to* ***publish a fake "Bob's public key"***

And then?

With what consequences?

Once Eve intercepts and decrypts the message, the consequences can be devastating, splitting into two main scenarios:
- Loss of Confidentiality: Eve reads the entire content of Alice's message. To avoid detection, Eve can then re-encrypt the message using Bob's real public key and forward it to Bob. The Consequence: Bob receives the message and it decrypts perfectly. Neither Alice nor Bob realizes that Eve has just read their private data.
- Loss of Data Integrity: Eve doesn't just read the message; she changes it. For example, if Alice sent a message saying "Transfer $100 to Bob's account," Eve can alter the plaintext to read "Transfer $10,000 to Eve's account." Eve then encrypts this modified message with Bob's real public key and sends it to him. The Consequence: Bob decrypts a message that looks mathematically valid, but the content has been maliciously altered, leading to financial or operational fraud.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

GNU Privacy Guard

Usage examples of GPG

Encrypted emails?

Check it out!

T4

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

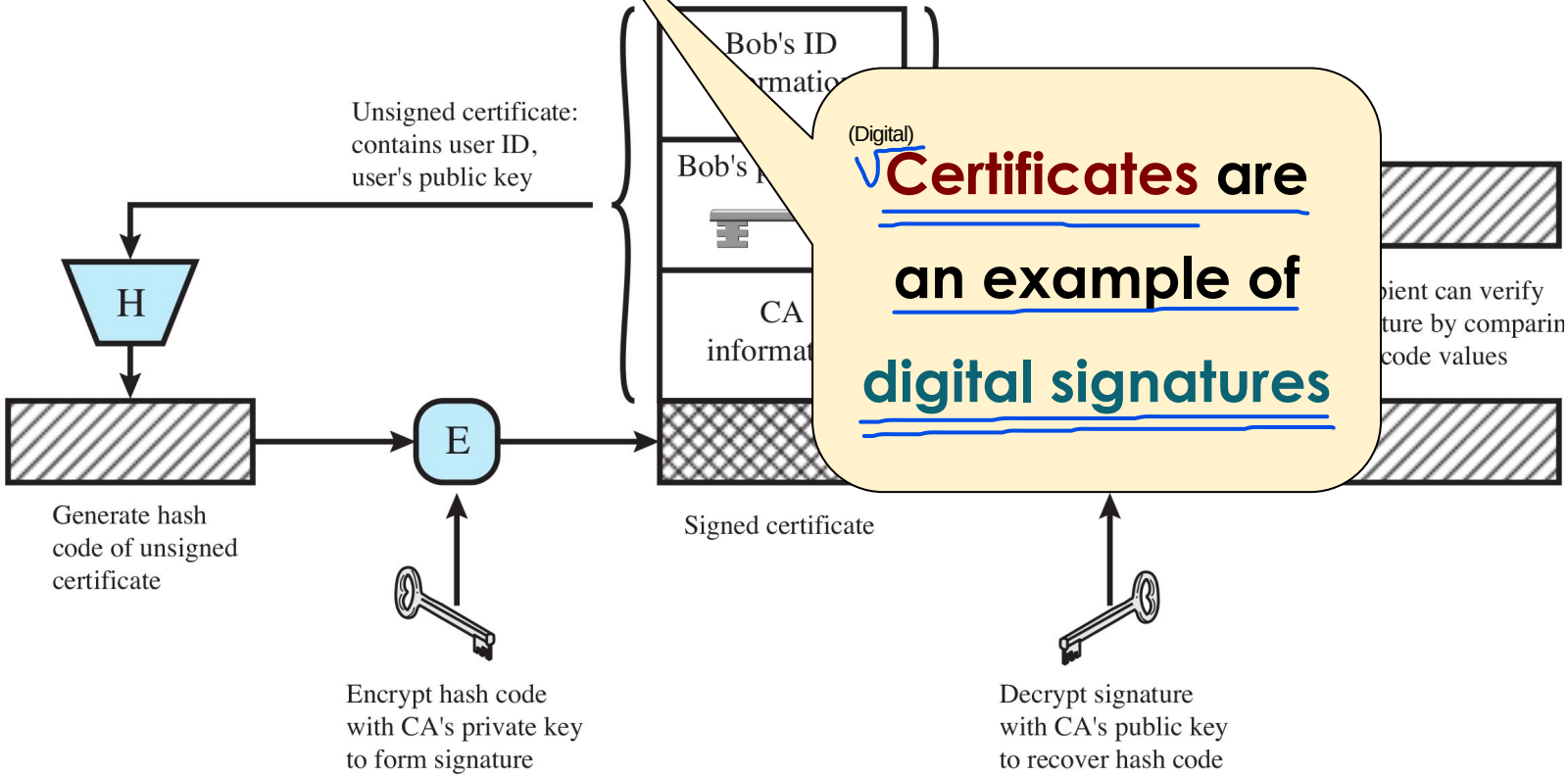
Digital Signatures

Digital Signatures

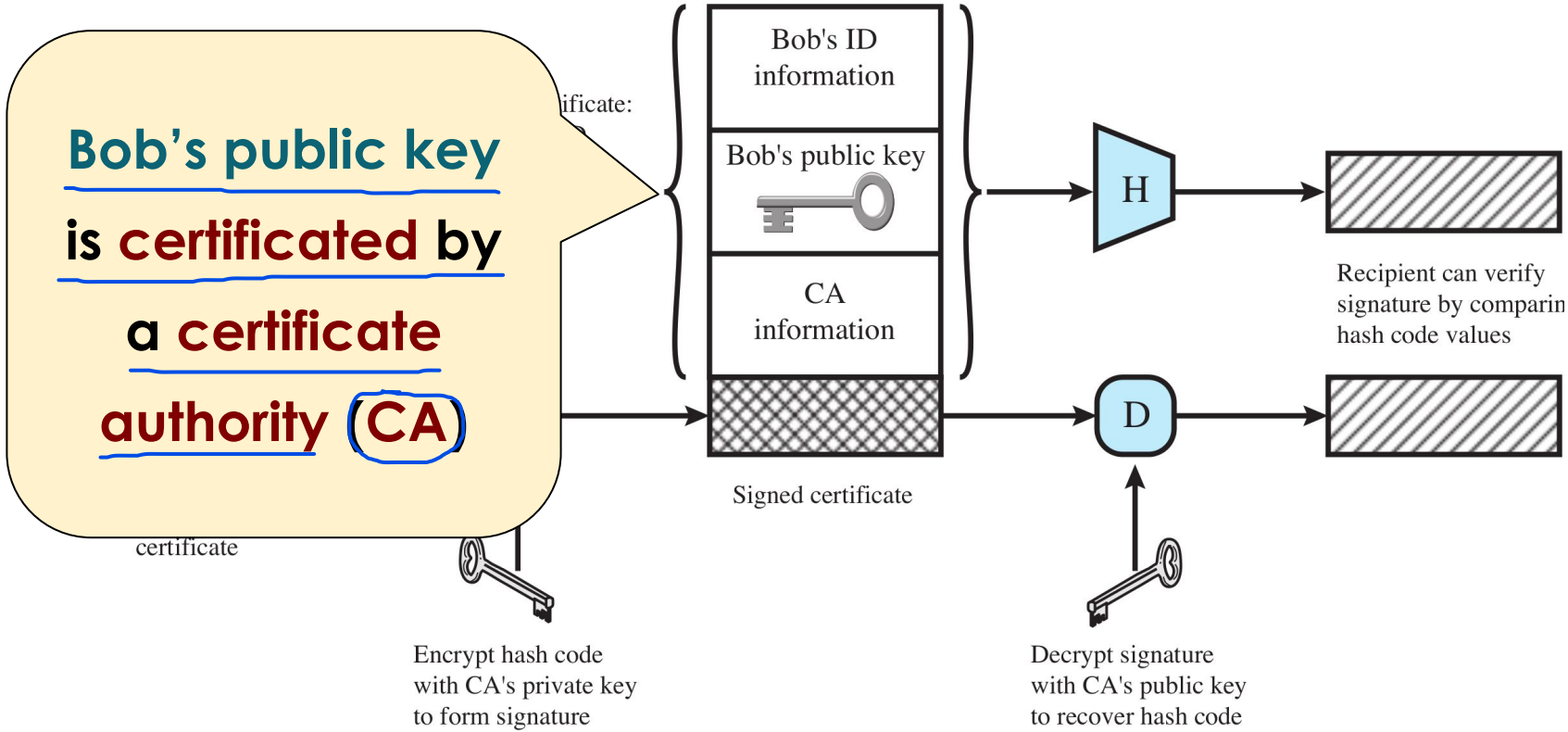
A digital signature provides both Authenticity (proving who sent it) and Data Integrity (proving it hasn't been changed), but not Confidentiality.

- The **Digital Signature** is used for **authenticating** both **source** and **data integrity** → 1) Source Authentication: It guarantees the identity of the person who signed the document. If you receive a digitally signed file from Bob, you have mathematical certainty that he sent it and not an imposter (such as Eve). This is possible because the signature is created using Bob's Private Key, which only he possesses and keeps secret. Consequently, Bob can never say, "I didn't sign this" (the property of Non-repudiation), because no one else could have used his private key.
2) Data Integrity: It ensures that the message has not been altered, modified, or corrupted during transmission (no tampering attacks). If a malicious user changes even a single comma or a single bit of the message after it has been signed, the digital signature will instantly become invalid, and the recipient's computer will flag the error.
- Created by **encrypting hash code with private key**
- Does **not** provide **confidentiality**:
  - even in the case of **complete encryption** (e.g., not only the hash code)
  - the message is **safe from alteration** but **not eavesdropping** (in caso di intercettazione)

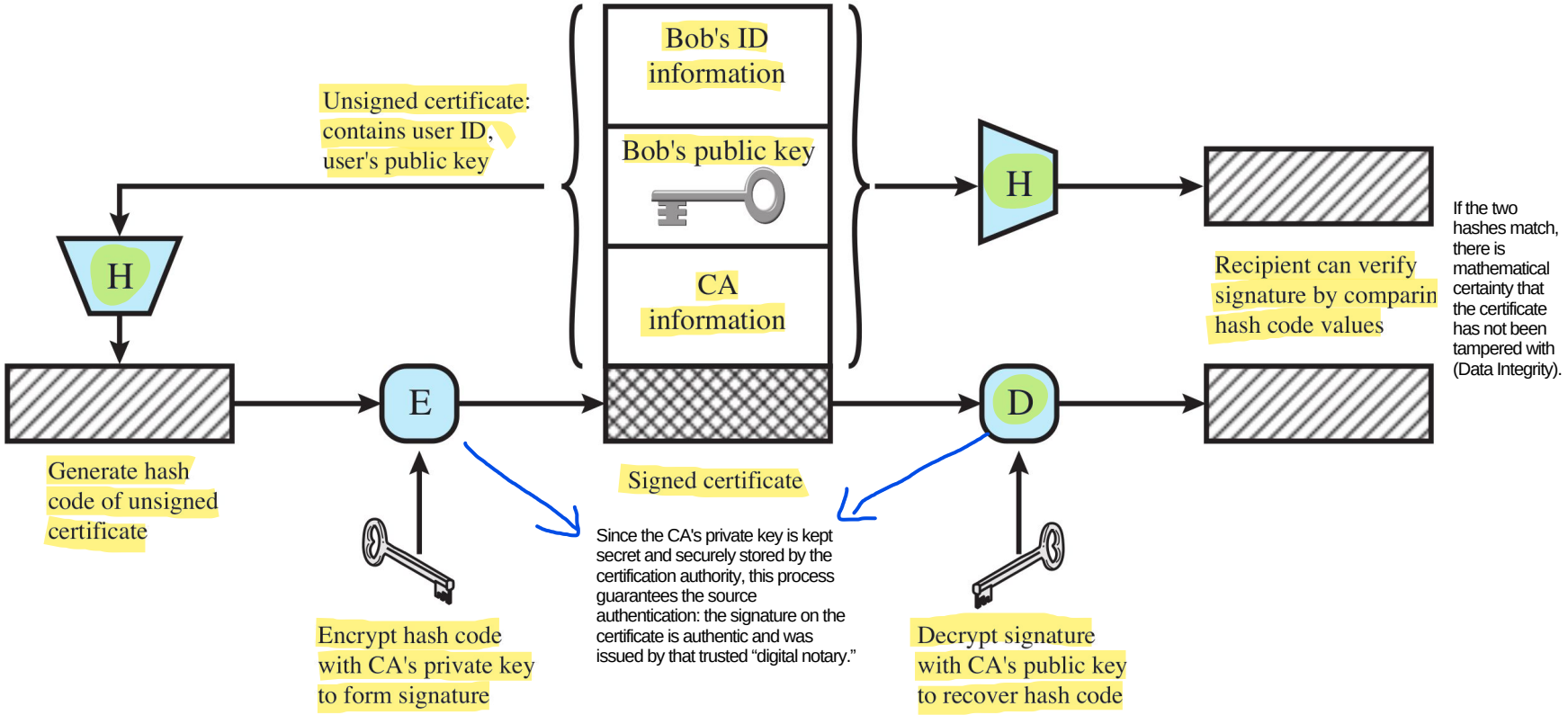
(Digital) Public-Key Certificates



Public-Key Certificates



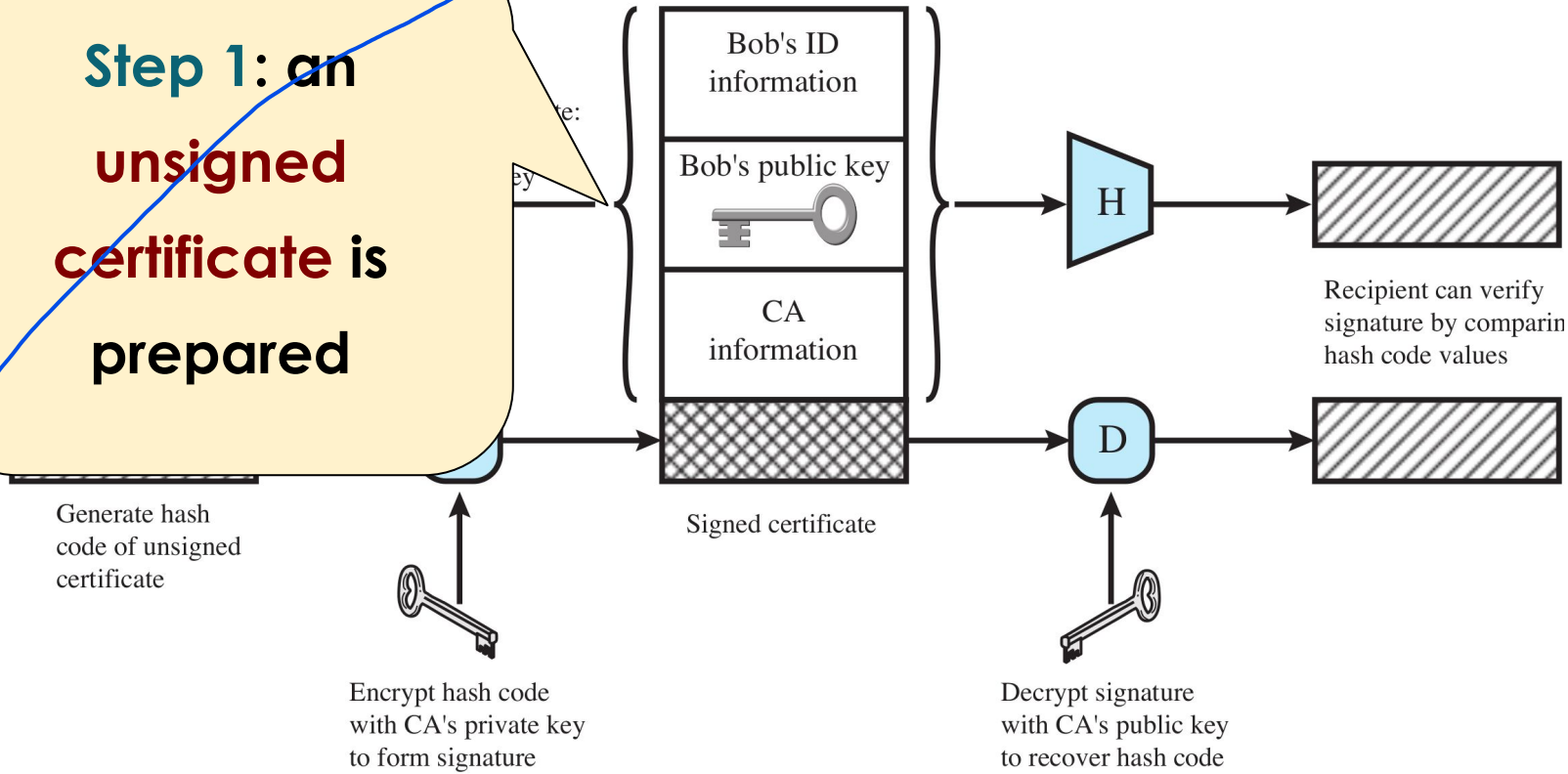
Public-Key Certificates



Public-Key Certificates

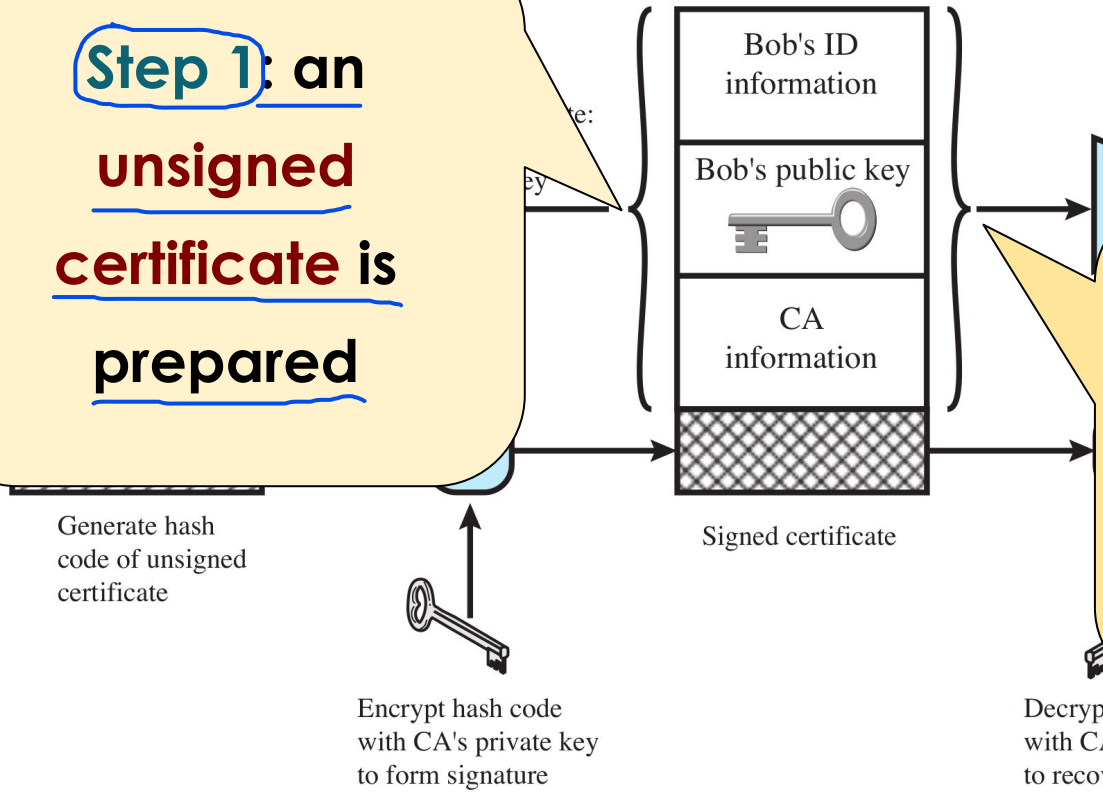
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Step 1: an unsigned certificate is prepared



Public-Key Certificates

Step 1: an unsigned certificate is prepared



Since it is unsigned then it is easy to forge the certificate

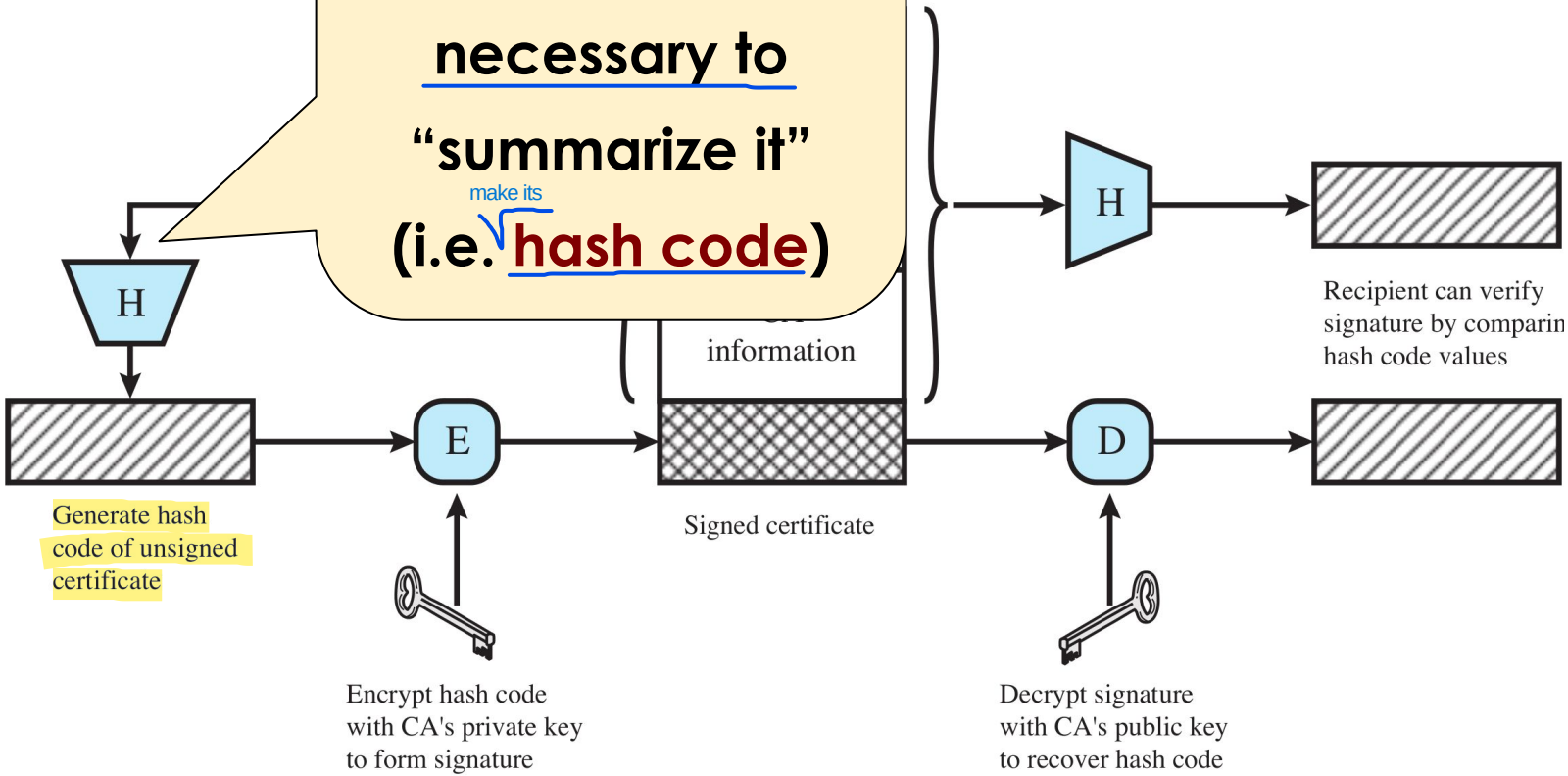
i.e.

(create a fake one)

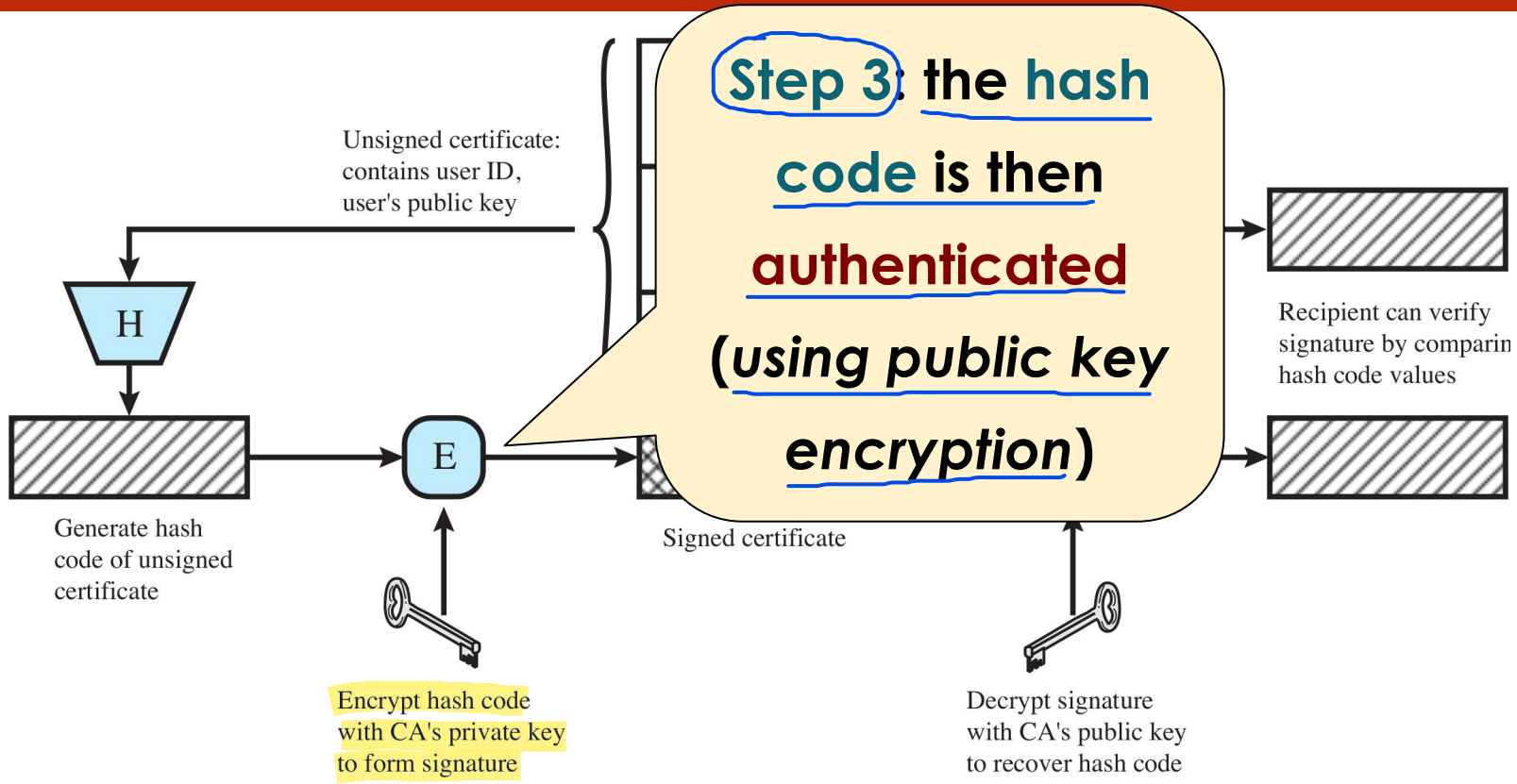
Public-Ke

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Step 2: to sign the certificate it is necessary to "summarize it" (i.e. make its hash code)

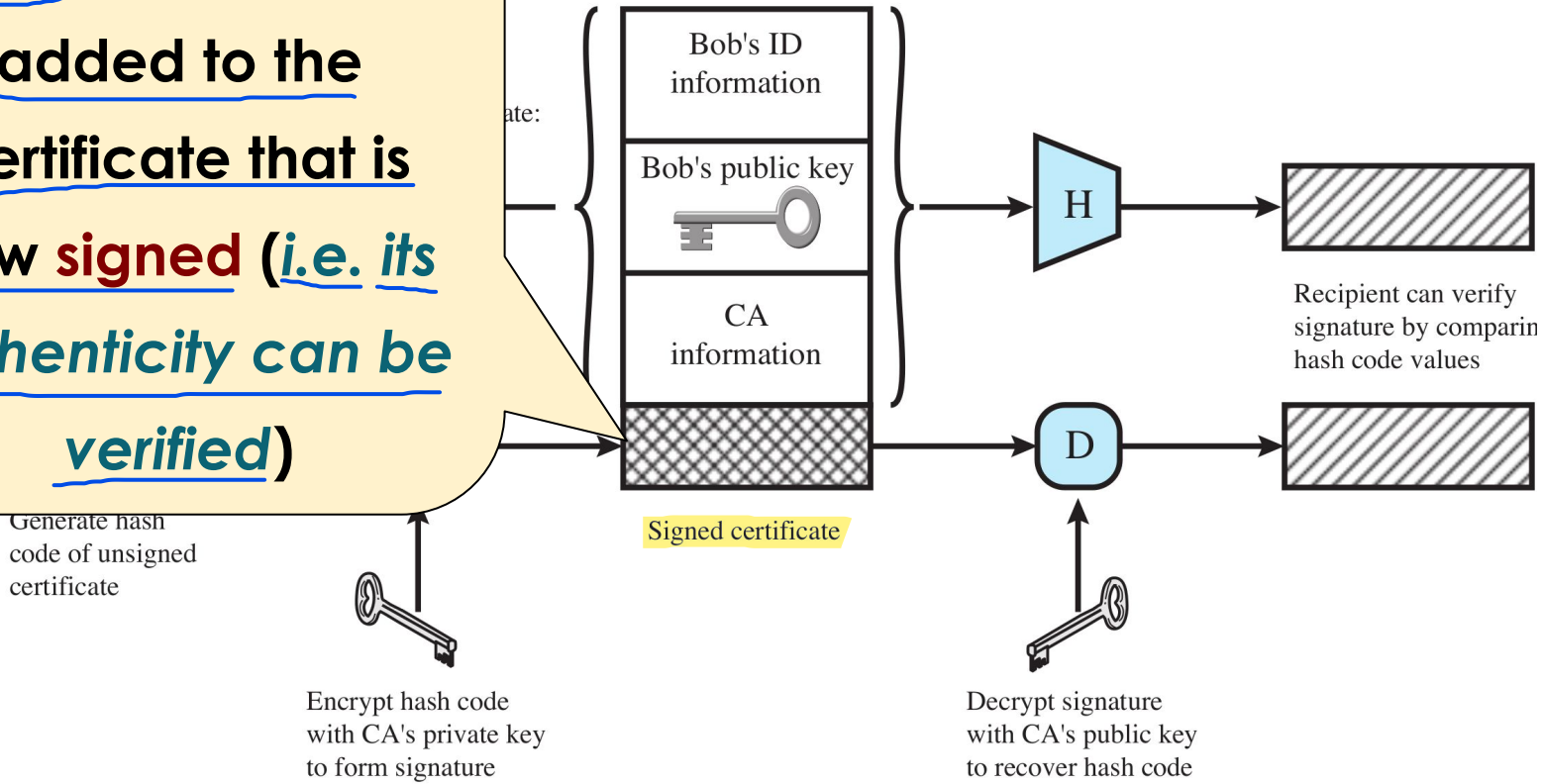


Public-Key Certificates

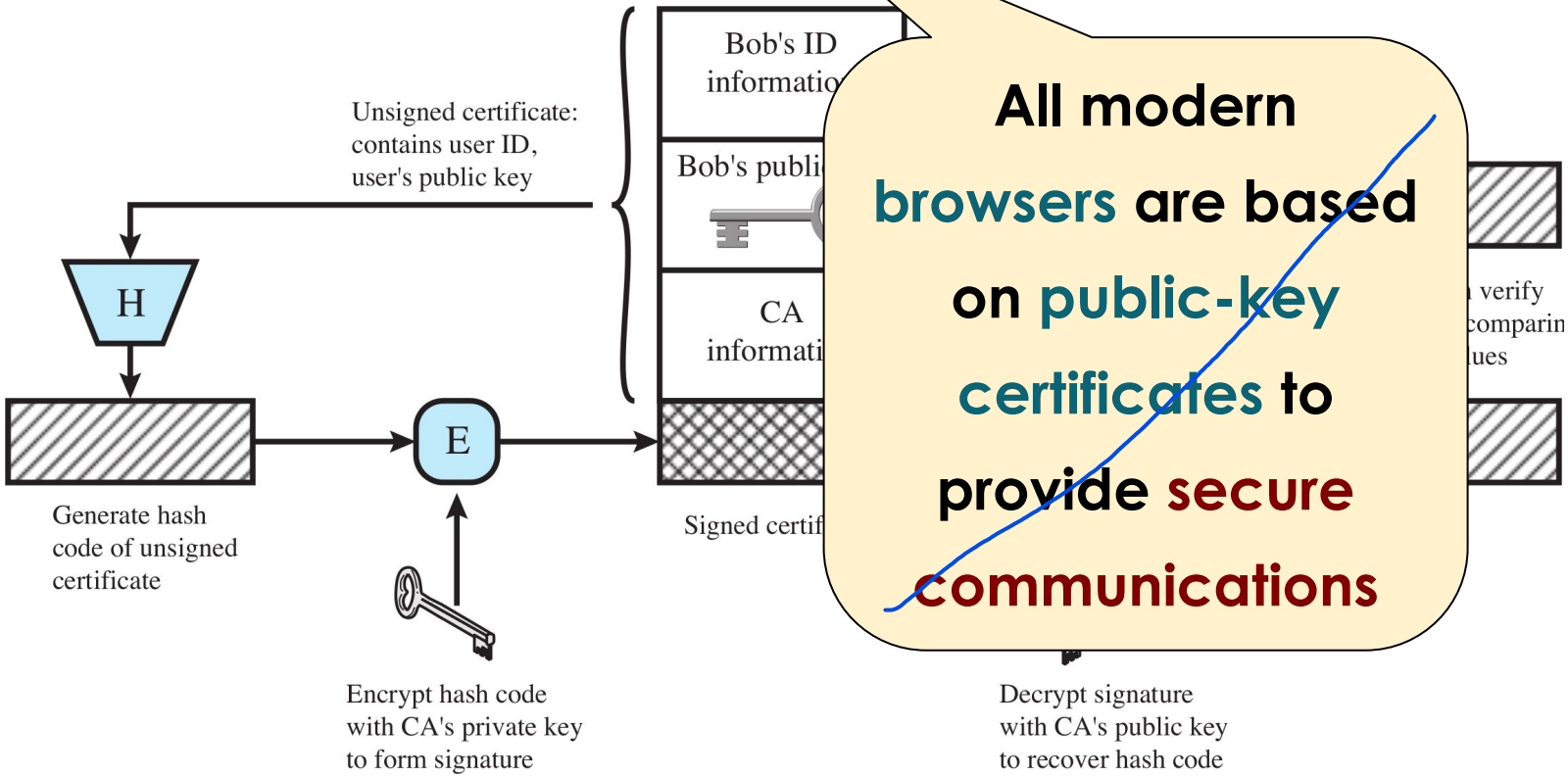


Public-Key Certificates

Step 4: … and then added to the certificate that is now signed (*i.e. its authenticity can be verified*)

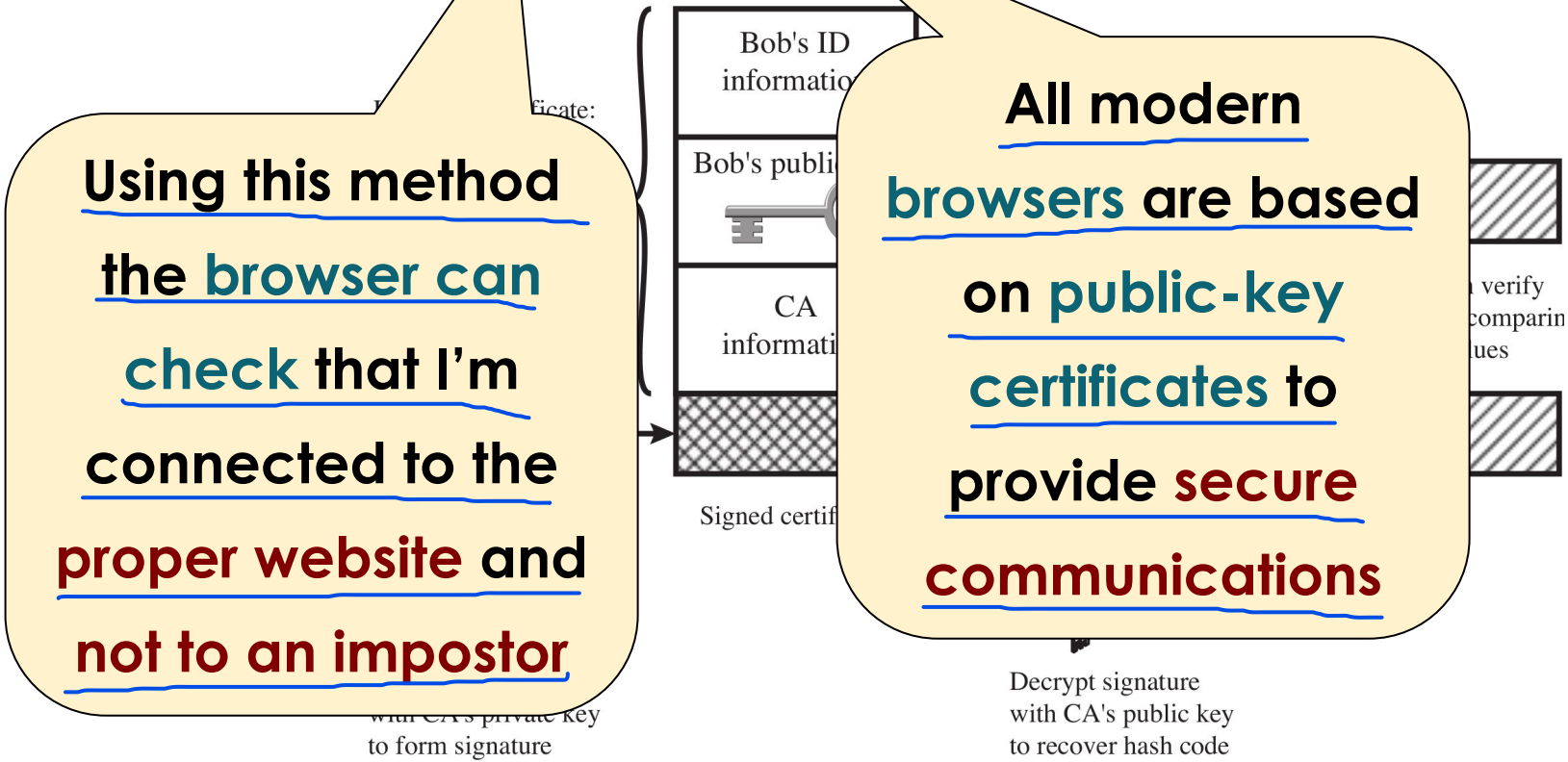


Public-Key Certificates



All modern browsers are based on public-key certificates to provide secure communications

Public-Key Certificates



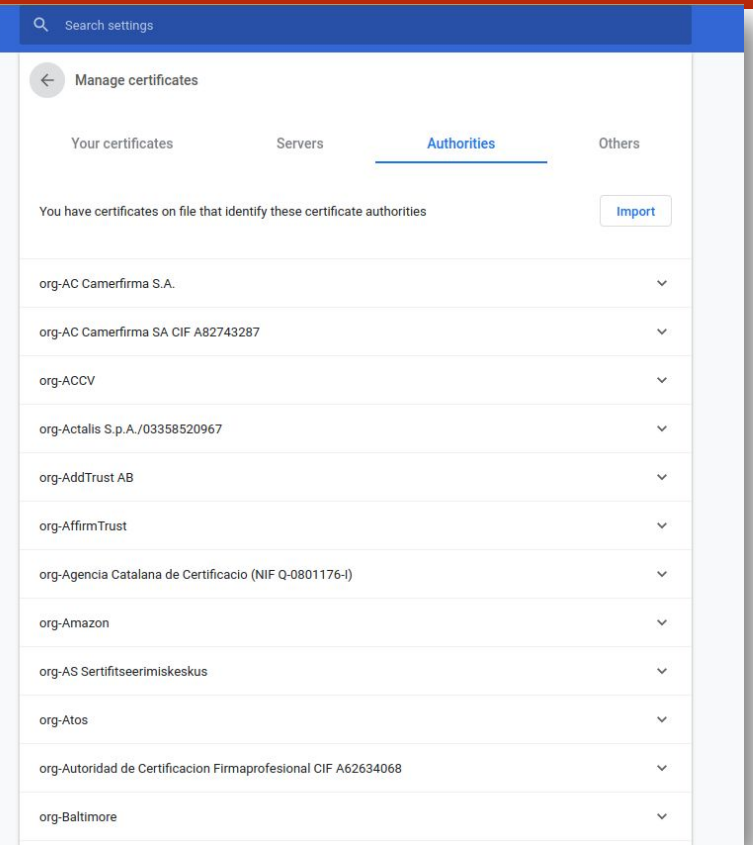
Using this method the browser can check that I'm connected to the proper website and not to an impostor

All modern browsers are based on public-key certificates to provide secure communications

Web Browsers

Browsers do not know the public keys of all the billions of websites that exist. They only know the public keys of a few dozen Certificate Authorities. Thanks to the "chain of trust," by trusting the CA, the browser is able to verify and trust the public key of any website in the world signed by that CA.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



In Google Chrome (like in any other Web Browser) there is a list of recognized certificates from trusted Certificate Authorities

That are the public keys (of the Certificate Authorities) used to verify the certificates provided by the Web sites (*Amazon, Google...*)

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Pharming Attacks

Example of pharming attack to HTTP and HTTPS

How does it works?

Check it out!

T5

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

What's next?

Quantum Computing vs. Encryption

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- "A **quantum computer** is a computer that takes advantage of **quantum mechanical phenomena**" (Wikipedia)
- Thanks to the "**quantum parallelism**" it can **speedup the execution of some specific computations** (**NOT all**)
- What is the expected effect of quantum computing on encryption?

Quantum Computing vs. Encryption

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Most current **symmetric cryptographic algorithms** and **hash functions** are considered to be **relatively secure**

  Quantum computers can attack these algorithms using Grover's Algorithm. Grover's algorithm does not break the logic of AES or SHA; it just speeds up the brute-force process. Specifically, it applies a quadratic speedup (it turns the difficulty from 2^n into 2^{n/2}). Because it only halfs the security level, the fix is incredibly simple. To achieve 128-bit security, you just upgrade from AES-128 to AES-256 (2^{256} halved by Grover is still 2^{128}). For hashes, you just move to SHA-384 or SHA-512.

- In case, it is possible to switch to **longer keys** and **hashes**

- **This is not true for current public-key algorithms**

  Quantum computers can run Shor's Algorithm. Unlike Grover's algorithm, which just speeds up brute force, Shor's algorithm completely solves the underlying mathematical problems in polynomial time. If you increase an RSA key from 2048 bits to 4096 bits, it makes almost no difference to Shor's algorithm. A quantum computer would only need a fraction more time to break the larger key. The mathematical foundation itself becomes useless.

- A new generation of (**post-quantum cryptography**) algorithms is being developed, standardized and adopted in software (e.g. browsers)

Quantum Computing vs. Encryption

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Barış Ege

PRINCIPAL SECURITY ANALYST, KEYSIGHT DEVICE SECURITY TESTING

#Cybersecurity

#Quantum Computer

#Quantum Computing

NETWORK VISIBILITY + SECURITY

Security Highlight: China's Quantum Leap, and Why RSA Isn't at Risk (Yet)

2024-10-28 | 4 min read

Recent headlines suggested that Chinese scientists have the potential to break military-grade RSA encryption using quantum computers, causing concerns over online data security. However, upon closer inspection, these claims do not yet signal the end of modern encryption. Here's what you need to know.

Understanding the breakthrough

The Chinese Journal of Computers recently published a paper in which researchers from Shanghai University factored a 22-bit integer using D-Wave's quantum annealing machine. The research showcases how quantum annealing can be leveraged to transform cryptographic attacks into solvable combinatorial optimization problems.

The team factored a 22-bit number, a far cry from the 2048-bit keys currently used for secure encryption. The difference between a 22-bit and a 2048-bit number is exponential, and this study, while a step forward, doesn't yet threaten RSA encryption as we know it.

The article points out that, while the experiment is an interesting technical advancement, the threat is not immediate for two reasons:

- Scale: The researchers cracked a 22-bit key. Current security standards use 2048-bit keys. The difference in complexity between 22 and 2048 bits is not linear, but exponential; quantum computers vastly more powerful than current ones would be needed to handle real-world keys.

- Methodology: The approach used (quantum annealing) transforms the attack into an optimization problem, but it has not yet been demonstrated that this method can scale to affect modern security systems.

Quantum Computing vs. Encryption

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Barış Ege

Key-size:

22 vs. 2048 bits

#Quantum Computer

#Quantum Computing

NETWORK VISIBILITY + SECURITY

Security Highlight: China's Quantum Leap, and Why RSA Isn't at Risk (Yet)

2024-10-28 | 4 min read

Recent headlines suggested that Chinese scientists have the potential to break military-grade RSA encryption using quantum computers, causing concerns over online data security. However, upon closer inspection, these claims do not yet signal the end of modern encryption. Here's what you need to know.

Understanding the breakthrough

The Chinese Journal of Computers recently published a paper in which researchers from Shanghai University factored a 22-bit integer using D-Wave's um annealing machine. The research showcases how quantum annealing ged to transform cryptographic attacks into solvable combinatorial optimization p lems.

The team factored a 22-bit number, a far cry from the 2048-bit keys currently used for secure encryption. The difference between a 22-bit and a 2048-bit number is exponential, and this study, while a step forward, doesn't yet threaten RSA encryption as we know it.

Quantum Computing vs. Encryption

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

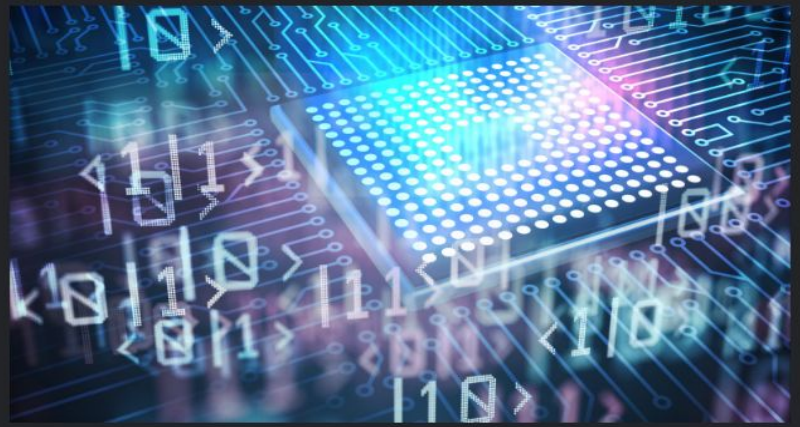
ars TECHNICA AI BIZ & IT CARS CULTURE GAMING HEALTH POLICY SCIENCE SECURITY SPACE TECH FORUM SUBSCRIBE

CRYPTOGRAPHICALLY RELEVANT QUANTUM COMPUTING

Quantum computers need vastly fewer resources than thought to break vital encryption

No, the sky isn't falling, but Q Day *is* coming, and it won't be as expensive as thought.

DAN GOODIN – MAR 31, 2026 8:25 PM | 39



Credit: vital

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

How It Works (Step-by-Step)
1. Encryption Phase (The Sender)
- The sender generates a temporary symmetric key and uses it to encrypt the message (even one as large as 1GB).
- The sender takes the recipient's public key and uses it to encrypt only the small temporary symmetric key (a different one for each message).
- The final "package" (encrypted message + encrypted key) is the Digital Envelope.

2. Decryption Phase (The Recipient)
- The recipient receives the envelope.
- They use their own private key to decrypt and retrieve the symmetric key.
- They use the newly retrieved symmetric key to decrypt the actual message.

Digital Envelopes

The Digital Envelope is the final solution to the key distribution problem we discussed earlier. It is a hybrid technique that combines the best of both worlds:
- Symmetric encryption (AES): It is extremely fast, but it has the problem of how to exchange the key.
- Public-key encryption (RSA/ECC): It solves the exchange problem, but it is very slow when encrypting large files (videos, large documents).

Digital Envelopes

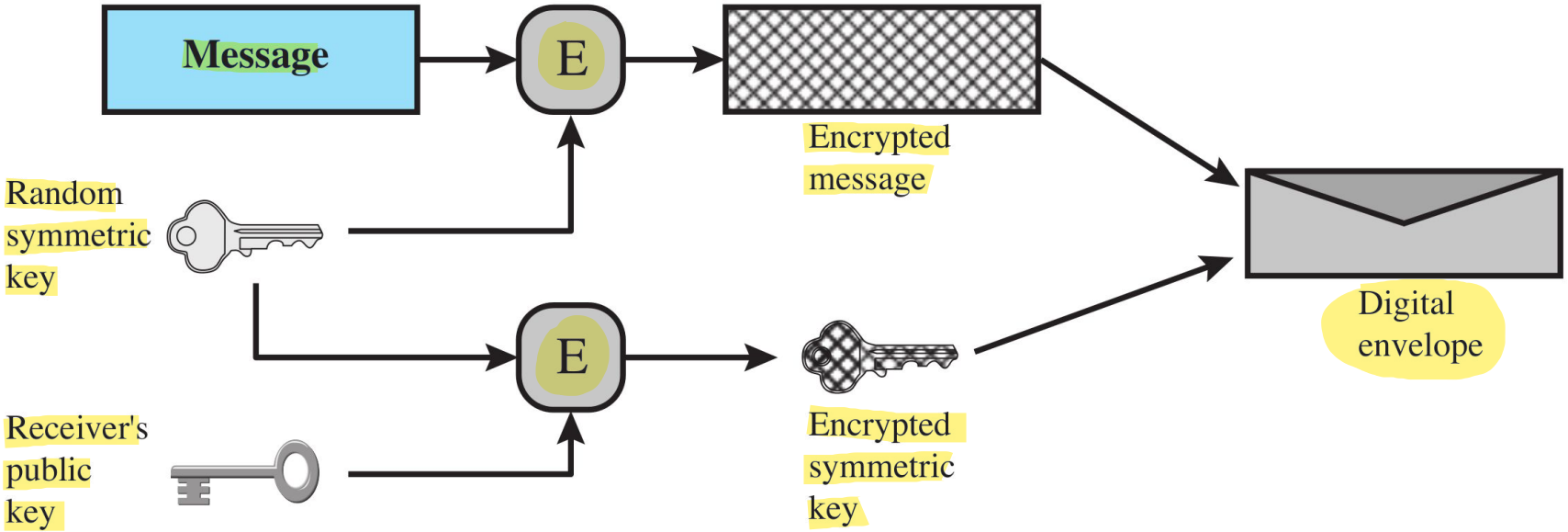
- They are used to **protect a message** without ~~needing to arrange~~ the sender and recipient having to agree in advance on a shared secret key ~~beforehand, for sender and receiver, to have the **same secret key**~~

- ~~It equates~~ It is equivalent to the same thing as a **sealed envelope** locked containing an **unsigned letter**

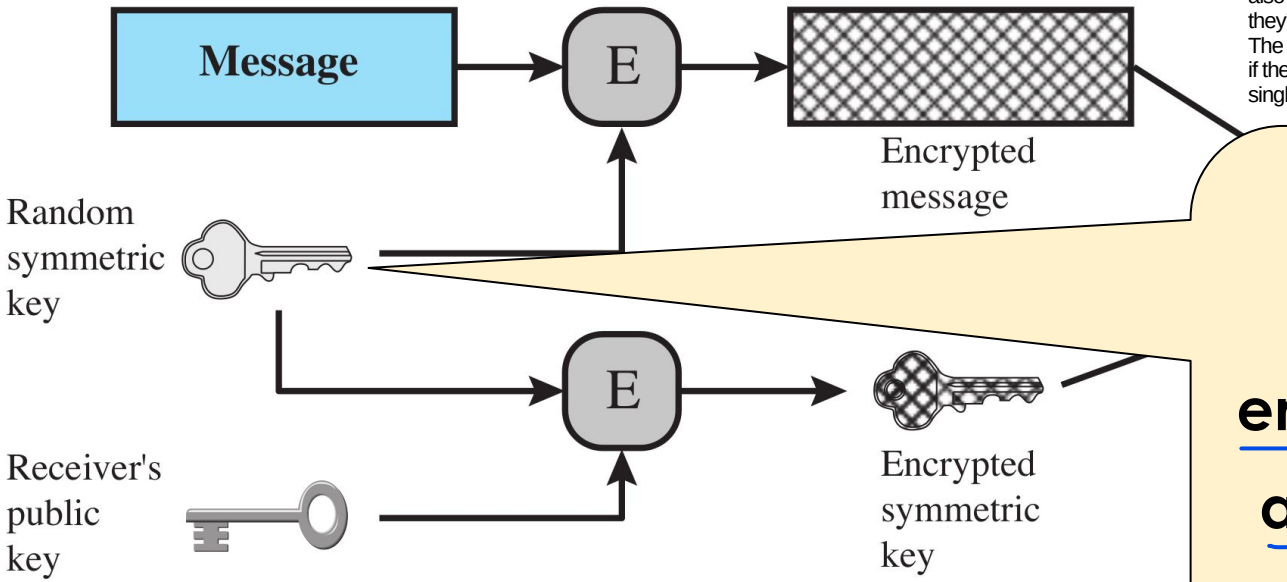
1. "Sealed envelope" = Confidentiality
Public-key encryption ensures that only you can open that envelope. In technical terms, this means that the sender used your Public Key to “lock” the message. Since only you possess the corresponding Private Key, you are the only person in the world who can read the contents. Result: No one has read the message along the way.
2. "Unsigned letter" = Lack of Authenticity
Once you open the envelope, you find a computer-printed sheet of paper, with no signature and no stamp. The problem: Anyone could have put that sheet of paper in the envelope. Even if the envelope was correctly addressed to you, you have no mathematical proof of the sender’s identity. It could have been a friend of yours, but it could also have been an imposter who simply found your public key and sent you a message pretending to be someone else.

- They have some interesting characteristics
  - Maximum Efficiency: You don’t have to encrypt the entire file using the public-key encryption algorithm; with it, you only encrypt a few bits of the key.
  - No Prior Agreement: You and I don’t need to meet beforehand. I just need to find your public key, and I can send you a digital envelope immediately.
  - One-Time: Every time I send a message, I generate a different symmetric key. If a hacker manages to steal the key for one message, the others remain secure (Forward Secrecy).

Digital Envelopes: creation



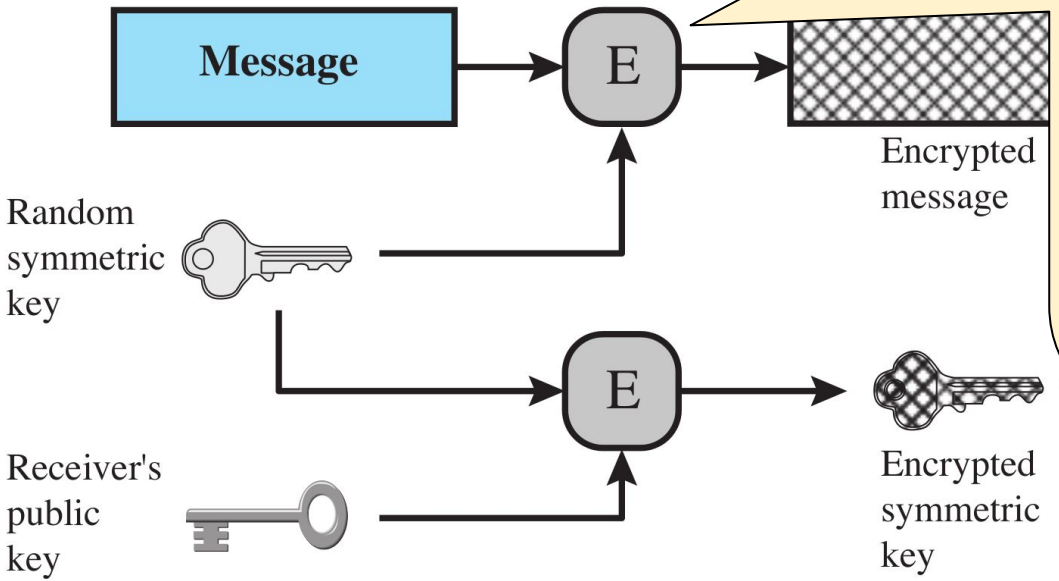
Digital Envelopes: creation



Imagine using the same symmetric key for all the messages you send me over the year.
The risk: If a hacker manages to steal that single key today, they could decrypt not only today's messages, but also all the ones you've sent over the past 12 months that they've intercepted and saved.
The protection: By using a different key for each envelope, if the hacker discovers one, they'll only be able to read that single message. All the others remain secure.

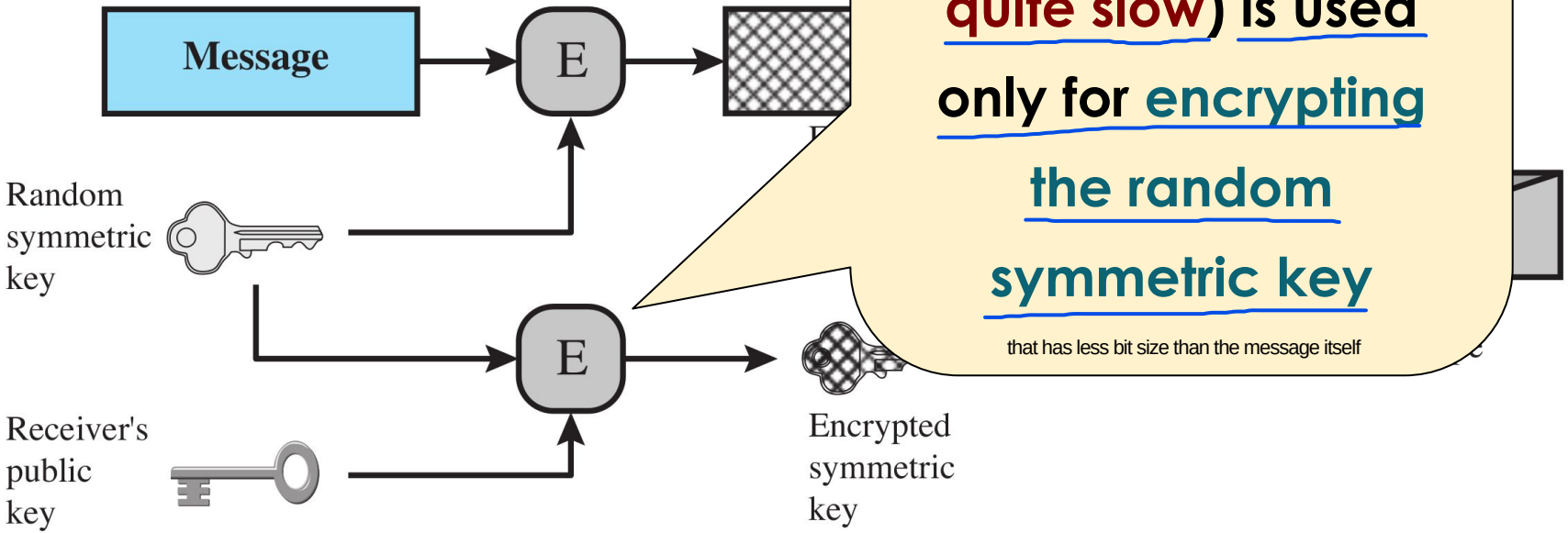
Every digital envelope is encrypted using a new random key

Digital Envelopes: creation



The digital envelope is encrypted using a symmetric algorithm that is much faster than public key encryption

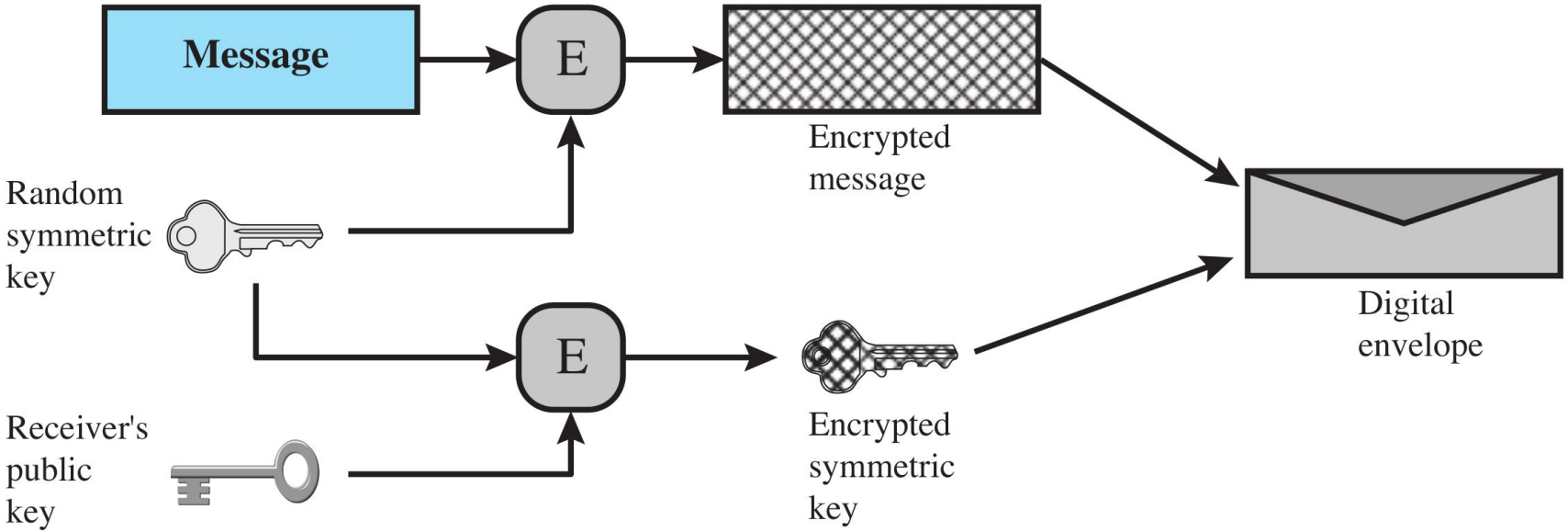
Digital Envelopes: creatio



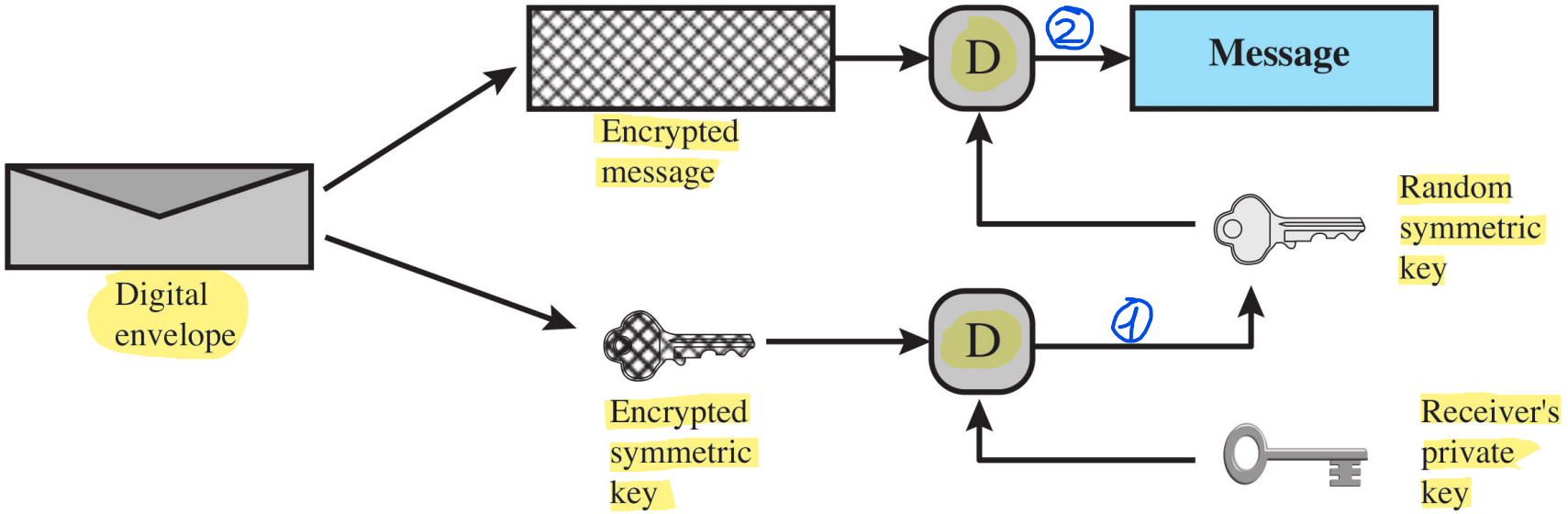
The public key encryption (that is quite slow) is used only for encrypting the random symmetric key

that has less bit size than the message itself

Digital Envelopes: creation



Digital Envelopes: opening



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Random Numbers

Random numbers (generation)

- **Random numbers** are required by many different tasks:
  - A ○ **generation of keys** for **public-key algorithms** → To create an RSA key pair, the computer must select two very large prime numbers. If these numbers are not truly random, an attacker could regenerate your private key.
  - B ○ **stream key** for **symmetric stream cipher** → Stream ciphers (such as ChaCha20) transform a random number (stream key) into an infinite string of bits to encrypt data.
  - C ○ **symmetric key** to use as a **temporary session key** or to create a **digital envelope**

As we have seen, every digital envelope requires a new symmetric key. This key must be unpredictable to prevent anyone from intercepting the envelope and opening it.

  - D ○ **handshaking** to **prevent replay attacks** → During the initial handshake, the computers exchange random numbers known as nonces (numbers used once). These cryptographic nonces guarantee session freshness and ensure that every connection is entirely unique.
  - E ○ **session key** (i.e., randomly generated symmetric key)

Random numbers (generation)

- **Random numbers** are required by many different tasks:
  - **generation of keys** for **public-k**
  - **stream key** for **symmetric strea**
  - **symmetric key** to use as a **tem** **digital envelope**
  - **handshaking** to **prevent replay attacks**
  - **session key** (i.e., randomly generated symmetric key)

An adversary is repeating a previously captured user response

Example: If a hacker records your login process and tries to replay it (replay attack), the server will reject it because the random session number from the previous session has already expired.

Random numbers (requirements)

The quality of the generated sequence of random numbers is evaluated to determine whether the generator is valid (i.e. truly random).

Randomness criteria:

1. **uniform distribution**: **frequency of occurrence** of each of the numbers should be **approximately the same**
2. **independence**: **no one value** in the sequence can be **inferred from the others**

Random numbers (requirements)

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Randomness criteria:**
  - **uniform distribution**: **frequency of occurrence** of each of the numbers should be **approximately the same**
  - **independence**: **no one value** in the sequence can be **inferred from the others**

Unpredictability

Random numbers (requirements)

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

1

Each number is statistically independent of other numbers in the sequence

quency

roxima

value i

2

Opponent should not be able to predict future elements of the sequence on the basis of earlier elements

Unpredictability

The property of unpredictability is attributed both to the generated sequence of numbers and, consequently, to the generator itself

Randomness vs. Pseudo-randomness

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Cryptographic applications** typically use **algorithms** for **random number generation**: algorithms are **deterministic** and therefore produce sequences of numbers that are **not statistically random**
- **Pseudo-random numbers** are:
  - sequences produced that **satisfy statistical randomness tests**
  - likely to be **predictable** →

Even though pseudo-random numbers pass statistical randomness tests, they are generated by deterministic mathematical algorithms. This implies that:
- The Seed Dependency: Generation relies entirely on an initial value called a seed. Knowledge of both the algorithm and the seed allows an observer to reproduce the exact same sequence of numbers.
- Determinism over Chaos: True randomness is absent. The sequence behaves like a pre-written text: while it may appear chaotic to an outsider, anyone with knowledge of the internal state can precisely predict the next output.
- Cryptographic Security Risks: If an attacker discovers or guesses the seed—for instance, when it is weakly generated using only the system time—they can predict all subsequent cryptographic keys, completely compromising the system.

Randomness vs. Pseudo-randomness

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **True Random Number Generator** (**TRNG**):
  - uses a **nondeterministic source** to produce **randomness**
  - most operate by measuring **unpredictable natural processes**

    (e.g., radiation, gas discharge, leaky capacitors)
  - **increasingly provided on modern processors**

In the past, achieving "true" randomness required expensive external hardware. Today, manufacturers like Intel and AMD integrate a small TRNG circuit directly into the CPU. Example: The RDRAND instruction in Intel processors queries a hardware circuit that uses thermal noise to generate extremely high-quality random bits at a very high speed.

Randomness vs. Pseudo-randomness

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **True Random Number Generator** (**TRNG**):
  - uses a **nondeterministic source** to produce **randomness**
  - most operate by measuring **unpredictable natural processes** (e.g., radiation, gas discharg
  - **increasingly provided on modern processors**

Backdoors?

When a hardware manufacturer (such as Intel or AMD) embeds a “black-box” random number generator directly into the processor, the cryptography community immediately faces a trust issue.

Mitigation: Never Trust a Single Source
Instead of using the processor's random number as-is, the operating system mixes it with other sources:
- It takes the random number generated by the processor.
- It mixes them with hard drive noise, mouse movements, and network packet timings.
- It passes everything through a hash function (such as SHA-256).
Result: Even if the processor were compromised and produced predictable numbers, the addition of other sources of chaos would disrupt the backdoor's pattern, making the final output secure again.

Randomness vs. Ps

- **True Random Number G**
  - uses a **nondetermini**
  - most operate by me

    (e.g., radiation, gas discharge, apacitors)
  - **increasingly provided on modern processors**

In cybersecurity, nothing is ever completely "secure," but everything depends on your threat model. Claiming that there is a backdoor in your processor's hardware random number generator (TRNG) is a valid hypothesis, but the question you need to ask yourself is: who is my adversary?

Backdoors? It is possible but carefully consider your **threat model**

Do you **trust your CPU?**

- ○ increasingly provided on **modern processors**

The phrase "consider your threat model" means that there is no one-size-fits-all solution. For 99% of people, a hardware TRNG is a technological miracle that makes the internet safer. For the remaining 1%, it's a potential Trojan horse.

Randomness (NOT MANDATORY)

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- If you are interested in **how random numbers generation is managed** in the new versions of the **Linux kernel**: "Uniting the Linux random-number devices"
- If you are interested in a (potential) **backdoor in an encryption standard** (**not related to random numbers generation**): "Dual Elliptic Curve Deterministic Random Bit Generator"

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

CyberSecurity

Gabriele D'Angelo

<g.dangelo@unibo.it>

https://www.unibo.it/sitoweb/g.dangelo/en

Attributions and Notes

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Most of the slides are from: "**Computer Security: Principles and Practice**", 4th edition, W. Stallings and Lawrie Brown
- Some others are from: Dr. **Mehmet H. Gunes** (Department of Computer Science & Engineering - University of Nevada, Reno)
- Some others are from: **Mario Cagalj**, Ph.D. (Faculty of Electrical Engineering, Mechanical Engineering and Naval Architecture (FESB), University of Split)
- I wish to **thank all of them**, **any errors that remain are my sole responsibility**